

TEMPLATE-BASED AND OPTIMAL CONTROL OF DYNAMIC
QUADRUPEDAL GAITS

A THESIS SUBMITTED TO
THE GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES
OF
MIDDLE EAST TECHNICAL UNIVERSITY

BY

MUSTAFA AKBABA

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR
THE DEGREE OF MASTER OF SCIENCE
IN
ELECTRICAL AND ELECTRONICS ENGINEERING

NOVEMBER 2025

Approval of the thesis:

**TEMPLATE-BASED AND OPTIMAL CONTROL OF DYNAMIC
QUADRUPEDAL GAITS**

submitted by **MUSTAFA AKBABA** in partial fulfillment of the requirements for the degree of **Master of Science in Electrical and Electronics Engineering Department, Middle East Technical University** by,

Prof. Dr. Naci Emre Altun
Dean, Graduate School of **Natural and Applied Sciences** _____

Prof. Dr. Nevzat Güneri Gençer
Head of Department, **Electrical and Electronics Engineering** _____

Assoc. Prof. Dr. Mustafa Mert Ankaralı
Supervisor, **Electrical and Electronics Engineering, METU** _____

Assoc. Prof. Dr. Ali Emre Turgut
Co-supervisor, **Mechanical Engineering, METU** _____

Examining Committee Members:

Prof. Dr. Afşar Saranlı
Electrical and Electronics Engineering, METU _____

Assoc. Prof. Dr. Mustafa Mert Ankaralı
Electrical and Electronics Engineering, METU _____

Assoc. Prof. Dr. İsmail Uyanık
Electrical and Electronics Engineering, Hacettepe University _____

Date: 26.11.2025

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

Name, Surname: Mustafa Akbaba

Signature :

ABSTRACT

TEMPLATE-BASED AND OPTIMAL CONTROL OF DYNAMIC QUADRUPEDAL GAITS

Akbaba, Mustafa

M.S., Department of Electrical and Electronics Engineering

Supervisor: Assoc. Prof. Dr. Mustafa Mert Ankaralı

Co-Supervisor: Assoc. Prof. Dr. Ali Emre Turgut

November 2025, 110 pages

This work presents a unified, template-based control framework for dynamic quadrupedal locomotion that covers trotting, pronking, and landing. First, we extend the clock-torque actuated spring-loaded inverted pendulum (CT-SLIP) template to 3D. This single template enables walking trot, regular trot, and flying (running) trot. The controller adds a projectile motion model for flight phases and a yaw control that rotates the swing foot plane for better turning stability. A whole-body controller (WBC) is integrated to map template forces to the full robot. The approach is tested extensively in MuJoCo simulation and on the Unitree GO1 robot, showing smooth gait transitions, strong disturbance rejection, and real-world performance. Second, we develop a robust pronking controller by combining the same template dynamics with predictive control. A linear time-varying MPC tracks reference trajectories generated by the template, while a nonlinear MPC (NMPC) plans footsteps using analytical template solutions. The method is implemented in simulation and on hardware. Convergence analysis and disturbance tests confirm stable and adaptable pronking from various initial conditions. Finally, we propose an optimization-based foot placement strategy

for safe landing on flat ground. The stance phase uses a spring-damper template. An NMPC minimizes the error between template-predicted moments and actual ground reaction forces at touchdown, while respecting friction constraints. Simulation and hardware experiments demonstrate successful landings at horizontal velocities up to 5 m/s with higher stability than previous methods. Overall, combining simple template models with modern predictive control yields robust, versatile, and high-performance locomotion across multiple gaits and tasks.

Keywords: Template-based control, legged locomotion, optimal control, quadrupedal robotics, spring loaded inverted pendulum (SLIP)

ÖZ

DİNAMİK DÖRT BACAĞLI YÜRÜYÜŞLERİN ŞABLON TABANLI VE OPTİMAL KONTROLÜ

Akbaba, Mustafa

Yüksek Lisans, Elektrik ve Elektronik Mühendisliği Bölümü

Tez Yöneticisi: Doç. Dr. Mustafa Mert Ankaralı

Ortak Tez Yöneticisi: Doç. Dr. Ali Emre Turgut

Kasım 2025 , 110 sayfa

Bu çalışma, trotting, pronking ve iniş davranışlarını kapsayan dinamik dört ayaklı lokomasyon için birleşik ve şablon tabanlı bir kontrol çerçevesi sunar. İlk olarak, saat torklu yaylı ters sarkaç (CT-SLIP) şablonu 3D'ye genişletilmiştir. Tek bir şablonla yürüme trot, normal trot ve uçuşlu (koşu) trot gerçekleştirilebilmektedir. Kontrolcü, uçuş fazları için parabolik hareket modeli ve daha iyi dönüş kararlılığı için salınım ayağını döndüren yaw kontrolü içermektedir. Şablon kuvvetlerini tam robota aktarmak için bütüncül vücut kontrolcüsü (WBC) entegre edilmiştir. Yöntem MuJoCo simülasyonunda kapsamlıca test edilmiş ve Unitree GO1 robotunda doğrulanmıştır; akıcı gait geçişleri, güçlü bozucu reddi ve gerçek dünya performansı sergilemiştir. İkinci olarak, aynı şablon dinamiklerini öngörücü kontrolle birleştiren sağlam bir pronking kontrolcüsü geliştirilmiştir. Zamana bağlı doğrusal MPC, şablonun ürettiği referans yörüngeleri takip ederken, doğrusal olmayan MPC (NMPC) analitik şablon çözümleriyle adım planlaması yapar. Yöntem simülasyon ve gerçek donanımda uygulanmış, yakınsama analizi ve bozucu testleriyle değerlendirilmiştir. Farklı başlangıç koşullarından

kararlı ve uyarlanabilir pronking elde edilmiştir. Son olarak, düz zeminde güvenli iniş için optimizasyon tabanlı ayak yerleştirme stratejisi önerilmiştir. Yere basma fazı yay-sönümleyici şablonla modellenmiş, temas anında şablon momentleriyle gerçek yer tepki momentleri arasındaki hatayı en aza indiren ve sürtünme kısıtlarını dikkate alan bir NMPC kullanılmıştır. Simülasyon ve deneyler, 5 m/s'ye kadar yatay hızlarda başarılı iniş ve önceki yöntemlere üstün kararlılık göstermiştir. Sonuç olarak, basit şablon modelleriyle modern öngörücü kontrolün birleşimi, çok çeşitli gait ve görevlerde sağlam, esnek ve yüksek performanslı dört ayaklı hareket sağlamaktadır.

Anahtar Kelimeler: Şablon tabanlı kontrol, bacaklı hareket, optimal kontrol, 4 bacaklı robotlar, yay yüklü ters sarkaç modeli

To my family, and to my beloved partner.

ACKNOWLEDGMENTS

First and foremost, I would like to express my sincere gratitude to my supervisor, Assoc. Prof. Mustafa Mert Ankaralı, for his invaluable guidance, support, and encouragement throughout my graduate studies. His mentorship has shaped not only the technical direction of this thesis but also my overall approach to research. I am also deeply thankful to my co-supervisor, Assoc. Prof. Ali Emre Turgut, whose positive attitude and continuous support have been instrumental in the development of this work.

I would also like to extend my appreciation to Prof. Dr. Uluç Saranlı for his generous feedback, technical advice, and the inspiration he has provided through his work in the field of legged robotics.

I am grateful to the Center for Robotics and AI (ROMER) at METU for offering a collaborative and well-equipped research environment. I would also like to thank my colleagues and friends at ROMER for their valuable support and encouragement throughout this work. My special thanks go to Türk Telekom for their support during this study. Additionally, I acknowledge the financial support provided by the Scientific and Technological Research Council of Turkey (TÜBİTAK), which made this work possible through its scholarship and funding programs. This work was supported by TÜBİTAK under the 2210-A Scholarship Program. Finally, I would like to express my sincere gratitude to my friends Özgür Gülsuna, Haktan Yalçın, Hasan Doğan and Ertürk Karagönen from the Department of Electrical and Electronics Engineering for their help and motivation.

Last but certainly not least, I would like to thank my family for their endless love, patience, and belief in me, even during the most challenging times. Their unwavering support has been the foundation of all my achievements. Finally, I am profoundly grateful to my partner, Betül Saplı, for her constant encouragement, understanding, and presence throughout this journey.

TABLE OF CONTENTS

ABSTRACT	v
ÖZ	vii
ACKNOWLEDGMENTS	x
TABLE OF CONTENTS	xi
LIST OF TABLES	xiv
LIST OF FIGURES	xv
LIST OF ABBREVIATIONS	xxii
CHAPTERS	
1 INTRODUCTION	1
1.1 Motivation and Background	1
1.2 Contributions	6
1.3 The Outline of the Thesis	8
2 A COMPREHENSIVE CLOCK TORQUE ACTUATED SLIP FRAME- WORK FOR TROTTING IN QUADRUPEDS	11
2.1 Template Model	11
2.2 Controller Model	16
2.3 Trotting Gaits: Walking to Running	20
2.4 Turning in the Yaw Direction	22

2.5	Whole-Body Controller	24
2.6	State Estimation	26
2.7	Experimental Setup	28
2.8	Simulation Results	31
2.8.1	Gait Transitions and Disturbance Rejection	31
2.8.2	Turning with and without compensation	33
2.8.3	Template-based Control vs. Whole-body Control Improved Template-based Control	34
2.9	Hardware Experiments	35
2.9.1	Comparison with Simulation	38
3	TEMPLATE-BASED QUADRUPEDAL PRONKING: ANALYSIS AND EXTENSION WITH MODEL PREDICTIVE CONTROL	47
3.1	Spring-Damper Template	47
3.2	Motion Generation From Template	48
3.3	Controller Design	49
3.4	Approximate Analytical Foot Step Controller	55
3.4.1	Formulating The Model	55
3.4.2	Solving For The Optimal Foot Location	57
3.5	Proposed Approach	59
3.6	Simulation Analysis	61
3.6.1	Results From Simulation	63
3.7	Hardware Experiments	65
4	AN OPTIMIZATION-DRIVEN FOOT PLACEMENT CONTROLLER FOR QUADRUPED LANDING	77

4.1	Template Model and Stance Controller	77
4.2	Foot Placement Controller	80
4.3	System-Level Control Architecture	85
4.4	Experimental Results	87
4.4.1	Simulation Experiments	88
4.4.2	Real Robot Experiments	90
5	CONCLUSIONS	97
	REFERENCES	99
A	EQUATIONS OF MOTION OF A QUADRUPEDAL ROBOT	107

LIST OF TABLES

TABLES

Table 2.1	Inertial and mass parameters of the quadruped robot used in simulation experiments	30
Table 2.2	Parameters for the controller.	36
Table 2.3	RMSE comparison between real robot, simulation, and reference trajectories.	39
Table 2.4	RMSE of stance duration deviations for each leg compared to the reference gait timing.	39
Table 3.1	Parameters for the template.	62
Table 4.1	Template parameters.	88

LIST OF FIGURES

FIGURES

- Figure 2.1 Spring-loaded inverted pendulum (SLIP) model. A point-mass body and a massless spring leg of length l are shown during stance, with the leg contacting the ground at angle Γ . The dashed curve represents the center-of-mass trajectory. 12
- Figure 2.2 Two-dimensional CT-SLIP template. (a) Point-mass CT-SLIP model, (b) CT-SLIP extended with a trunk and hip torques τ_1 and τ_2 , (c) corresponding anchor representation for the quadruped. 12
- Figure 2.3 Reference virtual leg angles. The reference angular trajectories for the two virtual leg pairs are shown, each sweeping an angle of Γ_s over the stance duration t_s 14
- Figure 2.4 Torque-actuated SLIP model. A SLIP leg with spring–damper compliance and hip torque τ acting on a point mass m is illustrated, with leg angle Γ and length l defined relative to the ground. 15
- Figure 2.5 Three-dimensional CT-SLIP template. The 3D extension of the CT-SLIP model is shown, where compliant virtual legs act in all translational and rotational directions to generate body forces and moments. . . 16
- Figure 2.6 Virtual leg–real leg relation. The virtual leg of the template is mapped to the corresponding real leg by using the hip–foot configuration, and its force is applied to the body’s center of mass. 17
- Figure 2.7 Motion of the swing leg. The planned swing trajectory is defined by initial, mid, and final foot positions (p_0, p_1, p_2) with corresponding velocities, reaching a vertical height $z_{\max}$ and a horizontal span $x_{\max}$. . . 19

Figure 2.8	Block diagram of the controller. The diagram illustrates the clock-driven phase selection, template-based body force generation, swing-leg motion planning, and the computation of joint torques.	20
Figure 2.9	Contact legs for trotting. Contact sequences for the four legs are shown for (a) walking trot, (b) trot, and (c) flying trot, highlighting the stance durations associated with different gait types.	21
Figure 2.10	Compression and de-compression phases of the SLIP model. The stance phase is illustrated with the leg spring compressing and subsequently de-compressing.	22
Figure 2.11	Swing foot planes. (a) Foot placement without yaw compensation, (b) foot placement with yaw-compensated swing plane adjustment.	23
Figure 2.12	Swing foot planes <i>a)</i> Foot placement with no compensation, <i>b)</i> foot placement with compensation	23
Figure 2.13	Block diagram of the controller with WBC. The whole-body controller is added after the template and controller models to refine the generated ground reaction forces.	26
Figure 2.14	GO1 robot in the MuJoCo simulator. Snapshots from the simulated trotting motion.	29
Figure 2.15	Clock parameters and corresponding reference body velocity. (a) Angle swept by the virtual leg Γ_s , (b) stance time t_s , (c) resulting reference forward velocity $\dot{x}_r$, for a cycle period $t_c = 0.5$ s.	32
Figure 2.16	States vs time. (a) Horizontal velocity, (b) body height, and (c) pitch angle during three gait phases: walking trot (0–10 s), trot (10–20 s), and running trot (20–30 s).	33

Figure 2.17	Disturbance rejection performance under different gaits, <i>a</i>) horizontal velocity of the robot vs time (walking trot), <i>b</i>) y position of the robot vs time (walking trot), <i>c</i>) y direction velocity of the robot vs time ((walking trot), <i>d</i>) horizontal velocity of the robot vs time (trot), <i>e</i>) y position of the robot vs time (trot), <i>f</i>) y direction velocity of the robot vs time (trot), <i>g</i>) horizontal velocity of the robot vs time (running trot), <i>h</i>) y position of the robot vs time (running trot), <i>i</i>) y direction velocity of the robot vs time (running trot)	40
Figure 2.18	Comparison between compensation and no-compensation cases. (a) Horizontal velocity in the body frame, (b) height of the robot, (c) yaw angle, showing the stabilizing effect of the compensation method. .	41
Figure 2.19	Comparison between TBC and TBC+WBC based on tracking performance: <i>a</i>) horizontal velocity vs time, <i>b</i>) y position vs time, <i>c</i>) z position vs time, <i>d</i>) Damping constant for the torque of the virtual leg vs time.	42
Figure 2.20	Comparison between TBC and TBC+WBC based on disturbance rejection <i>a</i>) horizontal velocity vs time <i>b</i>) robot's body height vs time, <i>c</i>) robot's y distance vs time, <i>d</i>) robot's y velocity vs time . . .	43
Figure 2.21	Hardwares for the experiments: <i>a</i>) Hardwares on the Unitree GO1 robot, 1) Raspberry Pi 4, 2) 12 V fan, 3) Pixhawk 4, 4) Powerbank, <i>b</i>) Robot Standing	43
Figure 2.22	States of the robot. (a) Horizontal position, (b) horizontal velocity, (c) vertical position, (d) vertical velocity over three gait phases: walking trot (0–10 s), trot (10–20 s), and running trot (20–30 s).	44
Figure 2.23	States of the robot. (a) Roll angle, (b) pitch angle, (c) yaw angle, (d) lateral position, over three gait phases: walking trot (0–10 s), trot (10–20 s), and running trot (20–30 s).	45
Figure 2.24	Walking trot hardware experiment. Snapshots from the robot performing a walking trot on hardware.	45

Figure 2.25	Comparison of state trajectories between the real robot and simulation over a 10-second locomotion trial. (a) Forward velocity $\dot{x}$, (b) vertical position z , and (c) pitch angle ϕ	46
Figure 3.1	Template for stance motion. The stance-phase template shows the force elements used to generate desired body forces, along with the reference inputs x_{ref} and $v_{x,\text{ref}}$ that regulate forward motion.	48
Figure 3.2	Simplified system for the controller. The body is modeled with four contact points, each applying 3D ground reaction forces F_{ix}, F_{iy}, F_{iz} , which are used by the controller to regulate the robot’s motion.	51
Figure 3.3	Apex-to-apex behavior of the robot. The figure illustrates one full locomotion cycle divided into three phases: Apex to Touch-Down, Touch-Down to Lift-Off, and Lift-Off to Apex, showing the robot’s body motion and foot contact transitions across the cycle.	56
Figure 3.4	Model used for the approximate analytical footstep controller. The simplified planar model shows body orientation, leg geometry, and ground reaction forces used to compute the next foot placement.	58
Figure 3.5	The proposed controller architecture. The diagram summarizes the stance and flight control pipelines, including the template model, MPC-based stance controller, NMPC footstep planner, inverse kinematics, and PD control, together with state and contact estimation modules.	60
Figure 3.6	Unitree GO1 Robot in MuJoCo Simulation Environment	61
Figure 3.7	Effect of adaptive pitch reference on locomotion performance. Comparison of robot pitch angle and forward velocity with and without the adaptive pitch reference at different desired velocities: (a) pitch angle at $\dot{x}_{des} = 0.5 \text{ m/s}$, (b) forward velocity at $\dot{x}_{des} = 0.5 \text{ m/s}$, (c) pitch angle at $\dot{x}_{des} = 1 \text{ m/s}$, (d) forward velocity at $\dot{x}_{des} = 1 \text{ m/s}$, (e) pitch angle at $\dot{x}_{des} = 1.5 \text{ m/s}$, (f) forward velocity at $\dot{x}_{des} = 1.5 \text{ m/s}$. The adaptive pitch reference improves stability and reduces pitch oscillations across all speeds.	68

Figure 3.8 Comparison of state-tracking performance between the Raibert footstep planner and the proposed approximate analytical planner. (a) Horizontal velocity $\dot{x}$, (b) height z , (c) pitch angle ϕ , (d) roll angle ψ during a 10-second locomotion trial. 69

Figure 3.9 Robustness comparison between the Raibert controller and the approximate analytical foot-step planner. State responses under a disturbance: (a) horizontal velocity $\dot{x}$, (b) height z , (c) pitch angle ϕ , (d) roll angle ψ . The analytical planner maintains stability, whereas the Raibert controller diverges under the same conditions. 70

Figure 3.10 State tracking performance of the proposed controller. *a)* Horizontal Velocity $\dot{x}$ of the robot, *b)* height of the robot z (dashed line represents reference apex height), *c)* pitch angle of the robot ϕ 71

Figure 3.11 Comparison of velocity tracking performance for Template-Based Control (TBC), LTI-MPC, and LTV-MPC. (a) Horizontal velocity $\dot{x}$, (b) height z (dashed line: reference apex height), (c) pitch angle ϕ . TBC does not include future prediction, LTI-MPC incorporates future prediction with fixed Jacobians, and LTV-MPC includes both future prediction and online Jacobian updates. 72

Figure 3.12 Recovery from an external push: comparison of TBC, LTV-MPC, and LTI-MPC. The dashed line indicates the push instant. Subplots show (a) horizontal velocity $\dot{x}$, (b) height z , (c) lateral position y , and (d) lateral velocity $\dot{y}$, highlighting the difference in disturbance-rejection performance among the controllers. 73

Figure 3.13 Stability region and convergence behavior. The green area represents the set of initial apex states $(\dot{x}, z)$ that converge to a stable limit cycle, while the dashed lines illustrate example convergence trajectories from different initial conditions. 74

Figure 3.14	Sequential frames from a hardware experiment illustrating the pronking behavior of the Unitree GO1 quadruped robot. The motion involves a synchronized vertical push-off of all four legs, followed by an aerial phase and subsequent touchdown.	74
Figure 3.15	States of the hardware experiment using Unitree GO1 robot. <i>a)</i> horizontal position, <i>b)</i> horizontal velocity, <i>c)</i> vertical position, <i>d)</i> vertical velocity, <i>e)</i> pitch angle, <i>f)</i> pitch velocity	75
Figure 4.1	Template for stance motion. During stance, the body is regulated by a six-dimensional virtual wrench consisting of three linear and three angular virtual forces applied at the center of mass, illustrated through conceptual spring–damper elements.	78
Figure 4.2	Flowchart of the foot placement controller. For each candidate foot location, the controller evaluates a cost based on the mismatch between template-generated and GRF-induced moments. The optimal location is selected through iterative optimization.	80
Figure 4.3	Friction cone projection. The original tangential force $\mathbf{F}_{\text{leg}}$ is adjusted to its projection $\mathbf{F}_{\text{leg}}^{\text{updated}}$ on the boundary of the friction circle defined by μF_z , preserving the vertical force component.	83
Figure 4.4	Foot locations relative to the robot body. The foot position is defined by the components x_f , y_f , and z_f expressed in the body frame, illustrating the geometric relationship between the robot’s base and the foot placement.	84
Figure 4.5	Overview of the hybrid controller. The controller switches between stance and flight modes based on the robot’s state. In the stance phase, a template model and QP optimization generate desired body forces and joint torques. In the flight phase, the foot placement controller determines the optimal foot location, inverse kinematics updates the reference joint angles, and a PD controller computes the resulting joint torques.	86

Figure 4.6	Unitree GO1 robot: (a) real robot, (b) MuJoCo simulation environment.	87
Figure 4.7	Polar plot of successful landing conditions across different drop heights. Radial distance indicates the maximum achievable throwing velocity (m/s), and angular position corresponds to the throwing direction (degrees). Solid lines represent the proposed controller, while dashed lines correspond to a baseline method [45].	90
Figure 4.8	Time evolution of the robot's states during landing for different drop heights (1.00, 0.80, 0.50, and 0.35 m) at a 0° throwing direction, with each trajectory executed at the maximum velocity that results in a stable landing. Subplots show the vertical position (a), forward velocity (b), lateral velocity (c), and vertical velocity (d) of the center of mass. Green markers indicate the estimated touchdown moment for each trajectory.	93
Figure 4.9	Landing states of the GO1 robot. (a) Horizontal position, (b) lateral position, (c) vertical position during the landing maneuver. The red marker denotes the throwing moment, and the green marker indicates the touchdown event.	94
Figure 4.10	Comparison of landing behavior between the GO1 robot and simulation. (a) Horizontal position, (b) lateral position, (c) vertical position during landing, showing the agreement between real-robot measurements and the simulated model.	95
Figure A.1	Robot body parameters. The geometric properties of the robot body are shown, including the center of mass (COM), body dimensions l_1 and l_2 , and hip locations given by a_1 , a_2 , and b_1 . The body-fixed coordinate frame is also illustrated.	109

LIST OF ABBREVIATIONS

COM	Center Of Mass
2D	2 Dimensional
3D	3 Dimensional
TD	Touch-Down
LO	Lift Off
PD	Proportional-Derrivative
SLIP	Spring-Loaded Inverted Pendulum
CT-SLIP	Clock Torque-Actuated Spring-Loaded Inverted Pendulum
MPC	Model Predictive Controller
NMPC	Nonlinear Model Predictive Controller
TBC	Template-Based Control
WBC	Whole-Body Controller
LTI	Linear Time Invariant
LTV	Linear Time Varying
GRF	Ground Reaction Forces
DOF	Degree Of Freedom

CHAPTER 1

INTRODUCTION

1.1 Motivation and Background

Legged robots have emerged as a powerful solution for navigating environments that are inaccessible or impractical for wheeled or tracked systems. Their ability to replicate the agility and mobility of animals enables them to traverse uneven terrain, climb obstacles, and maintain stability on complex surfaces—capabilities that are critical for applications such as search and rescue, planetary exploration, and autonomous delivery. This versatility has attracted significant attention from the robotics community, highlighting the potential of legged systems in real-world, unstructured environments. However, with this adaptability comes a set of complex control challenges. The dynamic nature of legged locomotion demands advanced algorithms to maintain balance, handle external disturbances, and ensure smooth transitions between different gaits. These challenges become even more pronounced during high-impact maneuvers such as jumping and landing, where precise control is essential.

To address these challenges, researchers have turned to simplified models that distill the essence of legged dynamics while offering tractable control frameworks.

One common approach to understanding and controlling legged locomotion involves the use of templates—simplified mechanical models that capture the essential dynamics of movement while abstracting away the complexities of joints, muscles, and neural control [1]. This idea forms the foundation of template-based control (TBC), a strategy that employs these reduced-order models to guide the behavior of more complex robotic systems. By embedding the simplified “template” within a more detailed “anchor” system, TBC facilitates the realization of stable, robust, and effi-

cient locomotion [2]. Among the most widely used templates is the Spring-Loaded Inverted Pendulum (SLIP) model, which effectively captures the dynamics of various animal and robotic gaits such as trotting [3], [4], [5] and pronking [2], [6], and serves as the foundation for many locomotion strategies [7], [8]. This model has inspired a range of implementations, from Raibert's early work on virtual legged systems that exploit gait symmetry [9], to more sophisticated versions like the clock-torqued SLIP (CT-SLIP), which includes realistic hip-torque inputs to better replicate animal and robot dynamics [10]. Using these template-based strategies, a range of robust and agile locomotion behaviors has been demonstrated in both simulation and hardware [11], [12], [13].

Another prevalent approach for controlling dynamic locomotion behavior is the implementation of prediction-based "optimal" controllers, specifically Model Predictive Control (MPC). MPC works by creating an internal model of the system, predicting its future states, and then finding the best inputs to achieve the desired outcomes. The benefits of MPC have been well-documented in the literature [14], [15], [16]. Moreover, Whole-Body Control (WBC) has emerged as an effective control module that can be integrated into MPC and/or template-based locomotion control algorithms. WBC enhances locomotion by accounting for the effects of leg masses and inertias often neglected in simpler models. The advantages of this approach have been thoroughly explored in various studies [17], [18].

Quadrupedal systems are capable of performing various gaits such as trotting [19], [20], pacing [21], [22], bounding [23], [24] and galloping [25], [26]. In trotting, diagonally opposite legs move together as pairs (i.e., left front with right hind and right front with left hind), creating a stable and symmetric gait that offers good speed and energy efficiency. Pacing, in contrast, involves the lateral legs moving in unison (i.e., left front with left hind and right front with right hind), reducing vertical oscillations but requiring active stabilization. Bounding is characterized by both front legs moving together followed by both hind legs, resulting in an aerial phase between front and hind leg contacts; this gait is typically used at higher speeds. Galloping is an even faster gait where the legs move in a specific sequence with a prolonged aerial phase, often involving asymmetry in footfall timing, and is primarily used for sprinting and rapid maneuvering. While these individual gaits have been extensively studied, the

transitions between them remain under-explored. In animals, smooth transitions between gaits occur naturally and frequently, allowing for efficient adaptation to different speeds and terrains. This ability to transition seamlessly is crucial for versatile and adaptive locomotion, yet most current approaches in robotic systems focus on achieving a specific gait without addressing the dynamic transitions between them. Although studies like [14], [17] successfully implement different gaits, they overlook the essential challenge of transitioning between gaits, which is vital for achieving fluid and efficient movement in changing environments.

Several approaches have been developed to facilitate gait transitions in quadrupedal robots, each addressing different aspects of stability, energy efficiency, and control adaptability. Some methods focus on optimizing energy consumption to enable smooth transitions between gaits. For example, the method presented in [27] employs a hierarchical learning framework, where a high-level reinforcement learning policy determines gait parameters, and a convex Model Predictive Controller (MPC) executes low-level control. While this approach effectively enables gait transitions, it relies on data-driven optimization without explicitly modeling the underlying dynamics of different trotting gaits, which may limit its ability to generalize across varying conditions.

Another approach involves linear interpolation between pre-tuned control parameters, specifically for walking trot and running trot, to facilitate transitions between these gaits [28]. While this method enables smooth shifts from walking to running, it requires separate parameter tuning for different motions, making it less adaptable to new speeds or terrain conditions.

Biologically inspired methods, such as Central Pattern Generators (CPGs), provide an alternative approach by enabling self-organized gait transitions through velocity-dependent phase modulation [29]. These methods allow seamless transitions between walking, trotting, and galloping without pre-programmed patterns. However, since CPG-based methods do not use an explicitly structured template, they often require extensive tuning to maintain stable transitions across different speeds and terrains.

Additionally, the Froude number, a dimensionless parameter that relates gait transitions to speed, has been explored to facilitate shifts between different trotting mo-

tions. For example, [30] uses the Froude number to model walking-to-trotting transitions, while [31] extends this concept to regulate transitions between different trotting speeds. While these approaches provide an analytical framework for predicting gait changes, they do not explicitly define control strategies for dynamic gait adaptation.

Achieving a flying phase in trotting gaits presents an additional challenge, as the controller must manage the takeoff dynamics while handling the system’s uncontrollability during flight. The method introduced in [32] addresses this by combining Virtual Model Control (VMC) with open-loop hopping control to introduce the flight phase into trotting. This approach decouples torso control into a 3D five-linkage model and a planar inverted pendulum model, allowing independent control of posture, forward velocity, and vertical hopping motion. However, the reliance on open-loop hopping control limits adaptability to external disturbances. Furthermore, this method does not leverage a unified template, requiring separate control strategies for stance and flight phases, which may hinder seamless gait transitions.

Among the various gaits exhibited by quadrupedal animals, pronking—a synchronous, bouncing movement involving all four limbs—stands out due to its distinctiveness and functional advantages. This gait, often observed in wild animals, is used not only to signal strength but also to facilitate rapid changes in direction and speed, as well as high jumping heights, making it well-suited for navigating unpredictable terrains [33], [34]. Inspired by such natural dynamics, roboticists have investigated pronking motion in legged platforms, especially hexapodal robots, using template-based control approaches [2], [35].

To model pronking motion in quadrupedal robots, this thesis employs a spring-damper template that effectively captures the elastic properties of leg-ground interactions. Pronking, characterized by simultaneous leg extension during jumping, lends itself well to spring-based modeling, as compliant elements naturally reflect the energy storage and release cycles intrinsic to this gait. The spring-damper system simplifies the otherwise complex dynamics of the robot’s limbs by approximating them as energy-conserving mechanical elements. While previous pronking studies have often relied on SLIP models and focused on hexapodal systems like RHex [36], which operate in planar environments using C-shaped legs, our work extends these princi-

ples to quadrupedal robots operating in full three-dimensional space. Furthermore, the spring-damper framework we adopt shares similarities with virtual model control (VMC) approaches [37] and supports the generation of multiple gaits—including trotting [28]—demonstrating its versatility for diverse locomotion tasks.

Achieving effective legged locomotion in robots also requires precise footstep planning, which is crucial for both motion stability and trajectory tracking. One foundational method, Raibert’s heuristics, provides a set of rules and a dynamic model to determine optimal foot placement, and has been widely adopted for various gait strategies [38], [39], [40]. Beyond heuristics, analytical methods based on simplified models such as the Spring-Loaded Inverted Pendulum (SLIP) template have also been utilized to better understand and predict foot trajectories during the stance phase [41].

However, the dynamic nature of legged motion introduces challenges beyond foot placement. High-impact actions like jumping or transitioning between gaits amplify issues such as maintaining balance and mitigating disturbances. Among these, landing is particularly critical. A poorly controlled landing can result in a loss of stability or mechanical damage, making it a vital phase for robust locomotion. Although significant progress has been made in locomotion and jumping behaviors like pronking and galloping [2], [6], [42], the mechanics and control strategies for safe and stable landings remain relatively underexplored.

Effective landing control is essential for both abrupt gait transitions and deliberate jumps, such as those used to overcome obstacles. Addressing this, recent studies have proposed various strategies. Tail-assisted stabilization, for instance, allows robots to adjust their body orientation mid-air for controlled landings [43], [44]. In tail-less systems, template-based models like the Variable Height Springy Inverted Pendulum (VHSIP) have shown success in achieving landings from considerable horizontal velocities, although they often neglect angular dynamics [45].

Advanced methods such as those presented in [46] use neural networks and nonlinear optimization to compute optimal touchdown poses, enabling landings from heights of up to 8 meters in simulation and 2 meters in physical tests. Despite its promise, the method’s reliance on real-time inference introduces latency (up to 300 ms), which may be prohibitive for rapid landing scenarios. To address such limitations, other ap-

proaches integrate hardware modifications—like adding parallel springs—and continue using the SLIP framework to enhance landing robustness [47].

Moreover, in domains like space exploration where gravity and terrain differ significantly, reinforcement learning-based controllers have been developed to adapt landing behaviors accordingly [48], [49]. These approaches highlight the growing interest in making legged robots not only agile in motion but also resilient upon touchdown, thereby expanding their operational capability in complex and dynamic environments.

1.2 Contributions

In this thesis, we investigate and advance three key aspects of dynamic quadrupedal locomotion: trotting with a generalized CT-SLIP template, pronking through template-based control enhanced with model predictive control (MPC), and robust landing via an optimization-based foot placement strategy. Each of these behaviors is studied in depth, with the proposed methods tailored to the specific challenges of each gait. By developing distinct yet complementary control frameworks for trotting, pronking, and landing, we provide a comprehensive approach to improving adaptability and stability in quadrupedal locomotion.

We first present a novel approach by extending the CT-SLIP template to three dimensions, creating a unified control framework for quadrupedal locomotion. This controller leverages the template to generate stable trotting motions and is further enhanced with a projectile motion model to enable a flying trot by facilitating the flight phase. Unlike previous approaches that rely on separate controllers or manually tuned gait parameters, our method achieves walking trot, regular trot, and flying trot using a single control strategy, ensuring seamless transitions between these gaits. Additionally, the controller is integrated with a whole-body controller, resulting in improved locomotion performance. A comparative analysis is conducted to highlight the benefits of these improvements, and the method is validated through both simulation and real-world experiments.

The contributions of this part of the thesis are as follows: First, we generalize the clock torque-actuated SLIP template to three dimensions, creating a more versatile

framework for controlling quadrupedal locomotion. Second, we incorporate projectile motion into the template, enabling the successful execution of the flight phase during the flying trot. Third, we demonstrate smooth transitions among walking trot, regular trot, and flying trot, all managed by a unified controller. This highlights the versatility of our approach, where a single controller governs the entire spectrum of trotting gaits. Furthermore, the proposed method is validated through real hardware experiments, demonstrating stable locomotion and practical feasibility in real-world conditions. Finally, the CT-SLIP template is enhanced with a whole-body controller to improve stability and overall performance.

While template-based controllers are effective for generating dynamic gaits, their robustness and stability can be further improved. The second part of the thesis proposes a novel controller that combines template-based control (TBC) with model predictive control (MPC). The proposed controller does not rely on any pre-specified trajectory; instead, it uses the template to generate reference motions at each time step. An MPC is rigorously formulated using the system’s Jacobian matrices and is used to track the motion reference generated by the template. In addition to this control strategy, we provide an analytical examination of the template, based on which an approximate analytical footstep planner is developed and implemented as a nonlinear model predictive controller (NMPC). The novelty of this work lies in the integration of TBC with MPC to achieve more robust and adaptive pronking, supported by a carefully designed NMPC for real-time control.

Although legged robotics has progressed significantly, developing a robust and adaptive landing control system remains a major challenge, particularly in optimizing foot placement to ensure stable touchdowns. Existing methods often emphasize trajectory optimization or tail-assisted stabilization, but they typically fail to minimize touchdown moments or account for angular dynamics during the stance phase. The third part of the thesis introduces a novel foot placement controller that integrates template-based control with NMPC to enhance landing stability. The stance phase is modeled using a spring-damper template, which enables analytical prediction of stance motion. Unlike prior approaches, this controller formally optimizes foot placement by minimizing the difference between desired touchdown moments—derived from the template—and the estimated moments resulting from ground reaction forces, while

incorporating friction and geometric constraints. Importantly, the method also considers angular dynamics during stance, which is frequently neglected in related work.

The main contributions of this part are as follows: First, we propose an optimization-based foot placement controller, formulated as an NMPC problem, that explicitly minimizes the discrepancy between desired touchdown moments from the template and those induced by ground reaction forces, while accounting for angular dynamics, friction limits, and body-ground interactions. Second, we validate the effectiveness of the proposed method through extensive simulations and real-robot experiments. Systematic analyses show that the controller enables stable landings even at high horizontal velocities and outperforms existing methods in terms of touchdown stability and robustness. These contributions represent a significant step forward in quadrupedal landing control, enabling more dynamic and adaptive locomotion.

1.3 The Outline of the Thesis

This thesis is organized into five main chapters, each addressing a specific aspect of quadrupedal locomotion using template-based and predictive control strategies. The structure follows a progression from foundational modeling and control of trotting, to the development of more dynamic behaviors such as pronking and landing.

Chapter 2 introduces the Clock-Torque Actuated Spring-Loaded Inverted Pendulum (CT-SLIP) model and presents its extension to three-dimensional space for quadrupedal robots. A control framework is developed to enable various trotting behaviors, including walking trot, regular trot, and flying trot. The chapter also details the formulation of stance and swing phase controllers, a state estimation algorithm, and a whole-body controller to enhance stability. Both simulation and hardware experiments are provided to validate the proposed method’s effectiveness in generating smooth and robust gait transitions.

In Chapter 3, the focus shifts to pronking behavior. A three-dimensional spring-damper template is introduced as the foundation for this gait. A time-varying model predictive controller (MPC) is designed to track the motion references generated by the template, while a nonlinear MPC-based footstep planner is developed to deter-

mine optimal foot placement. The proposed method is first validated in simulation, followed by successful implementation on a physical quadruped robot, demonstrating stable and repeatable pronking performance.

Chapter 4 addresses the landing problem, which is particularly challenging due to the high-impact nature of aerial locomotion. A spring-damper stance model is used to develop a nonlinear model predictive controller (NMPC) for optimizing foot placement at touchdown. The controller aims to minimize the discrepancy between template-predicted and actual moments, while accounting for ground reaction forces and friction. Simulation results indicate superior performance compared to existing approaches, and preliminary hardware experiments show promising outcomes, with future work aimed at further improving real-world applicability.

Chapter 5 concludes the thesis by summarizing the key contributions and findings from each chapter. A discussion on the limitations of the current work is provided, along with suggestions for future research directions, such as extending the controllers to operate on uneven terrain and under higher levels of uncertainty or disturbance.

Appendix A contains the full symbolic derivation of the equations of motion for the quadrupedal robot used throughout the thesis. This includes the kinematic and dynamic modeling, as well as the application of Lagrangian mechanics, forming the basis for all control strategies developed in the earlier chapters.

CHAPTER 2

A COMPREHENSIVE CLOCK TORQUE ACTUATED SLIP FRAMEWORK FOR TROTTING IN QUADRUPEDS

In this chapter, we present a control framework designed to generate various trotting behaviors in quadrupedal robots. Trotting is a symmetrical gait commonly observed in both animals and quadrupedal robots, where diagonally opposite legs (i.e., front-left and rear-right, front-right and rear-left) move together in coordinated pairs. The chapter begins with the introduction of the two-dimensional Clock-Torque Actuated Spring-Loaded Inverted Pendulum (CT-SLIP) template, which serves as the foundation of the approach. This model is then extended to three dimensions to more accurately capture the dynamics of quadrupedal locomotion. Based on this template, dedicated controllers for the stance and swing phases are formulated. A state estimation algorithm is also introduced to infer the robot’s body states using onboard sensor data. The chapter concludes with simulation studies and hardware experiments that validate the performance of the proposed control strategy.

2.1 Template Model

The Spring-Loaded Inverted Pendulum (SLIP) model is a widely used template in legged locomotion studies, offering a simplified yet insightful representation of running and hopping dynamics. In this model, the leg is abstracted as a massless spring attached to a point mass representing the body, allowing researchers to capture the essential behaviors of leg compression, energy storage, and release during ground contact. Despite its simplicity, the SLIP model successfully replicates key features of biological locomotion, such as the alternating stance and flight phases and the passive

stabilization mechanisms observed in animals. Figure 2.1 illustrates the basic configuration of the SLIP model, which serves as the foundation for more complex template designs such as CT-SLIP.

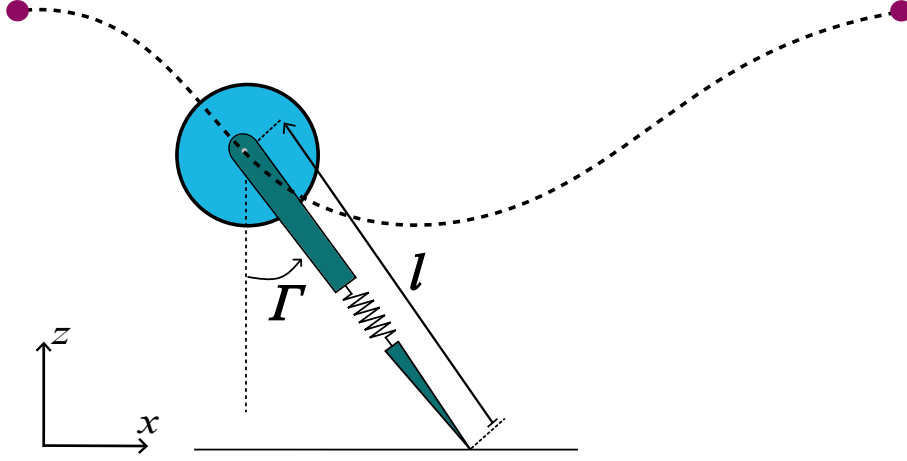


Figure 2.1: Spring-loaded inverted pendulum (SLIP) model. A point-mass body and a massless spring leg of length l are shown during stance, with the leg contacting the ground at angle Γ . The dashed curve represents the center-of-mass trajectory.

The 2-dimensional template model for CT-SLIP is given in Figure 2.2. The template is based on a bipedal SLIP model. Each of the template's legs represents a pair of the real robot's legs. In the template's legs, there are springs and dampers, and each leg is actuated with a torque from the hip. In 2 dimensions, the bipedal SLIP is combined with a trunk. The spiral represents the rotational spring and damper.

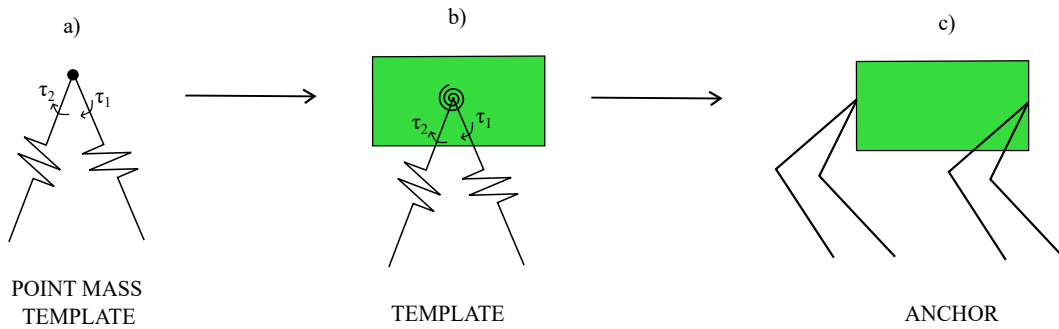


Figure 2.2: Two-dimensional CT-SLIP template. (a) Point-mass CT-SLIP model, (b) CT-SLIP extended with a trunk and hip torques τ_1 and τ_2 , (c) corresponding anchor representation for the quadruped.

The template uses 3 parameters t_c for the time for one period of motion (cycle time), t_s for the stance time of each virtual leg in one period, and Γ_s for the angle that will be swept for each leg. Based on these parameters, the template uses a clock signal to determine which legs should be in contact with the ground and their reference angles.

For a given time t^* , the clock first normalizes this time using equation 2.1. Each of the virtual legs are associated with a diagonal pair of the quadruped legs. These are named as $pair_1$ and $pair_2$. The conditions for whether the pairs are in contact with the ground or not are given in equations 2.2 and 2.3.

$$t_{norm} = t^* - \text{floor}((t^* + t_c/2)/t_c)t_c \quad (2.1)$$

$$s(pair_1) = \begin{cases} 1, & \text{if } \text{abs}(t_{norm}) < t_s/2 \\ 0, & \text{else} \end{cases} \quad (2.2)$$

$$s(pair_2) = \begin{cases} 1, & \text{if } \text{abs}(t_{norm}) > (t_c - t_s)/2 \\ 0, & \text{else} \end{cases} \quad (2.3)$$

Aside from the template parameters, we need to define Γ_i and l_i terms for each leg. Γ_i is defined as the virtual leg angle measured from the vertical axis, and l_i is defined as the leg length. The reference leg angles for the template are shown in Figure 2.3. In the figure, only the contact leg angles are shown, because contrary to [30], when the legs are not in the contact phase, the template is not used. The reference angular velocity for the template legs is $\dot{\Gamma}_{ref} = -\Gamma_s/t_s$.

The required torque caused by the clock (τ_i) and the leg forces caused by springs and dampers (F_i) can be calculated from equation 2.4. In the equation, $K_{p\tau}$, $K_{d\tau}$, K_{ps} , K_{ds} are proportional and derivative gains for τ_i and F_i , respectively. The error terms are defined as $\delta\Gamma_i = \Gamma_{ref,i} - \Gamma_i$, $\delta\dot{\Gamma}_i = \dot{\Gamma}_{ref} - \dot{\Gamma}_i$, $\delta l_i = l_0 - l_i$, $\delta\dot{l}_i = -\dot{l}_i$.

$$\begin{aligned} \tau_i &= K_{p\tau}\delta\Gamma_i + K_{d\tau}\delta\dot{\Gamma}_i \\ F_i &= K_{ps}\delta l_i + K_{ds}\delta\dot{l}_i \end{aligned} \quad (2.4)$$

The equations of motion for the torque actuated spring loaded inverted pendulum model, given in Figure 2.4 are given in equations 2.5, similar to [50]. For each leg the equations can be written as in equations 2.6.

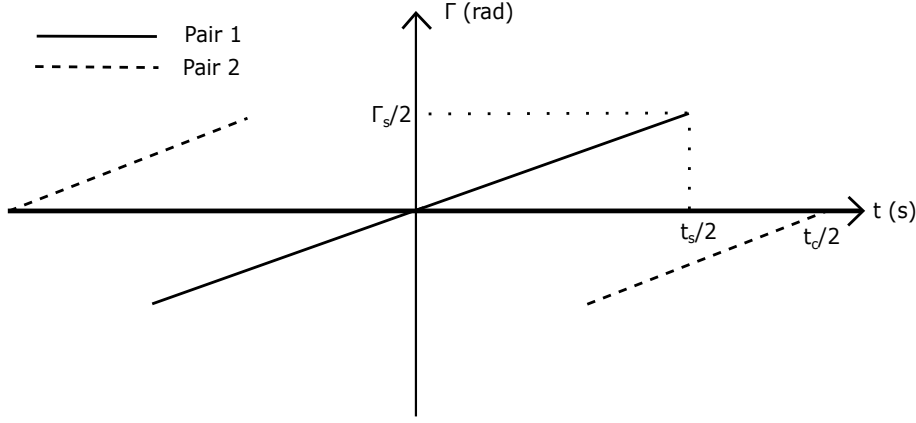


Figure 2.3: Reference virtual leg angles. The reference angular trajectories for the two virtual leg pairs are shown, each sweeping an angle of Γ_s over the stance duration t_s .

$$\begin{aligned}
 m\ddot{l} + mg\cos(\Gamma) &= m\dot{\Gamma}^2 + K_{ps}(l_0 - l) - K_{ds}\dot{l} \\
 ml^2\ddot{\Gamma} - mgl\sin(\Gamma) &= -2ml\dot{\Gamma} + K_{p\tau}(\Gamma_{ref} - \Gamma) + K_{d\tau}(\dot{\Gamma}_{ref} - \dot{\Gamma})
 \end{aligned} \tag{2.5}$$

$$\begin{aligned}
 m\ddot{l}_i + mg\cos(\Gamma_i) &= m\dot{\Gamma}_i^2 + F_i \\
 ml_i^2\ddot{\Gamma}_i - mgl_i\sin(\Gamma_i) &= -2ml_i\dot{\Gamma}_i + \tau_i
 \end{aligned} \tag{2.6}$$

In this paper, we calculate the external forces to be applied to the system based on the template model, compensating for the effects of gravity. This approach effectively eliminates the affine relationship imposed by gravity, allowing the SLIP model to behave as though it is in a gravity-free environment. By doing this, our method avoids energy losses caused by gravity. The specific equations for these external forces that are applied to the body are provided in equations 2.7.

$$\begin{aligned}
 F_i^{rad} &= F_i + mg\cos(\Gamma_i) \\
 M_i^{ang} &= \tau_i - mgl_i\sin(\Gamma_i)
 \end{aligned} \tag{2.7}$$

To apply this template to the real robot, the template needs to be generalized to 3 dimensions as shown in Figure 2.5. In 3D, it is assumed that in each rotational axis and the axis normal to the sagittal plane, y-axis on Figure 2.5, there are springs and dampers. The template forces are rotated to the global coordinate frame using equation 2.8, where $\mathbf{R}_i$ is the i-th rotational matrix. The 3-dimensional force generated by the legs can be written as in equation 2.9. In the double stance phase,

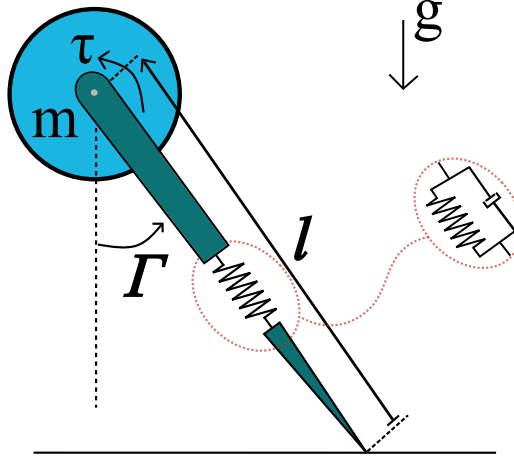


Figure 2.4: Torque-actuated SLIP model. A SLIP leg with spring–damper compliance and hip torque τ acting on a point mass m is illustrated, with leg angle Γ and length l defined relative to the ground.

where both legs of the template are in contact, the 3-dimensional force is defined as $\mathbf{F}_t^{3D} = \mathbf{F}_{t1}^{3D} + \mathbf{F}_{t2}^{3D}$ with dividing gravity terms in equations 2.7 by 2, in single stance phase it is defined as $\mathbf{F}_t^{3D} = \mathbf{F}_{ti}^{3D}$. In equation 2.8, $\mathbf{e}_i$ denotes the i th standard basis vector in $\mathbb{R}^3$, defined as $\mathbf{e}_1 = [1 \ 0 \ 0]^\top$, $\mathbf{e}_2 = [0 \ 1 \ 0]^\top$, $\mathbf{e}_3 = [0 \ 0 \ 1]^\top$.

$$\begin{aligned} \mathbf{F}_{ri}^{3D} &= -\mathbf{R}_i F_i^{rad} \mathbf{e}_1 \\ \mathbf{F}_{ai}^{3D} &= -\frac{1}{l_i} \mathbf{R}_i M_i^{ang} \mathbf{e}_3 \end{aligned} \quad (2.8)$$

$$\mathbf{F}_{ti}^{3D} = \mathbf{F}_{ri}^{3D} + \mathbf{F}_{ai}^{3D} \quad (2.9)$$

The forces generated by the 3-dimensional template are given in equation 2.10. The global frame directions are named as $x, y, z, \psi, \phi, \gamma$.

$$\begin{aligned} F_x &= \mathbf{F}_t^{3D}(1) \\ F_z &= \mathbf{F}_t^{3D}(3) \\ F_y &= K_{py}(y_{des} - y) + K_{dy}(\dot{y}_{des} - \dot{y}) \\ F_\psi &= K_{p\psi}(\psi_{des} - \psi) + K_{d\psi}(\dot{\psi}_{des} - \dot{\psi}) \\ F_\phi &= K_{p\phi}(\phi_{des} - \phi) + K_{d\phi}(\dot{\phi}_{des} - \dot{\phi}) \\ F_\gamma &= K_{p\gamma}(\gamma_{des} - \gamma) + K_{d\gamma}(\dot{\gamma}_{des} - \dot{\gamma}) \end{aligned} \quad (2.10)$$

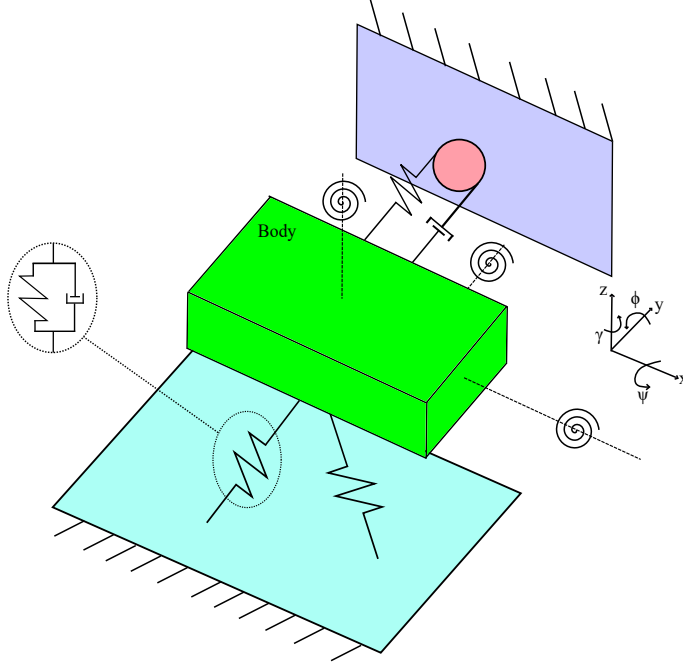


Figure 2.5: Three-dimensional CT-SLIP template. The 3D extension of the CT-SLIP model is shown, where compliant virtual legs act in all translational and rotational directions to generate body forces and moments.

Each virtual leg in the template represents a diagonal pair of legs on the physical robot. The properties of these virtual legs—such as leg length, angle, and velocity—are computed based on the measurements of the front leg within each diagonal pair. An illustration of the virtual leg configuration is provided in Figure 2.6.

2.2 Controller Model

To control the quadruped robot according to the template, we first need to deal with the equations of motion for the quadrupedal robot given in equation 2.11. In the equation, $\mathbf{M}$ is the mass matrix, $\mathbf{B}$ is the Coriolis force vector, $\mathbf{G}$ is the gravitational weight vector, $\boldsymbol{\tau}$ is the actuation vector, $\mathbf{J}$ is the jacobian matrix of the system and $\mathbf{F}$ represents the forces externally applied to system. Equation A.8 gives the generalized coordinates of the system, the first 6 elements are defined as the body states, roll, pitch, yaw angles and x, y, z positions. Remaining 12 elements are joint angles.

$$\mathbf{M}\ddot{\mathbf{q}} + \mathbf{B} + \mathbf{G} = \boldsymbol{\tau} + \mathbf{J}^\top \mathbf{F} \quad (2.11)$$

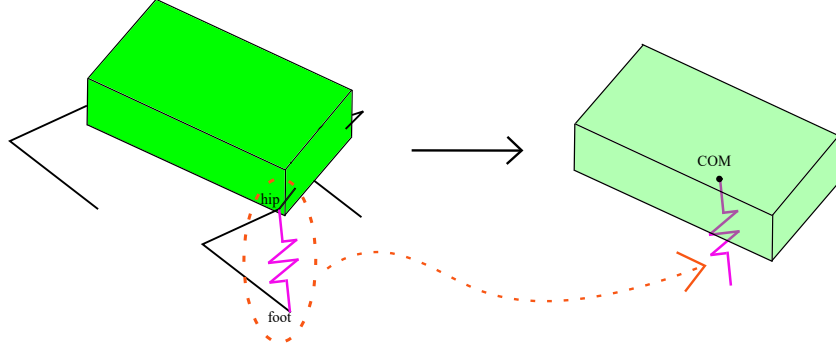


Figure 2.6: Virtual leg–real leg relation. The virtual leg of the template is mapped to the corresponding real leg by using the hip–foot configuration, and its force is applied to the body’s center of mass.

$$\mathbf{q} = [\psi \ \phi \ \gamma \ x \ y \ z \ \boldsymbol{\theta}_{12 \times 1}]^\top \quad (2.12)$$

Since most of the quadrupeds carry the majority of their weights on their body rather than their legs, a common approach used in literature is neglecting the leg masses and inertias [51], [52]. If the leg masses and inertias are neglected, the coriolis force vector becomes zero, also the first 6×6 elements of the mass matrix becomes non-zero. Since the body states are not directly actuated, first 6 elements of the $\boldsymbol{\tau}$ are zero. Therefore, the system can be decomposed into two systems given in equation 2.13 and 2.14. In the equation 2.13, $\mathbf{M}_{\text{body}}$ represents the first 6×6 elements of the mass matrix, $\mathbf{G}_{\text{body}}$ corresponds to the first 6 elements of the weight vector, $\mathbf{J}_{\text{body}}^\top$ refers to first 6 rows of the transpose of the jacobian matrix which maps the ground reaction forces to body forces. In the equation 2.14, $\boldsymbol{\tau}_{\text{joint}}$ represents the joint torques and $\mathbf{J}_{\text{joint}}^\top$ corresponds to the remaining rows of the transpose of the jacobian matrix.

$$\mathbf{M}_{\text{body}} \ddot{\mathbf{q}}_{\text{body}} + \mathbf{G}_{\text{body}} = \mathbf{J}_{\text{body}}^\top \mathbf{F} \quad (2.13)$$

$$\boldsymbol{\tau}_{\text{joint}} = -\mathbf{J}_{\text{joint}}^\top \mathbf{F} \quad (2.14)$$

Once the desired body forces are calculated from the template, the next step is finding the required joint torques. In order to find the required joint torques, necessary ground reaction forces (GRFs) are first calculated. To find GRFs, the optimization problem given in equation 2.15 is solved.

$$\min_U \quad \|\mathbf{J}_{\text{body}}^\top \mathbf{U} - \mathbf{H}\|_{\mathbf{Q}^{\text{tmp}}} \quad (2.15)$$

subject to

$$-\tau_{\max} \leq \mathbf{L}\mathbf{U} \leq \tau_{\max} \quad (2.15a)$$

$$\mathbf{D}\mathbf{U} \leq 0 \quad (2.15b)$$

$$\bar{\mathbf{u}}_{\min} \leq \mathbf{U} \quad (2.15c)$$

In equation 2.15, the $\mathbf{H}$ term is defined as the template forces $\mathbf{H} = \mathbf{F}^{3D}$, given in equation 2.10. $\mathbf{Q}^{\text{tmp}}$ is the diagonal weight matrix. The equation 2.15a is for not violating the joint torques, the equation 2.15b is for friction constraint, in order to prevent slippage and the equation 2.15c is for maintaining the ground contact. The friction cone is linearized as an octogonal pyramid.

After the GRFs are found, they are converted to joint torques using equation 2.14. These are the joint torques for the stance legs, legs that are in contact with the ground.

For the swing legs, a 5th order polynomial is used for motion planning. The planned motion for the swing leg is given in Figure 2.7. The motion of the foot is planned relative to hip location. In the figure, $\mathbf{p}_0$ is the starting position of the foot and $\mathbf{v}_0$ is the starting velocity, $\mathbf{p}_1$ is the middle position $\mathbf{v}_1$ is the corresponding velocity and the $\mathbf{p}_2$ is the end position with velocity of $\mathbf{v}_2$. $x_{\max}$ is the furthest horizontal position that foot tip takes relative to hip and $z_{\max}$ is the furthest vertical position that foot tip takes in one swing motion.

In legged systems, deciding where the foot should be placed when it touches to the ground is crucial. Most approaches rely on heuristics, or rule-based methods for planning the footstep, as seen in [53], [54]. However, this template does not require an additional footstep planner, such as Raibert's footstep planner [38]. Instead, the horizontal distance for the footstep can be directly determined using equation 2.16 without needing any heuristic methods. This simplifies the process and makes it more straightforward to plan the foot placements.

$$x_{\max} = l_0 \sin(\Gamma_s/2) \quad (2.16)$$

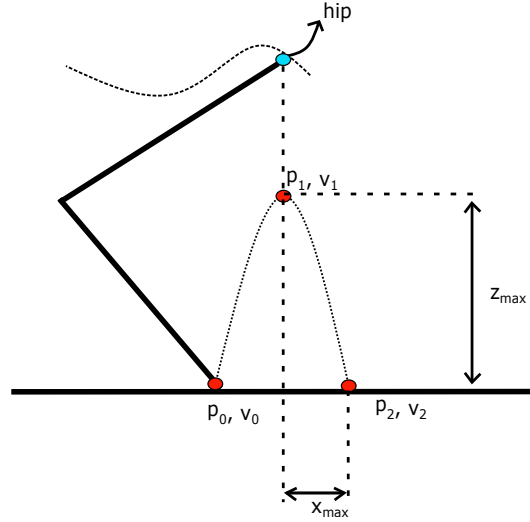


Figure 2.7: Motion of the swing leg. The planned swing trajectory is defined by initial, mid, and final foot positions (p_0, p_1, p_2) with corresponding velocities, reaching a vertical height $z_{\max}$ and a horizontal span $x_{\max}$.

The equations for the desired position and velocity of the foot tip is given in equation 2.17. $\mathbf{a}_i$ terms are 2×1 vectors. The unknown $\mathbf{a}_i$ vectors are found from the $\mathbf{p}_i$ and $\mathbf{v}_i$ terms, which are defined in equations 2.17a, 2.17b, and 2.17c.

$$\mathbf{p}(t) = \mathbf{a}_0 + \mathbf{a}_1 t + \frac{1}{2} \mathbf{a}_2 t^2 + \frac{1}{6} \mathbf{a}_3 t^3 + \frac{1}{24} \mathbf{a}_4 t^4 + \frac{1}{120} \mathbf{a}_5 t^5 \quad (2.17)$$

$$\mathbf{v}(t) = \mathbf{a}_1 + \mathbf{a}_2 t + \frac{1}{2} \mathbf{a}_3 t^2 + \frac{1}{6} \mathbf{a}_4 t^3 + \frac{1}{24} \mathbf{a}_5 t^4$$

$$\mathbf{p}_0 = [-x_{\max} \ 0]^\top \quad \mathbf{v}_0 = [0 \ \epsilon]^\top \quad (2.17a)$$

$$\mathbf{p}_2 = [x_{\max} \ 0]^\top \quad \mathbf{v}_2 = [0 \ -\epsilon]^\top \quad (2.17b)$$

$$\mathbf{p}_1 = [0 \ z_{\max}]^\top \quad \mathbf{v}_1 = (\mathbf{p}_2 - \mathbf{p}_0)/t_{\text{swing}} \quad (2.17c)$$

Once the parameters for the polynomials are found, the desired position and velocity for the foot tip is calculated at each controller time step. Afterwards, they are converted to desired joint angles and velocities using inverse kinematics and tracked with a PD controller.

The block diagram for the controller is given in Figure 2.8. First, the clock determines the phase of the robot. The clock computes which leg pairs of the robot is in contact with the ground. If both virtual legs are in contact with the ground at a given moment,

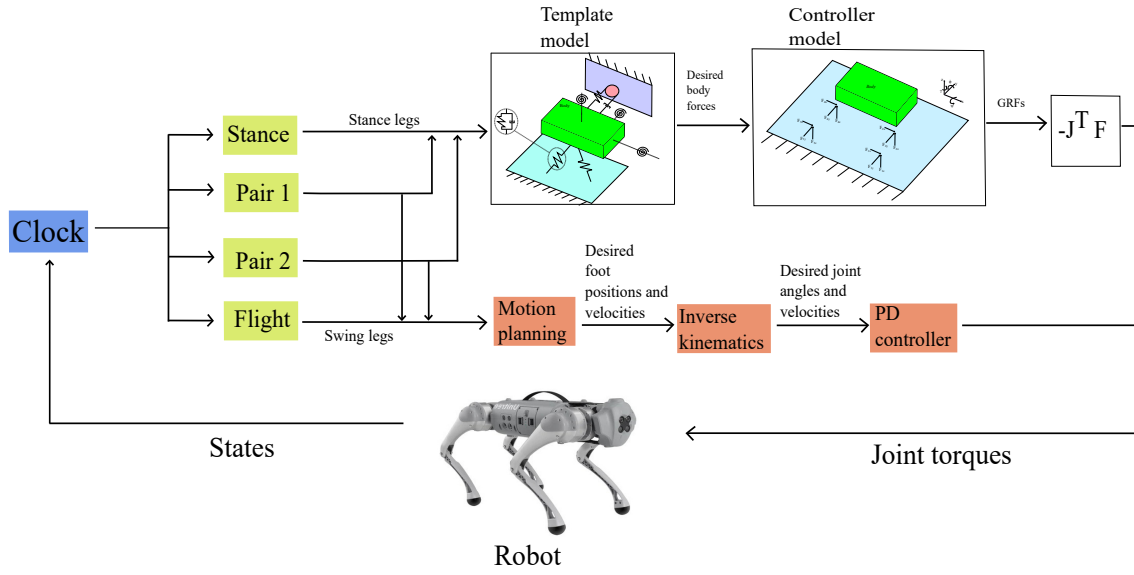


Figure 2.8: Block diagram of the controller. The diagram illustrates the clock-driven phase selection, template-based body force generation, swing-leg motion planning, and the computation of joint torques.

the robot is in the full stance phase, or simply the stance phase. If neither leg is in contact with the ground, the robot is in the flight phase. Based on these phases, the desired body forces are calculated from the template. Then, the optimal GRFs are calculated by solving an optimization problem and they are converted to joint torques. For the swing legs, the desired foot tip positions and velocities are determined using polynomial motion planning. These are then converted into reference joint angles and velocities, which are used to compute joint torques via a PD controller. Once the joint torques are calculated, they are applied as inputs to the robot. The state estimation part will be explained in the later sections.

2.3 Trotting Gaits: Walking to Running

Trotting gait in quadrupeds is a symmetrical gait where diagonally opposite pairs of legs move in unison. Based on the timings of the leg movements we can divide the trotting gait into three parts, namely walking trot, trot, and flying trot as seen in Figure 2.9. In Figure 2.9, the black rectangles indicate when the leg is in contact with the ground, while the white rectangles signify that the leg is in the swing phase, with no

ground contact. The walking trot includes a full support phase, meaning that all the legs are in contact with the ground. The trot does not include the full support phase or the flight phase. In the flying trot or running trot, a flight phase exists.

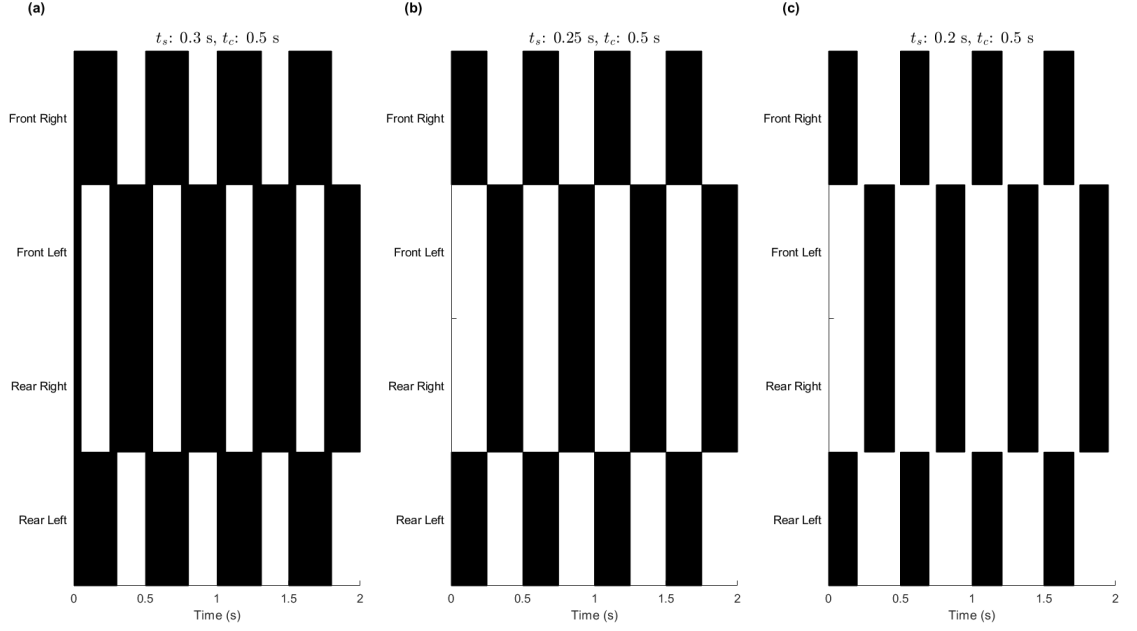


Figure 2.9: Contact legs for trotting. Contact sequences for the four legs are shown for (a) walking trot, (b) trot, and (c) flying trot, highlighting the stance durations associated with different gait types.

Although the CT-SLIP template deals with the walking trot and trot successfully, it does not tackle the flying phase. We modified the template for managing the flying behavior. Total flying time in one cycle can be calculated by $t_c - 2t_s$, after each pair is in contact, half of this time is spent in the air; therefore each flight time can be calculated by equation 2.18. By assuming the robot moves in the air as a projectile, the desired vertical velocity of the body at lift-off can be calculated by equation 2.19.

$$t_f = (t_c - 2t_s)/2 \quad (2.18)$$

$$\dot{z}_{des} = .5gt_f \quad (2.19)$$

The SLIP model undergoes two phases instance, compression and de-compression given in Figure 2.10. When the virtual legs go into the de-compression phase, the reference value for the velocity in the virtual leg direction changes to $\dot{z}_{des}/\cos(\Gamma_i)$

and the force equation in 2.4 changes to equation 2.20. While calculating the change from compression to de-compression phase, it is assumed that the stance phase is symmetric in time; therefore, the phase shift happens at the middle of the stance time.

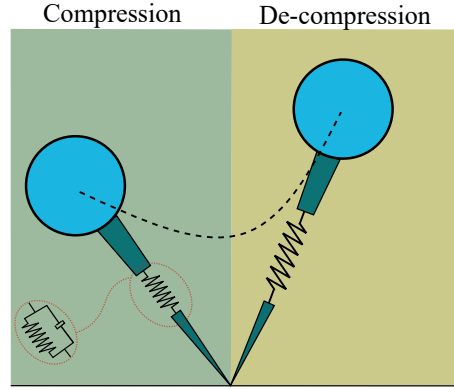


Figure 2.10: Compression and de-compression phases of the SLIP model. The stance phase is illustrated with the leg spring compressing and subsequently de-compressing.

$$F_i = K_{ps}\delta l_i + K_{ds}(\dot{z}_{des}/\cos(\Gamma_i) - \dot{l}_i) \quad (2.20)$$

2.4 Turning in the Yaw Direction

Animals move in the direction they face; thus, to navigate toward a location not directly ahead, they rotate in the yaw direction while moving forward. This natural mechanism enables legged systems to adjust their trajectory smoothly without requiring lateral motion. Figure 2.11 illustrates the forward and yaw directions, where the green arrow represents forward motion, and the red arrow indicates yaw rotation.

To achieve turning, the robot rotates in the yaw direction primarily through adjustments in the stance legs, which control the robot's pose. This maneuver can be further refined by modifying foot placements, making turns smoother and more responsive. In this paper, we propose rotating the swing foot plane to assist with yaw rotation. By doing so, the robot achieves more stable turning with higher rotation rates.

Figure 2.12 shows the swing leg motion planes with and without compensation, where compensation refers to the plane's rotation to aid yaw rotation. This rotation is cen-

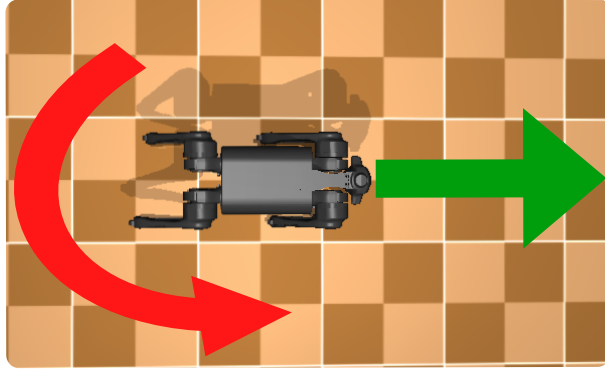


Figure 2.11: Swing foot planes. (a) Foot placement without yaw compensation, (b) foot placement with yaw-compensated swing plane adjustment.

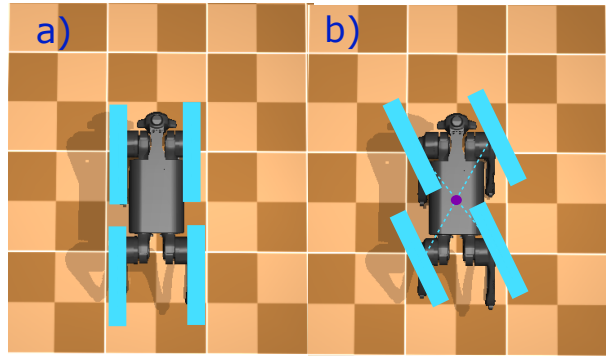


Figure 2.12: Swing foot planes a) Foot placement with no compensation, b) foot placement with compensation

tered at the robot's geometric center, and the rotation angle, $\tilde{\gamma}$, is calculated using equation 2.21, where $K_{\gamma,corr}$ is a gain, γ_{des} is the desired yaw angle, and γ is the current yaw angle.

$$\tilde{\gamma} = K_{\gamma,corr}(\gamma_{des} - \gamma) \quad (2.21)$$

In the motion planning for swing legs, after calculating the desired foot positions, the vector from the robot's geometric center to each foot's position is rotated in the yaw direction. This adjustment is then used in inverse kinematics to achieve the modified foot placements.

2.5 Whole-Body Controller

Typically, template models for legged robots do not take into account the masses of the robot's legs, and as a result, the effects of these masses on the robot's overall behavior are often ignored. However, the whole-body control approach addresses this limitation by considering all the dynamics involved. This method aims to find the closest dynamically feasible solution through optimization, similar to the approach described in [17], but with several important modifications.

In this paper, contrary to [17], we do not input the desired accelerations to the whole-body controller directly. Instead, these accelerations are calculated within the controller using a portion of the mass matrix. For the legs that are in contact with the ground (stance legs), the accelerations are determined by ensuring the foot tips do not slip, known as the no-slip condition. For the legs that are in the air (swing legs), the accelerations are derived from dynamic equations. Since the perturbations (δ_a) that will be added to the desired accelerations will affect the leg accelerations, more than one iterations are performed in order to accurately solve the dynamics of the equation.

The whole-body controller block takes the GRFs as inputs, and outputs the joint torques needed for movement. To achieve this, the controller introduces small perturbations to the GRFs and desired COM accelerations. It then works to minimize these perturbations while ensuring all constraints are met, using a quadratic programming (QP) optimization problem. The approach is given in equation 2.22. In the equation, S_F is the selection matrix, it is defined as a zero matrix but the first 6 diagonal elements are 1. The constraints 2.22a, 2.22b and 2.22c are for dynamic feasibility, and the constraint 2.22d is for joint torques limitations and contact constraint.

$$\min_{\delta_F, \delta_a} \quad \delta_F^\top Q_1 \delta_F + \delta_a^\top Q_2 \delta_a \quad (2.22)$$

$$S_F (M \ddot{\mathbf{q}} + \mathbf{B} + \mathbf{G}) = S_F \mathbf{J}^\top \mathbf{F}_{\text{wbc}} \quad (2.22a)$$

$$\ddot{\mathbf{q}} = \ddot{\mathbf{q}}_{\text{des}} + \begin{bmatrix} \delta_a \\ 0 \end{bmatrix} \quad (2.22b)$$

$$\mathbf{F}_{\text{wbc}} = \mathbf{F} + \delta_F \quad (2.22c)$$

$$\mathbf{R} \mathbf{F}_{\text{wbc}} \leq \mathbf{P} \quad (2.22d)$$

The optimization problem can be simplified by further analysis. By combining the equality constraints, we can write equation 2.23. Also, since the only first 6 elements of the δ_a are non-zero, instead of $-M\delta_a$ term, $-M_{\text{red}}\delta_a$ term can be used, where M_{red} is the first 6 columns of the mass matrix. Then the equality constraint takes the form of equation 2.24. The size of the M_{red} matrix is 18×6 .

$$S_F (M\ddot{q} + B + G - J^\top F_{\text{wbc}}) = S_F \left(-M \begin{bmatrix} \delta_a \\ 0 \end{bmatrix} + J^\top \delta_F \right) \quad (2.23)$$

$$S_F (M\ddot{q} + B + G - J^\top F_{\text{wbc}}) = -S_F M_{\text{red}} \delta_a + S_F J^\top \delta_F \quad (2.24)$$

If we combine the δ_F and δ_a terms, we can write equation 2.27 as

$$W = \left[S_F J^\top \mid -S_F M_{\text{red}} \right], \quad (2.25)$$

$$\delta = \begin{bmatrix} \delta_F \\ \delta_a \end{bmatrix}. \quad (2.26)$$

The size of W is 18×18 for full stance and 18×12 when only a leg pair is in contact. Similarly, the size of δ is 18×1 for full stance and 12×1 for a single-contact pair.

$$S_F (M\ddot{q} + B + G - J^\top F_{\text{wbc}}) = W \delta \quad (2.27)$$

The last step for formulating the whole-body controller is contact constraint formulation. The leg in contact should not violate the friction cone constraints, also the joint torques should be in the allowable range. The friction cone constraints are linearized as an octagonal pyramid. Inequality constraints can be written in the form of equation 2.28.

$$R(F + \delta_F) \leq P \rightarrow R\delta_F \leq P - RF \quad (2.28)$$

This equation can also be written as equation 2.29, where $\bar{L} = \begin{bmatrix} L & 0 \end{bmatrix}$, $\bar{P} = P - RF$.

$$\bar{R}\delta \leq \bar{P} \quad (2.29)$$

After the simplifications, the optimization problem takes the form in equation 2.30, where $Q = \text{blkdiag}(Q_1, Q_2)$.

$$\min_{\delta} \quad \delta^\top \mathbf{Q} \delta \quad (2.30)$$

subject to

$$\mathbf{S}_F (\mathbf{M} \ddot{\mathbf{q}} + \mathbf{B} + \mathbf{G} - \mathbf{J}^\top \mathbf{F}_{\text{wbc}}) = \mathbf{W} \delta \quad (2.30a)$$

$$\bar{\mathbf{R}} \delta \leq \bar{\mathbf{P}} \quad (2.30b)$$

After the optimization is solved, the accelerations are updated by $\ddot{\mathbf{q}} = \ddot{\mathbf{q}}_{\text{des}} + \begin{bmatrix} \delta_a \\ 0 \end{bmatrix}$. Because of this update, the joint accelerations should also be updated, since they do no longer satisfy the equations for no slip condition. The optimization algorithm should be iterated until the solution converges. It is observed that 2 iterations are sufficient for convergence for this system.

Since all the quantities on the right-hand side are known, the joint torques that satisfy the dynamic equation can be found by equation 2.31.

$$\boldsymbol{\tau} = \mathbf{M} \ddot{\mathbf{q}} + \mathbf{B} + \mathbf{G} - \mathbf{J}^\top \mathbf{F} \quad (2.31)$$

The block diagram given in figure 2.8 is modified as figure 2.13. Instead of using equation $-\mathbf{J}^\top \mathbf{F}$, the whole-body controller block is added to find the joint torques.

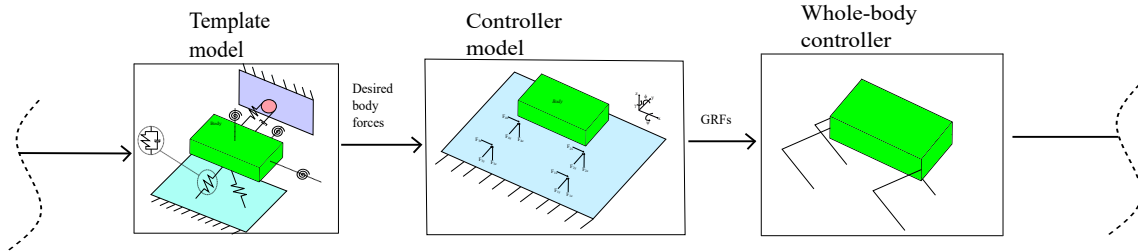


Figure 2.13: Block diagram of the controller with WBC. The whole-body controller is added after the template and controller models to refine the generated ground reaction forces.

2.6 State Estimation

To effectively control the robot, it is essential to estimate the body states, as they cannot be directly measured. The available data for estimating these states are ob-

tained from the accelerometer, gyroscope, and encoders. This information is fused using a Kalman Filter [55], [56], which combines sensor data to produce accurate state estimates. The approach leverages the IMU equations to model the motion dynamics, while kinematic calculations derived from contact information and encoder measurements are used as corrective inputs. These kinematic calculations help refine the state estimates by providing additional information about the robot's motion, ensuring robust and precise control.

In this paper, the gyro measurements are assumed to be accurate, which allows the use of a linear Kalman Filter for state estimation [56]. The states for the model are defined in Equation 2.32, where $\mathbf{p}_b$ represents the position of the body in the global frame, $\mathbf{v}_b$ denotes the velocity of the body in the global frame, and $\mathbf{p}_{f_i}$ corresponds to the positions of the feet in the global frame.

$$\Phi = \left[\mathbf{p}_b^\top \mathbf{v}_b^\top \mathbf{p}_{f1}^\top \mathbf{p}_{f2}^\top \mathbf{p}_{f3}^\top \mathbf{p}_{f4}^\top \right]^\top \quad (2.32)$$

The continuous-time update equations are presented in Equations 2.33, 2.34, and 2.35. Here, $\mathbf{R}_b$ represents the rotation matrix transforming coordinates from the global frame to the IMU orientation, $\mathbf{a}_b$ denotes the linear acceleration in the body frame, and $\mathbf{a}_g$ corresponds to the gravitational acceleration in the global frame. The terms $\mathbf{w}_v$ and $\mathbf{w}_{f_i}$ represent Gaussian noise, accounting for uncertainties in the velocity and foot position measurements, respectively.

$$\dot{\mathbf{p}}_b = \mathbf{v}_b \quad (2.33)$$

$$\dot{\mathbf{v}}_b = \mathbf{R}_b \mathbf{a}_b + \mathbf{a}_g + \mathbf{w}_v \quad (2.34)$$

$$\dot{\mathbf{p}}_{f_i} = \mathbf{w}_{f_i} \quad (2.35)$$

In discrete time these equations turns into equations in 2.36, 2.37, 2.38.

$$\mathbf{p}_b[k+1] = \mathbf{p}_b[k] + T \mathbf{v}_b[k] + \frac{1}{2} T^2 (\mathbf{R}_b \mathbf{a}_b + \mathbf{a}_g + \mathbf{w}_v) \quad (2.36)$$

$$\mathbf{v}_b[k+1] = \mathbf{v}_b[k] + T (\mathbf{R}_b \mathbf{a}_b + \mathbf{a}_g + \mathbf{w}_v) \quad (2.37)$$

$$\mathbf{p}_{f_i}[k+1] = \mathbf{p}_{f_i}[k] + \mathbf{T}\mathbf{w}_{f_i} \quad (2.38)$$

In these equations the input can be thought of as $(\mathbf{R}_b\mathbf{a}_b + \mathbf{a}_g)$ and the state space equations can be written as equation 2.39.

$$\Phi[k+1] = \bar{\mathbf{A}}\Phi[k] + \bar{\mathbf{B}}\mathbf{u}[k] + \bar{\mathbf{G}}\mathbf{w}[k] \quad (2.39)$$

The kinematic calculations provide the robot's body states relative to a foot in contact with the ground. When a foot is in contact, the robot's relative body position and velocity can be determined based on the position of the contact foot. This relationship can also be derived from the state equations. Assuming the i -th foot is in contact with the ground, the corresponding output is given in Equation 2.40. In this equation, the term $p_{f_i}^3[k]$ represents the third element of the contact foot's position, which corresponds to its height. For a foot in contact with the ground, this height measurement should be zero, ensuring the constraint is satisfied. The contact information is retrieved from the clock and assumed to be accurate.

$$\mathbf{y}_i[k] = \left[(\mathbf{p}_b[k] - \mathbf{p}_{f_i}[k])^\top \quad \mathbf{v}_b[k]^\top \quad p_{f_i}^3[k] \right]^\top \quad (2.40)$$

The output state equation is given in equation 2.41. Here the $\bar{\mathbf{C}}$ matrix is a dynamic matrix, changing size with the contact information.

$$\mathbf{y}[k] = \bar{\mathbf{C}}\Phi[k] \quad (2.41)$$

2.7 Experimental Setup

The Multi-Joint dynamics with Contact (MuJoCo) simulator is utilized as the primary simulation environment for this study. MuJoCo is a high-performance physics engine designed for real-time simulation of complex multi-body systems [57]. It models impact and contact dynamics using a soft-contact approach, enabling continuous representation of contact forces. This method reduces the abruptness of interactions during collisions by applying a penalty-based force model, where the forces exerted

are proportional to the penetration depth. This results in a smoother and more realistic physical response. In addition to the impact model, MuJoCo incorporates a Coulomb friction model that accounts for both static and dynamic friction during contact scenarios. This combination of penalty-based force modeling and friction modeling enables a comprehensive and realistic analysis of contact dynamics in various simulation environments [57], [58]. Known for its computational efficiency, MuJoCo is particularly well-suited for simulating dynamic systems with frictional contact, making it ideal for robotics applications [59], [60], [61]. Its ability to handle complex interactions between bodies in real-time makes it an excellent tool for studying the intricacies of multi-body system behaviors.

For the simulations, the Unitree GO1 robot is utilized, as shown in Figure 2.14. The GO1 is an advanced quadruped robot developed by Unitree Robotics, renowned for its agility, stability, and versatility across various terrains. Equipped with powerful actuators and sophisticated control algorithms, the GO1 enables precise, smooth movements, making it highly suitable for research applications. The model used in MuJoCo is a detailed representation of the physical robot, incorporating parameters such as leg masses, rotational inertias, and joint torque limits. The robot model was sourced from Google DeepMind’s GitHub repository [62]. In this XML configuration, the actuators are set for torque control rather than position control, with specific torque limits to simulate real-world dynamics. The quadrupedal robot has a total mass of approximately 12 kg. Each of its legs consists of three segments: the hip (8 cm), thigh (21.3 cm), and calf (21.3 cm). The torque limits are defined as 23 Nm for the abduction and hip joints, while the knee joint has a higher torque limit of 35 Nm. Detailed inertia matrices and segment masses are provided in Table 2.1.

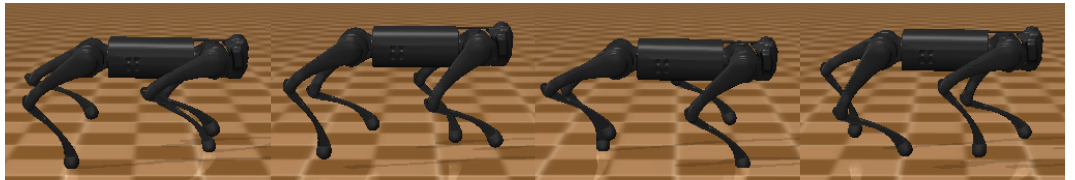


Figure 2.14: GO1 robot in the MuJoCo simulator. Snapshots from the simulated trotting motion.

In order to obtain equations of motion for whole-body controller, Lagrangian me-

Table 2.1: Inertial and mass parameters of the quadruped robot used in simulation experiments

Link	Mass (kg)	Inertia ($kg \cdot m^2$)
Trunk	5.204	diag(0.0716565, 0.0630105, 0.0168101)
Hip	0.680	diag(0.000734064, 0.000468438, 0.000398719)
Thigh	1.009	diag(0.00478717, 0.00460903, 0.000709268)
Calf	0.196	diag(0.00149767, 0.00148468, 3.58427e-05)

chanics is used as given in equation 2.42. D_k is dissipation, K is kinetic energy, U is potential energy, F is force wrench, force+momentum as a column matrix, q_k 's are generalized coordinates, e_k is the standard basis vector. These equations are computed symbolically using MATLAB.

$$\dot{P}_k - S_k + G_k = Q_k - D_k + T_k \quad (2.42)$$

Where;

$$P_k = \frac{\partial K}{\partial \dot{q}_k} \quad S_k = \frac{\partial K}{\partial q_k} \quad G_k = \frac{\partial U}{\partial q_k} \quad T_k = e_k^\top J^\top F$$

To solve the optimization problem in equation 2.15, the qpOASES library is used [63]. This open-source C++ library is specifically designed for real-time quadratic programming (QP) problems, making it highly effective in applications like model predictive control due to its speed [64], [65], [66], [67]. Inverse kinematics for the swing legs are computed using the GNU Scientific Library (GSL) [68], which offers a broad range of numerical algorithms for root-finding, integration, and linear algebra, commonly used in scientific computing. For solving the quadratic programming problem with equality constraints in equation 2.30, ALGLIB is employed [69]. ALGLIB provides efficient algorithms for constrained optimization tasks, making it well-suited for this purpose. Finally, all linear algebra operations are performed using the Eigen library [70], a C++ template library known for its efficiency in matrix and vector operations, and solving systems of equations, which is widely used in scientific and engineering computations.

2.8 Simulation Results

In this section, we present the experimental results obtained from simulations. We begin by demonstrating how the proposed template is used to achieve various gaits, including walking trot, trot, and running trot, and evaluate the disturbance rejection performance for each gait. Then, we compare the performance of the template-based control (TBC) with the improved template-based control combined with a whole-body controller (TBC+WBC), highlighting the advantages of the latter.

2.8.1 Gait Transitions and Disturbance Rejection

To achieve different speeds and gaits using the CT-SLIP template, the following procedure is employed. Over a specified duration t_p , the sweep angle of the virtual legs, denoted by Γ_s , is linearly increased. After this period, Γ_s is held constant. This gradual adjustment of Γ_s leads to an increase in the reference velocity for the robot, which is determined by equation 2.43.

$$\dot{x}_{ref} = l_0 \Gamma_s / t_s \quad (2.43)$$

While the cycle time t_c is kept constant, the stance time t_s is dynamically adjusted to enable transitions between gaits. Figure 2.15 shows the reference leg angle, t_s , and the corresponding reference velocity. It is important to note that although Γ_s remains constant, varying t_s also changes the reference velocity, as described by equation 2.43.

In Figure 2.15, the cycle time t_c is set to 0.5 seconds, and t_p is chosen as 3 seconds. During the first 10 seconds, the stance time t_s is 0.3 seconds. Since this value is greater than half of the cycle time, the gait corresponds to a walking trot with a full stance phase. After 10 seconds, t_s is reduced to 0.25 seconds, causing the gait to transition to a trotting motion. Finally, at 20 seconds, t_s is further reduced to 0.2 seconds, transitioning the gait to a running trot or flying trot.

The states corresponding to the reference values shown in Figure 2.15 are presented in Figure 2.16. Figure 2.16(a) shows the horizontal velocity of the robot, where it

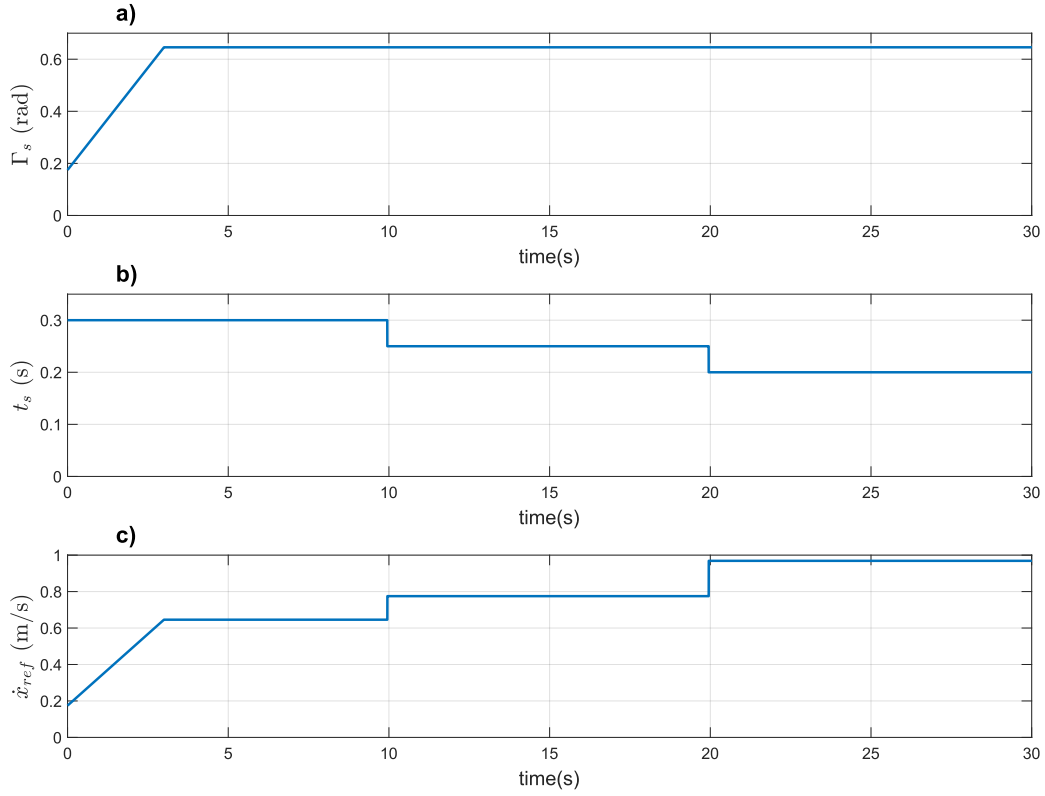


Figure 2.15: Clock parameters and corresponding reference body velocity. (a) Angle swept by the virtual leg Γ_s , (b) stance time t_s , (c) resulting reference forward velocity $\dot{x}_r$, for a cycle period $t_c = 0.5$ s.

can be observed that the robot tracks the desired velocity with a steady-state error. Figures 2.16(b) and 2.16(c) depict the robot's height and pitch angle, respectively. From these plots, it is evident that oscillations increase during the flying phase, which is expected due to the uncontrollable phase present in the running trot.

The next experiment evaluates the disturbance rejection performance of the controller across different gaits. In these experiments, Γ_s is kept constant. The cycle time is set to 0.5 seconds, while the stance times are chosen as 0.3, 0.25, and 0.2 seconds for walking trot, trot, and running trot, respectively. Due to the varying stance times, the robot's velocities differ, with velocities of 0.53 m/s, 0.61 m/s, and 0.75 m/s for the walking trot, trot, and running trot, respectively.

The disturbance is applied as a sudden jump in velocity in the y direction or lateral direction (normal to the sagittal plane). It is observed that the walking gait successfully

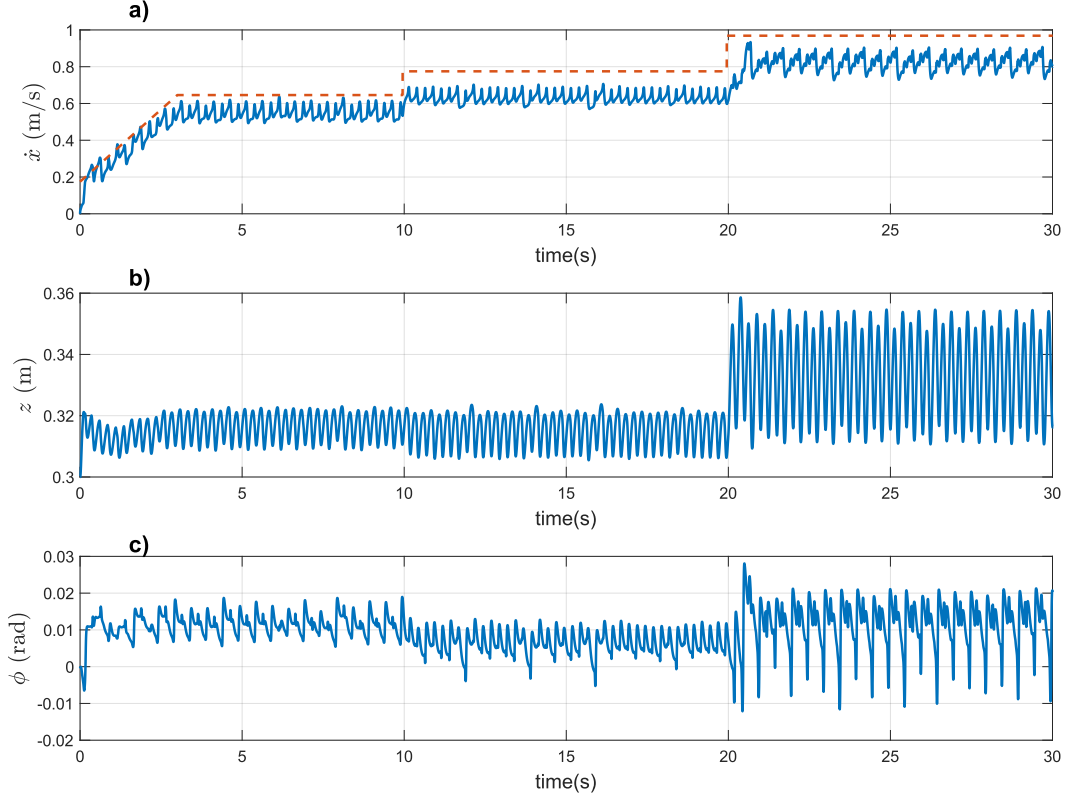


Figure 2.16: States vs time. (a) Horizontal velocity, (b) body height, and (c) pitch angle during three gait phases: walking trot (0–10 s), trot (10–20 s), and running trot (20–30 s).

recovers from disturbances up to 0.53 m/s, the trot gait recovers from disturbances up to 0.54 m/s, and the running trot gait recovers from disturbances up to 0.49 m/s. The states from these experiments are shown in Figure 2.17, where the moment of the disturbance is indicated by a dashed line, as clearly visible in Figures 2.17(c), 2.17(f), and 2.17(i). In each experiment, the robot was pushed with velocities of 0.53, 0.54, and 0.49 m/s for the walking trot, trot, and running trot, respectively, and it is evident that they successfully recovered from the disturbance and continued their motion.

2.8.2 Turning with and without compensation

This experiment assesses the effectiveness of rotating the swing foot plane in yaw control by comparing results with and without this compensation. During the experiment, the motion cycle time is set to 0.5 seconds, with a stance time of 0.25 seconds.

At the 3-second mark, the desired yaw angle is increased at a rate of 2.5 rad/s for a duration of 0.5 seconds to test the system's response. The $K_{\gamma,corr}$ is selected as 0.2. The resulting system states are shown in Figure 2.18.

In Figure 2.18, it is evident that the controller without compensation struggles to achieve the desired yaw rate of 2.5 rad/s, resulting in a loss of stability. This instability is visible in both the horizontal velocity (Figure 2.18a) and the height of the robot (Figure 2.18b), where the robot falls. By contrast, the controller with compensation maintains stability and successfully achieves the desired yaw rate, maintaining both horizontal velocity and height.

In further analysis, we observed that the controller without compensation could only reach a maximum yaw rate of approximately 2.1 rad/s under these settings. Meanwhile, the compensated controller was able to achieve up to 2.9 rad/s. This improvement in yaw rate demonstrates the effectiveness of the proposed swing foot plane rotation approach, providing increased stability and enhanced turning performance in legged systems.

2.8.3 Template-based Control vs. Whole-body Control Improved Template-based Control

The following experiments are done for measuring the effectiveness of the WBC by comparing TBC with TBC+WBC. The first experiment is done for measuring the performance improvement of the WBC for state tracking. Both of the controllers are commanded to same reference values and their states are given in Figure 2.19.

Figure 2.19 presents the tracking results for both the TBC and TBC+WBC controllers. In the first plot, both controllers track the desired horizontal velocity with some error. While the velocities are similar, TBC+WBC demonstrates greater stability, exhibiting lower variance compared to TBC. The steady-state error is attributed to system constraints, such as torque limitations and friction. In Figure 2.19(b), the robot's lateral (y-axis) position is shown, and TBC+WBC effectively reduces oscillations in this direction. Figure 2.19(c) displays the robot's height, where although both controllers perform comparably, TBC+WBC shows a smaller error. Lastly, Figure 2.19(d) illus-

trates the variation of K_{dr} over time. For TBC+WBC, the gain converges to a stable value with minimal oscillations, whereas for TBC, it exhibits larger oscillations and fails to converge to a specific value.

Another test that compares the controllers is disturbance rejection. In order to compare them, a push is applied to the robot while the robot moves at the same speeds. The template parameters are selected as $t_s = 0.2$ and $t_c = 0.3$ seconds and Γ_s increased linearly from $\pi/18$ to $(\pi/18 + \pi/10)$ in 2 seconds, then it is kept constant. After the robot walks for 4.5 seconds, a push in the lateral direction, normal to sagittal plane, is given by a velocity change of $1.22m/s$. This test helps to evaluate how well each controller handles disturbances.

Figure 2.20 shows the controller's performance in terms of disturbance rejection. The push applied to the robot is shown in the fourth plot. In the TBC case, the robot cannot recover from the push and falls down. However, the TBC+WBC controller successfully recovers from the push, demonstrating its robustness. This indicates that the whole body control improved TBC is more effective at handling disturbances compared to the TBC. It is observed that TBC can recover from lateral pushes up to $1.21m/s$ and TBC+WBC can recover from pushes up to $1.27m/s$.

2.9 Hardware Experiments

The Unitree GO1 robot is equipped with various hardware components to support its operation and enhance its performance. A Raspberry Pi 4 serves as the locomotion controller, providing computational power for real-time motion control. A 12 V fan is installed for cooling purposes, ensuring the hardware operates within optimal temperature ranges during extended use. A Pixhawk 4 is used to obtain precise accelerometer readings, as its sensors provide higher accuracy compared to the robot's built-in accelerometer. Additionally, a power bank is included to supply power to the Raspberry Pi, ensuring uninterrupted operation. Figure 2.21 illustrates the hardware setup on the Unitree GO1 robot (a) and the robot in its standing posture (b).

The experiment video can be viewed at the following link.

<https://youtu.be/Wq5bicvUTgA>

<https://youtu.be/dIRJy16wcts>

The control parameters are given in Table 2.2. The damping gain in the radial direction $K_{d\tau}$ is selected by virtual tuning. Similar approach is used in [71] in order to tune the damping constant of the leg in the SLIP template. K_d^* is selected as 40. Roll, pitch and yaw axes parameters are given with subscript ψ, ϕ, γ . While the legs are in the swing phase, they are tracked with a PD controller and $K_{pfa}, K_{dfa}, K_{pfh}, K_{dfh}, K_{pfk}, K_{dfk}$ are P and D gains for the abduction, hip and knee joints. The weight matrix for the controller is selected as $Q^{tmp} = blkdiag(50, 10, 10, 1, 5, 5)$. The Coulomb friction coefficient is selected as 0.2. The state estimation operates at 600 Hz, while the controller runs at 300 Hz.

Table 2.2: Parameters for the controller.

Parameter	Value	Unit	Parameter	Value	Unit
K_{ps}	1000	N/m	K_{ds}	50	Ns/m
$K_{p\tau}$	40	Nm	$K_{d\tau}$	$10 + K_d^*(\dot{x}_{ref} - \dot{x})$	Nms
K_{py}	850	N/m	K_{dy}	20	Ns/m
$K_{p\psi}$	50	Nm	$K_{d\psi}$	2	Nms
$K_{p\phi}$	500	Nm	$K_{d\phi}$	5	Nms
$K_{p\gamma}$	40	Nm	$K_{d\gamma}$	8	Nms
K_{pfa}	7	Nm	K_{dfa}	0.15	Nms
K_{pfh}	7	Nm	K_{dfh}	0.35	Nms
K_{pfk}	4.25	Nm	K_{dfk}	0.25	Nms

Figures 2.22 and 2.23 illustrate the experimental results conducted on the robot. Each figure is divided into three distinct regions, corresponding to the walking trot, regular trot, and running trot phases of the robot's gait. The parameter Γ_s is set to 14 degrees, with a cycle time of 0.6 seconds. The stance time for each virtual leg is 0.4 seconds for the walking trot, 0.3 seconds for the regular trot, and 0.27 seconds for the running trot. In Figure 2.22, subplots (a) and (b) depict the horizontal position and velocity of the robot over time. In subplot (b), both the raw (green dashed line)

and filtered (blue solid line) velocity data are presented, with a moving average filter applied to reduce noise and better visualize the robot's behavior. The filtered velocity highlights smoother transitions and variations across the different gait phases. Subplots (c) and (d) illustrate the vertical position and velocity of the robot. Similar to the horizontal velocity, subplot (d) includes both raw and filtered vertical velocity data. During the running trot phase, the magnitude of the vertical velocity increases significantly, indicating more pronounced vertical motion compared to the other phases. This observation reflects the expected dynamics of a running gait, where the robot exhibits greater vertical oscillations. The division of the figure into regions emphasizes the distinct characteristics of each gait phase, demonstrating how the robot's motion evolves across different locomotion modes.

In Figure 2.23, the roll, pitch, and yaw angles, along with the lateral displacement of the robot, are illustrated over time. The data clearly demonstrates that during the running trot phase, the oscillations in all three angular measurements (roll, pitch, and yaw) exhibit a noticeably higher amplitude compared to other phases of motion. Similarly, the lateral displacement shows significant variation, indicating increased dynamic instability or lateral sway. These larger oscillations during the running trot highlight the higher energy and dynamic nature of this gait, as opposed to more stable or controlled movements in other phases, which display comparatively smaller magnitudes of oscillation.

Figure 2.24 demonstrates the walking trot gait of the Unitree GO1 robot as observed during the hardware experiments. The sequence of images captures the robot's leg movements, showcasing the characteristic diagonal leg pairing of the trot gait, where a front leg on one side and a hind leg on the opposite side move in unison. This gait provides stability and dynamic balance during locomotion.

When the robot moves at an average horizontal velocity of approximately 0.35 m/s, its mechanical cost of transport is calculated as 0.7. In comparison, the GO1's default algorithm has a cost of transport of 1.94 at a similar speed. While this suggests improved efficiency, a direct comparison is challenging due to differences in factors such as swing leg trajectories and step frequencies.

2.9.1 Comparison with Simulation

To evaluate the consistency of the proposed control approach between simulation and real-world deployment, we conducted experiments under fully matched conditions. Both the simulation and the hardware trials were initialized with identical gait parameters, desired velocity, initial posture, and control gains. The same reference gait timing and CT-SLIP model parameters were applied in both cases. By ensuring a one-to-one match in all relevant settings, any differences observed between simulation and hardware results can be attributed to physical factors such as unmodeled dynamics, sensor noise, and real-world disturbances, rather than differences in control configuration.

To further assess the consistency between the simulation and real-world implementation, Figure 2.25 presents time series plots of key state variables. In subplot (a), the real robot’s forward velocity $\dot{x}$ exhibits greater variability due to sensor noise and unmodeled dynamics, while the simulation shows smoother, more regular oscillations. Nevertheless, both follow the same overall trend. Subplot (b) compares the vertical position z , where the real robot demonstrates minor fluctuations due to ground interactions, whereas the simulation maintains a consistent pattern. Subplot (c) shows the pitch angle ϕ , where both domains remain within a narrow range and oscillate around zero. The close match in frequency and amplitude of these oscillations indicates that the proposed controller maintains similar behavior in both the simulation and the real robot, despite differences in environmental complexity.

Table 2.3 presents the Root Mean Square Error (RMSE) values for three key body-level states: forward velocity, vertical CoM height, and pitch angle. Each quantity is compared across three pairings: real robot vs. reference, simulation vs. reference, and real robot vs. simulation. As expected, the simulation tracks the reference trajectory slightly more closely than the hardware due to the absence of real-world disturbances. However, the error between real and simulated signals remains low across all metrics, indicating that the simulation accurately captures the system’s behavior under the same control policy.

Table 2.4 reports the RMSE of stance durations for each leg, measured with respect to

Table 2.3: RMSE comparison between real robot, simulation, and reference trajectories.

Metric	Real vs Ref	Sim vs Ref	Real vs Sim
X Velocity (m/s)	0.0543	0.0590	0.0904
Z Height (m)	0.0198	0.0124	0.0077
Pitch Angle (rad)	0.0048	0.0058	0.0059

the reference gait timing. The real robot exhibits slightly higher variability in stance time, likely caused by mechanical compliance, small contact inconsistencies, and sensor quantization effects. In contrast, the simulation results are tightly aligned with the reference schedule due to the deterministic nature of the simulated environment.

Table 2.4: RMSE of stance duration deviations for each leg compared to the reference gait timing.

Leg	Real Robot RMSE (s)	Simulation RMSE (s)
Front Right	0.0252	0.0021
Front Left	0.0202	0.0020
Rear Right	0.0200	0.0020
Rear Left	0.0231	0.0021

Overall, the RMSE values in both state and gait timing comparisons are closely matched between the real robot and the simulation. The minor discrepancies can be attributed to unmodeled dynamics, actuator friction, sensor noise, and other uncertainties inherent to real-world operation. Despite these, the proposed control framework demonstrates robust and consistent performance across both domains.

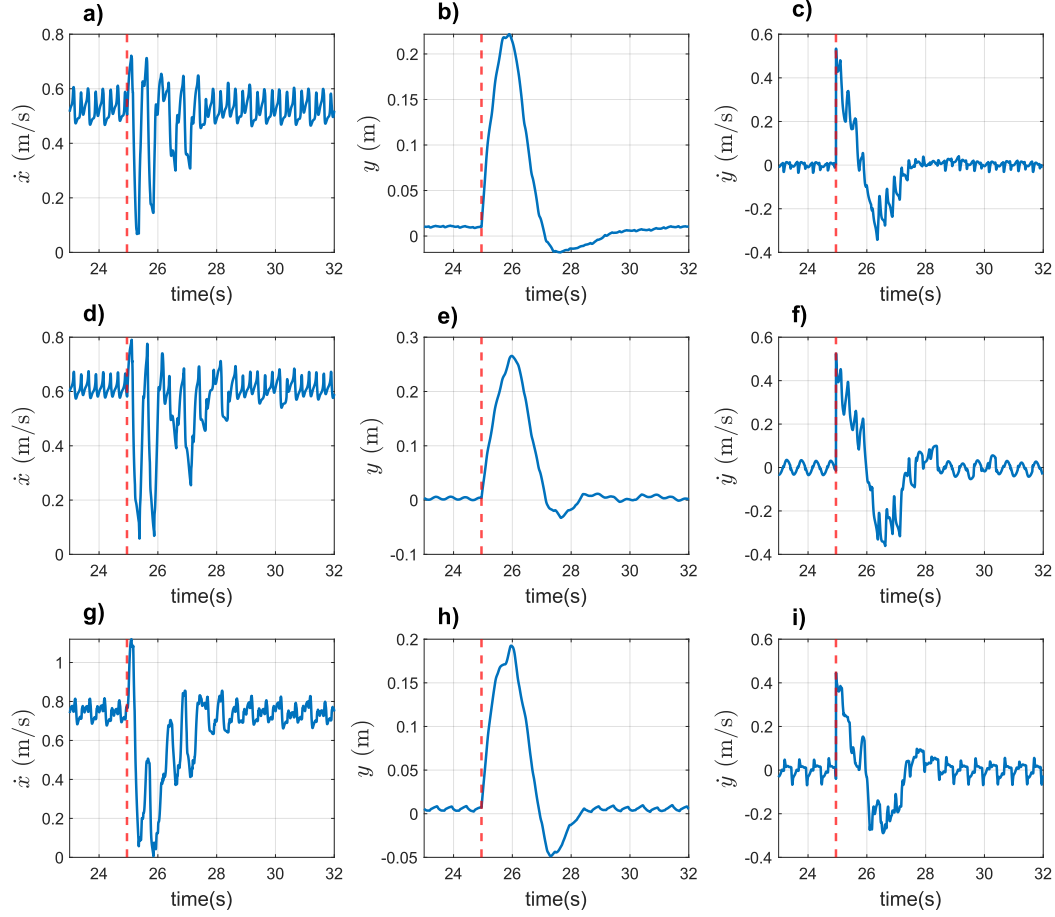


Figure 2.17: Disturbance rejection performance under different gaits, *a*) horizontal velocity of the robot vs time (walking trot), *b*) *y* position of the robot vs time (walking trot), *c*) *y* direction velocity of the robot vs time ((walking trot), *d*) horizontal velocity of the robot vs time (trot), *e*) *y* position of the robot vs time (trot), *f*) *y* direction velocity of the robot vs time (trot), *g*) horizontal velocity of the robot vs time (running trot), *h*) *y* position of the robot vs time (running trot), *i*) *y* direction velocity of the robot vs time (running trot)

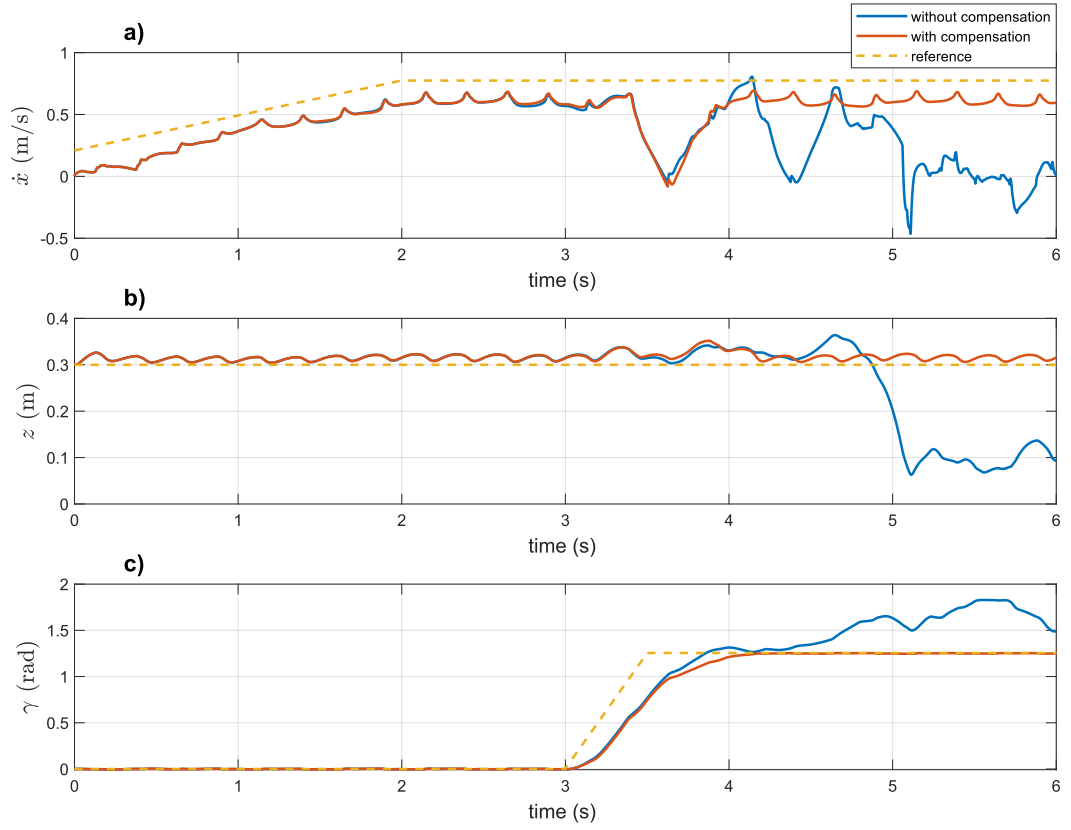


Figure 2.18: Comparison between compensation and no-compensation cases. (a) Horizontal velocity in the body frame, (b) height of the robot, (c) yaw angle, showing the stabilizing effect of the compensation method.

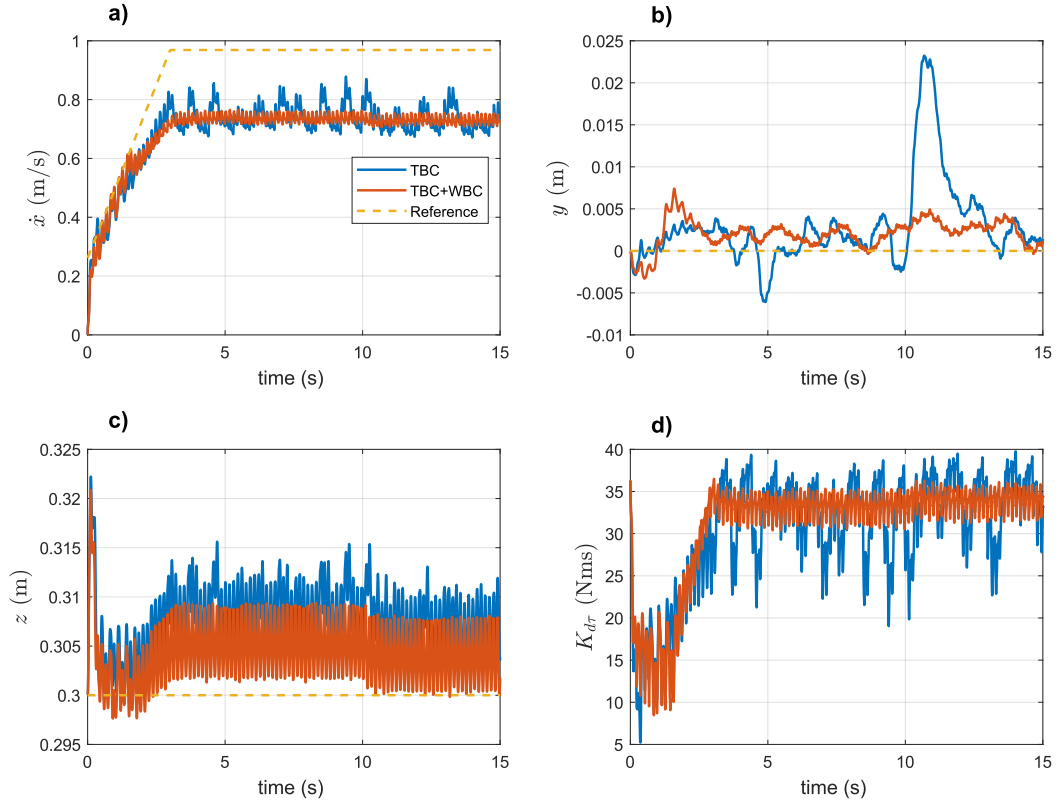


Figure 2.19: Comparison between TBC and TBC+WBC based on tracking performance: *a)* horizontal velocity vs time, *b)* y position vs time, *c)* z position vs time, *d)* Damping constant for the torque of the virtual leg vs time.

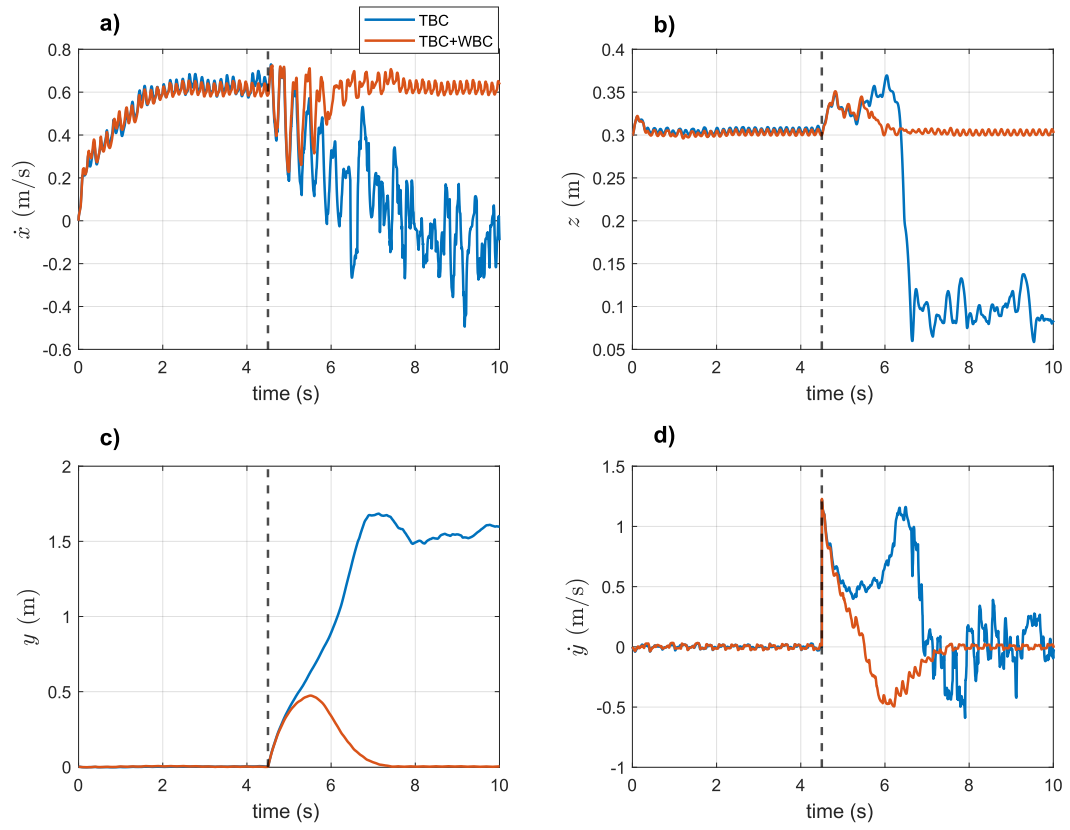


Figure 2.20: Comparison between TBC and TBC+WBC based on disturbance rejection *a)* horizontal velocity vs time *b)* robot's body height vs time, *c)* robot's y distance vs time, *d)* robot's y velocity vs time

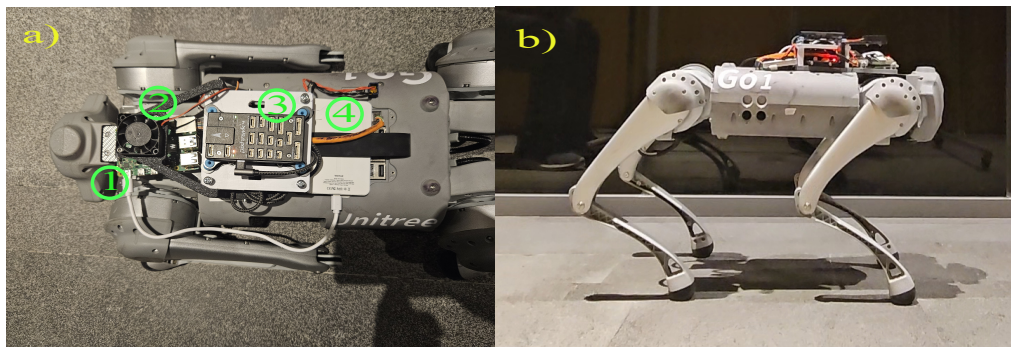


Figure 2.21: Hardware components for the experiments: *a)* Hardware components on the Unitree GO1 robot, 1) Raspberry Pi 4, 2) 12 V fan, 3) Pixhawk 4, 4) Powerbank, *b)* Robot Standing

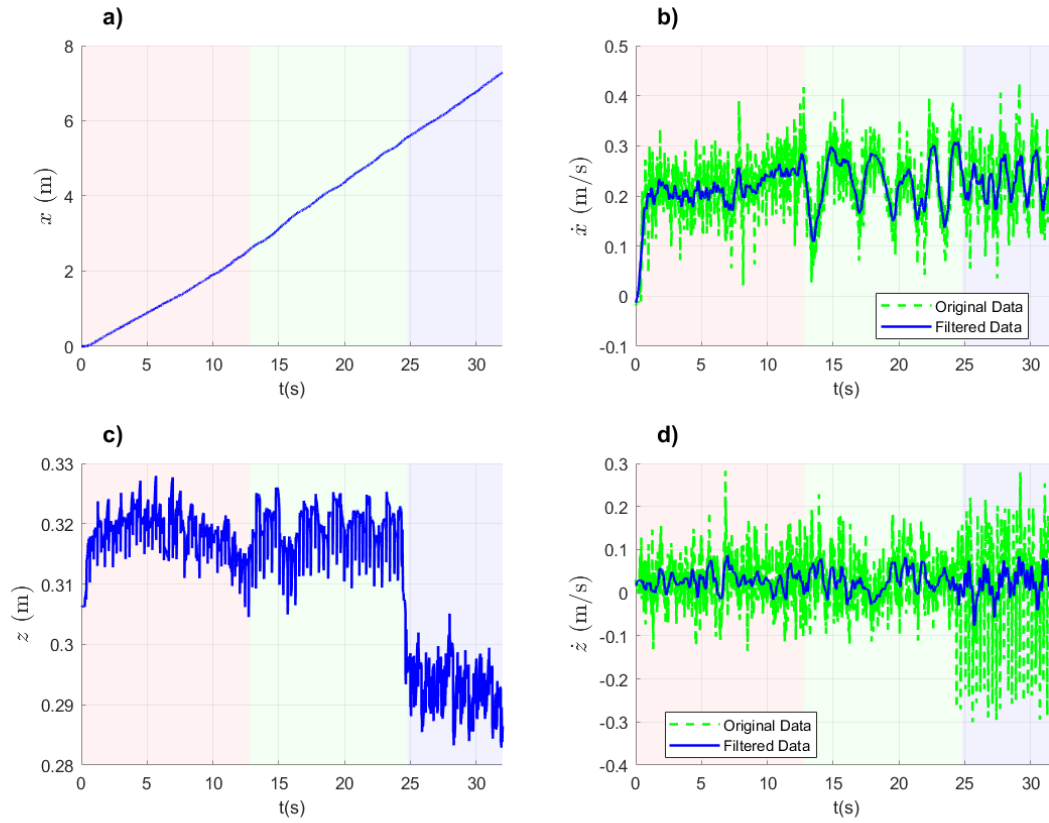


Figure 2.22: States of the robot. (a) Horizontal position, (b) horizontal velocity, (c) vertical position, (d) vertical velocity over three gait phases: walking trot (0–10 s), trot (10–20 s), and running trot (20–30 s).

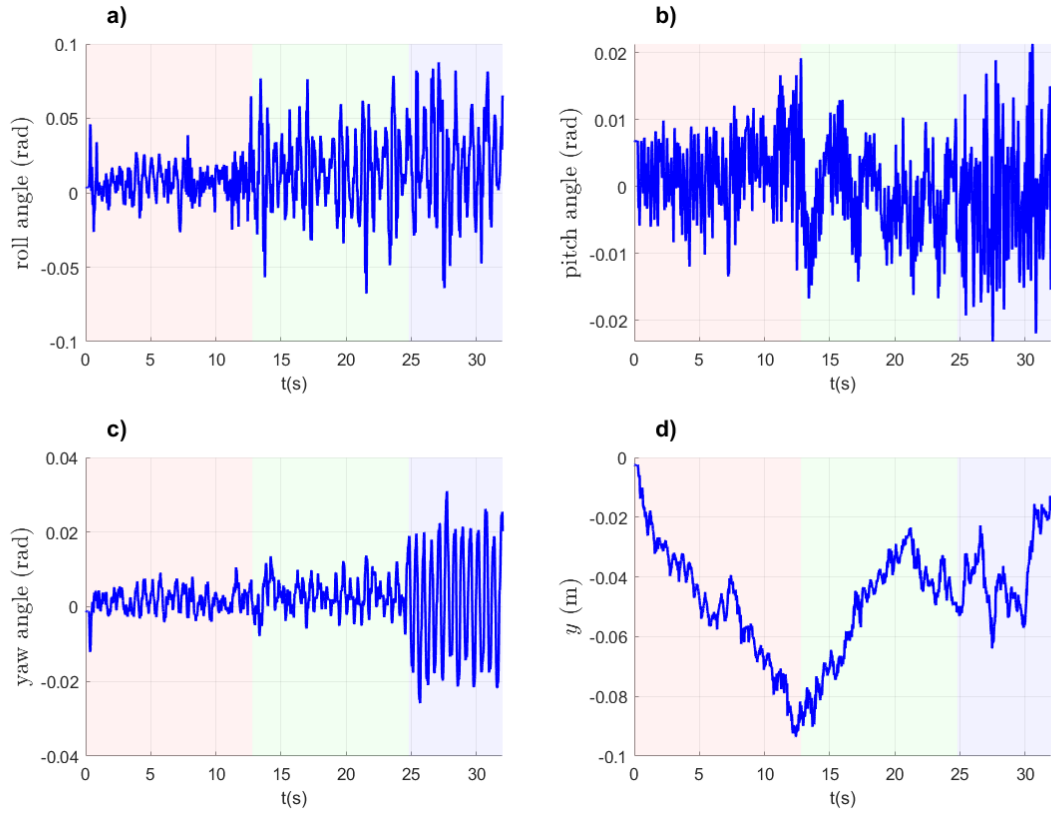


Figure 2.23: States of the robot. (a) Roll angle, (b) pitch angle, (c) yaw angle, (d) lateral position, over three gait phases: walking trot (0–10 s), trot (10–20 s), and running trot (20–30 s).

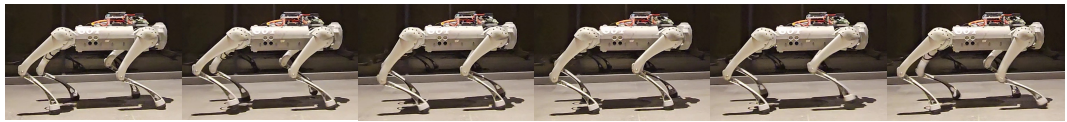


Figure 2.24: Walking trot hardware experiment. Snapshots from the robot performing a walking trot on hardware.

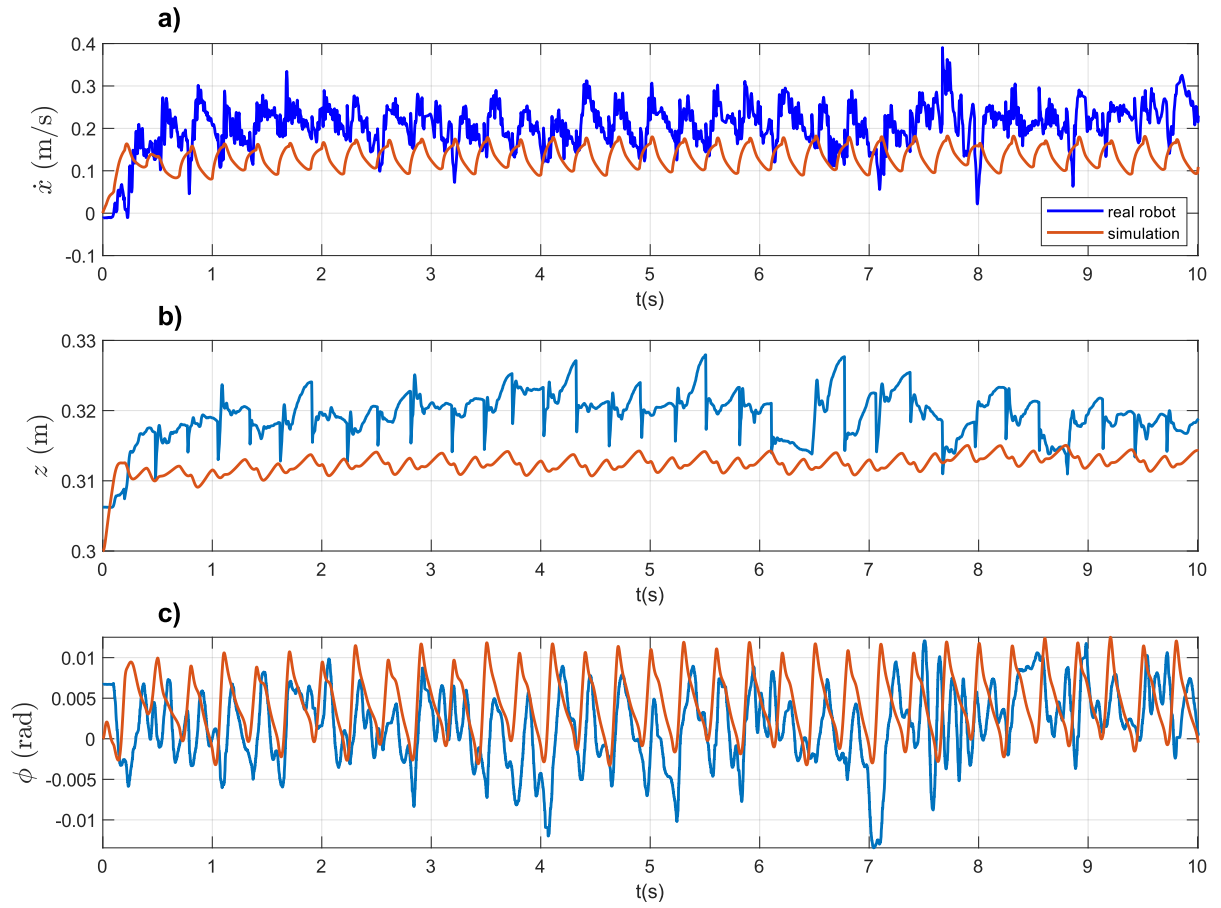


Figure 2.25: Comparison of state trajectories between the real robot and simulation over a 10-second locomotion trial. (a) Forward velocity $\dot{x}$, (b) vertical position z , and (c) pitch angle ϕ .

CHAPTER 3

TEMPLATE-BASED QUADRUPEDAL PRONKING: ANALYSIS AND EXTENSION WITH MODEL PREDICTIVE CONTROL

In this chapter, we propose a control approach for achieving pronking motion in quadrupedal robots. Pronking is a gait characterized by the simultaneous and symmetric extension of all four legs, resulting in vertical jumps where the robot experiences a flight phase with minimal rotational motion. The chapter begins with the introduction of a three-dimensional spring-damper template, which serves as the foundation for generating the desired behavior. Building upon this model, a model predictive controller (MPC) is formulated to manage the dynamics during the stance phase. To support this control structure, an approximate analytical footstep planner is developed. The chapter concludes with simulation studies and hardware experiments that validate the effectiveness of the proposed method.

3.1 Spring-Damper Template

The template used for stance motion is given in Figure 3.1, in each rotational (ψ, ϕ, γ) and linear (x, y, z) axis there are springs and dampers. The spirals represent the rotational springs and dampers. The template used in [37] is similar, but it only incorporates linear springs and dampers. For each axis, the applied force or moment is calculated as equation 3.1. In the equation, the generalized coordinates for the body are defined as $\zeta = [\psi, \phi, \gamma, x, y, z]^\top$, $K_{p\zeta}$ and $K_{d\zeta}$ are the diagonal matrices for the springs and damping constants, ζ_{des} and $\dot{\zeta}_{des}$ are the desired values for the states.

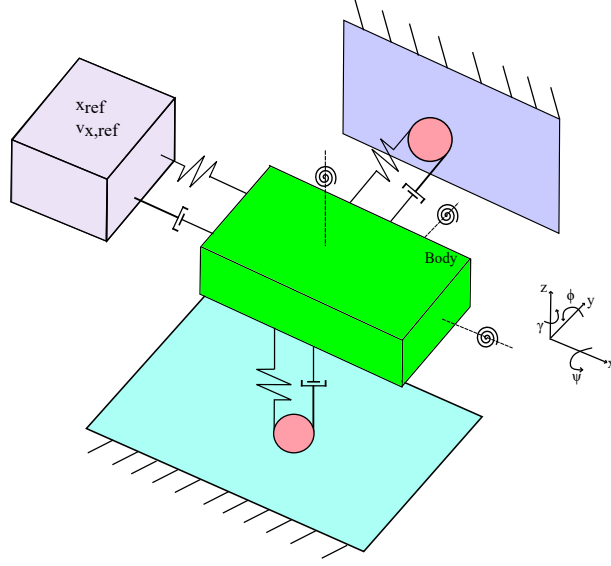


Figure 3.1: Template for stance motion. The stance-phase template shows the force elements used to generate desired body forces, along with the reference inputs x_{ref} and $v_{x,\text{ref}}$ that regulate forward motion.

$$\mathbf{F}_\zeta = \mathbf{K}_{p\zeta}(\zeta_{des} - \zeta) + \mathbf{K}_{d\zeta}(\dot{\zeta}_{des} - \dot{\zeta}) \quad (3.1)$$

3.2 Motion Generation From Template

In the literature, the templates are used for achieving a locomotion strategy in legged systems. The embedding of a template is used for finding the required ground reaction forces (GRFs), then converting them to joint torques [10], [72]. The template-based controllers do not predict the future, they just control the system for the given time instant. In this study, the template is used solely to generate reference motions for the robot, not for direct control. The reference motions are generated at the same frequency as the controller meaning that at each controller time step, the reference motions for future times are regenerated using the template. This makes the controller not rely on any prespecified trajectory, and by updating the references frequently, the controller can handle disturbances very successfully as explained in the upcoming sections. A different, specifically designed controller is then used to track the generated reference motion. For generating the reference, equation 3.1 is used for each

axis to compute the accelerations.

$$\ddot{\psi}_k = \frac{1}{I_\psi} [K_{p\psi}(\psi_{des} - \psi_k) + K_{d\psi}(\dot{\psi}_{des} - \dot{\psi}_k)] \quad (3.2)$$

$$\ddot{\phi}_k = \frac{1}{I_\phi} [K_{p\phi}(\phi_{des} - \phi_k) + K_{d\phi}(\dot{\phi}_{des} - \dot{\phi}_k)] \quad (3.3)$$

$$\ddot{\gamma}_k = \frac{1}{I_\gamma} [K_{p\gamma}(\gamma_{des} - \gamma_k) + K_{d\gamma}(\dot{\gamma}_{des} - \dot{\gamma}_k)] \quad (3.4)$$

$$\ddot{x}_k = \frac{1}{m} [K_{px}(x_{des} - x_k) + K_{dx}(\dot{x}_{des} - \dot{x}_k)] \quad (3.5)$$

$$\ddot{y}_k = \frac{1}{m} [K_{py}(y_{des} - y_k) + K_{dy}(\dot{y}_{des} - \dot{y}_k)] \quad (3.6)$$

$$\ddot{z}_k = \frac{1}{m} [K_{pz}(z_{des} - z_k) + K_{dz}(\dot{z}_{des} - \dot{z}_k) - mg] \quad (3.7)$$

In equations 3.2, 3.3, 3.4, 3.5, 3.6, 3.7, m is the mass of the robot, g is the gravitational acceleration. I_ψ, I_ϕ, I_γ are the inertias in roll, pitch, and yaw axes, respectively.

To predict future position trajectories, second-order Taylor approximation is used, and to predict future velocity trajectories, first-order Taylor approximation is used. The update equations for ζ and $\dot{\zeta}$ are given in equations 3.8 and 3.9. In these equations, T represents the sampling time.

$$\zeta_{k+1} = \zeta_k + \dot{\zeta}_k T + \ddot{\zeta}_k \frac{T^2}{2} \quad (3.8)$$

$$\dot{\zeta}_{k+1} = \dot{\zeta}_k + \ddot{\zeta}_k T \quad (3.9)$$

After every time update in the prediction horizon, the accelerations are computed again using the template, and new reference states for the next time are calculated.

3.3 Controller Design

For a quadrupedal robot that has 3 actuators in each leg, actuating ab/ad, hip, and knee joints, there are 18 generalized coordinates, as given in equation 3.10. The first 6 of them are the coordinates defining the pose of the trunk and the other 12 are the joint

angles. In this system, we want to control the trunk pose, so the controller design will focus on the center of mass (COM) coordinates.

$$\mathbf{q} = [\psi \ \phi \ \gamma \ x \ y \ z \ \theta_{10} \ \theta_{11} \ \theta_{12} \ \theta_{20} \ \theta_{21} \ \theta_{22} \ \theta_{30} \ \theta_{31} \ \theta_{32} \ \theta_{40} \ \theta_{41} \ \theta_{42}]^\top \quad (3.10)$$

While formulating the optimal control problems in literature, first the reference states are computed, then the controller tries to follow these states optimally [14], [53]. In this paper, the controller directly controls the robot using the generalized coordinate accelerations for the COM rather than the robot's states. This approach reduces tuning parameters from 12 to 6 for the optimal control problem. The MPC formulation is given in equations 3.11, 3.12, 3.13, 3.14. The $\boldsymbol{\rho}$ vector is defined as $\boldsymbol{\rho} = [\psi \ \phi \ \gamma \ x \ y \ z]^\top$. $\mathbf{A}$ matrix is the mass matrix for the COM coordinates and its size is 6×6 .

$$\min_{\mathbf{u}} \sum_{i=0}^{k-1} \|\mathbf{A}\ddot{\boldsymbol{\rho}}_i - \mathbf{A}\ddot{\boldsymbol{\rho}}_{i,ref}\|_Q \quad (3.11)$$

$$\text{s.t.} \quad \boldsymbol{\rho}_{i+1} = g(\boldsymbol{\rho}_i, \mathbf{u}_i) \quad (3.12)$$

$$\mathbf{B}\mathbf{u}_i \leq \mathbf{v}_u \quad (3.13)$$

$$\mathbf{u}_{min} \leq \mathbf{u}_i \quad (3.14)$$

The cost function is the Q-norm of the difference between the desired generalized coordinate accelerations and the computed generalized coordinate accelerations for the COM, where the Q norm is defined as $\|\mathbf{x}\|_Q = \mathbf{x}^\top \mathbf{Q} \mathbf{x}$. In this problem, the input vector consists of ground reaction forces, since the center of mass accelerations are linear combinations of these forces if the legs are assumed to be massless and the rotational states for COM are known. The first equality constraint, equation 3.12, is for the dynamic state update. The second constraint, equation 3.13, is for not violating torque limits and the found solution is in the friction cone, and the third constraint, equation 3.14, is for the solution to be feasible (if forces on the z axis are negative, then the leg lifts off, so we do not have ground reaction forces).

In the stance phase, we assume the legs are massless to simplify the calculations. During this phase, our goal is to control the pose of the center of mass, so the leg states

are not controlled, they are only used in the computation of the Jacobian matrix. With massless legs, the GRFs directly affect the COM states. This simplification allows the system to be represented as shown in Figure 3.2.

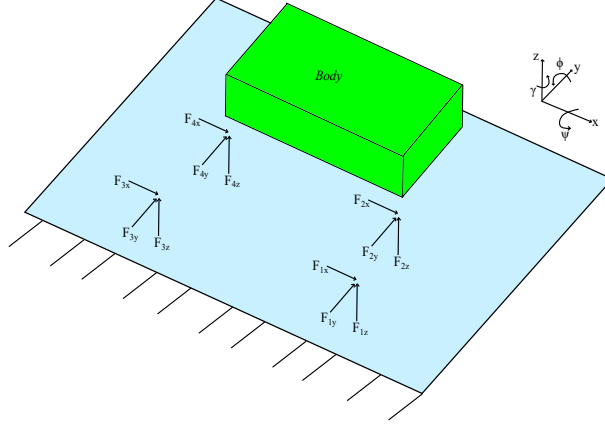


Figure 3.2: Simplified system for the controller. The body is modeled with four contact points, each applying 3D ground reaction forces F_{ix}, F_{iy}, F_{iz} , which are used by the controller to regulate the robot's motion.

For this system, we need to compute how GRFs affect the COM accelerations. Luckily, the transformation from GRFs to the COM accelerations can be extracted from the transpose of the Jacobian of the system; therefore, we can directly work with the Jacobian as in equation 3.15. Even though the given template is uncoupled, controlling the quadruped involves coupled dynamics. This coupling is described by the system's Jacobian.

For this system, the size of the transpose of the Jacobian matrix is 18×12 . In equation 3.15, the $\mathbf{J}_{red}^\top$ corresponds to the reduced transpose of the Jacobian where only the first 6 rows are used. $\mathbf{F}$ is defined as $\mathbf{F} = [F_{1x} F_{1y} F_{1z} F_{2x} F_{2y} F_{2z} F_{3x} F_{3y} F_{3z} F_{4x} F_{4y} F_{4z}]^\top$. $\mathbf{G}$ matrix is the gravitational force matrix defined as $\mathbf{G} = [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ mg]^\top$.

We have 6 COM accelerations and 12 GRFs, so the size of $\mathbf{J}_{red}^\top$ is 6×12 . The Jacobian can be found in the relationship between foot tip velocities and generalized coordinate velocities. For this system, it is computed symbolically using MATLAB.

$$\mathbf{A}\ddot{\mathbf{p}} + \mathbf{G} = \mathbf{J}_{red}^\top \mathbf{F} \quad \rightarrow \quad \mathbf{A}\ddot{\mathbf{p}}_k + \mathbf{G} = \mathbf{J}_{red}^\top \mathbf{u}_k \quad \rightarrow \quad \mathbf{C}_k = \mathbf{J}_{red}^\top \mathbf{u}_k \quad (3.15)$$

The first 3 rows of $\mathbf{J}_{red}^\top$ correspond to the rotational coordinates and depend on the

rotational states of the COM, making the system non-linear. However, in our specific scenario, we can introduce additional assumptions to linearize the system. By using Euler stepping to calculate future Jacobian matrices, we can linearize the system. This transforms the system into a quadratic programming problem, allowing us to use rapid and convex optimization techniques, which improves the overall system performance.

For the first time step of the MPC horizon, we know the COM states and leg states, so we can directly compute the Jacobian. For future time steps, we use Euler stepping to estimate the Jacobians. To apply this linearization, we need to calculate the derivative of the transpose of the Jacobian. Equation 3.16 shows the chain rule for computing the derivative of $\mathbf{J}_{red}^\top$ for each row.

$$\dot{\mathbf{J}}_{red}^\top(j) = \sum_{i=0}^{17} \frac{\partial \mathbf{J}_{red}^\top(j)}{\partial \mathbf{q}_i} \dot{\mathbf{q}}_i \quad (3.16)$$

After calculating the $\mathbf{J}_{red}^\top$, we can compute the Jacobian estimates for the future states as in equation 3.17.

$$\mathbf{J}_{red,i|0}^\top = \mathbf{J}_{red,0|0}^\top + iT \dot{\mathbf{J}}_{red}^\top \quad (3.17)$$

Horizon given as n , the system can be modeled like equation 3.18.

$$\underbrace{\begin{bmatrix} \mathbf{C}_{0|0} \\ \mathbf{C}_{1|0} \\ \vdots \\ \mathbf{C}_{n-1|0} \end{bmatrix}}_{\mathbf{X}} = \underbrace{\begin{bmatrix} \mathbf{J}_{red,0|0}^\top & 0 & 0 & \dots & 0 \\ 0 & \mathbf{J}_{red,1|0}^\top & 0 & \dots & 0 \\ 0 & 0 & \mathbf{J}_{red,2|0}^\top & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & 0 \\ 0 & 0 & 0 & \vdots & \mathbf{J}_{red,n-1|0}^\top \end{bmatrix}}_{\mathbf{G}} \underbrace{\begin{bmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \\ \vdots \\ \mathbf{u}_{n-1} \end{bmatrix}}_{\mathbf{U}} \quad (3.18)$$

The reference motion equation is given in equations 3.19 and 3.20. Then, the cost function can be written in the form of equation 3.21. In equation 3.21, $\bar{\mathbf{Q}}$ is defined as $\bar{\mathbf{Q}} = \text{blkdiag}(\mathbf{Q}, \mathbf{Q}, \dots, \mathbf{Q})$.

$$\ddot{\boldsymbol{\rho}}_{k,ref} = \begin{bmatrix} \ddot{\psi}_{k,ref} & \ddot{\phi}_{k,ref} & \ddot{\gamma}_{k,ref} & \ddot{x}_{k,ref} & \ddot{y}_{k,ref} & \ddot{z}_{k,ref} \end{bmatrix}^\top \quad (3.19)$$

$$\mathbf{X}_{ref} = \begin{bmatrix} (\mathbf{A}\ddot{\boldsymbol{\rho}}_{0,ref} + \mathbf{G})^\top & (\mathbf{A}\ddot{\boldsymbol{\rho}}_{1,ref} + \mathbf{G})^\top & \dots & (\mathbf{A}\ddot{\boldsymbol{\rho}}_{n,ref} + \mathbf{G})^\top \end{bmatrix}^\top \quad (3.20)$$

$$\begin{aligned}
J(U) &= \|X - X_{ref}\|_Q = (GU - X_{ref})^\top \bar{Q} (GU - X_{ref}) \\
&= U^\top G^\top \bar{Q} G U - 2 X_{ref}^\top \bar{Q} G U + X_{ref}^\top \bar{Q} X_{ref} \\
&= U^\top M U - 2 \alpha^\top U + C
\end{aligned} \tag{3.21}$$

Where, $M = G^\top \bar{Q} G$, $\alpha = X_{ref}^\top \bar{Q} G$, and $C = X_{ref}^\top \bar{Q} X_{ref}$.

After formulating the cost function, the constraints can be formulated as well. For the constraint formulation given in equation 3.13, we need to define the relationship between GRFs (inputs), and the joint torques. This relationship again is contained in the Jacobian. In equation 3.22, $J_{red2}^\top$ is comprised of the last 12 rows of the jacobian transpose, Γ is the joint torque vector defined as $\Gamma = [\tau_{10} \ \tau_{11} \ \tau_{12} \ \dots \ \tau_{40} \ \tau_{41} \ \tau_{42}]^\top$. Since we assumed that the legs are massless, we can directly write this equation without considering the leg accelerations and Coriolis terms. The torque constraint that is applied to the controller is given in equation 3.23, where τ_{max} is the maximum torque that the joint motors can supply.

$$\Gamma = -J_{red2}^\top F \tag{3.22}$$

$$-\tau_{max} \mathbf{1} \leq \Gamma \leq \tau_{max} \mathbf{1} \tag{3.23}$$

Again, similar to equation 3.17, we can calculate the derivative of $J_{red2}^\top$ as in equation 3.24. Horizon defined as n , the system can be modeled as the equation 3.25.

$$J_{red2,i|0}^\top = J_{red2,0|0}^\top + iT J_{red2}^\top \tag{3.24}$$

$$\underbrace{\begin{bmatrix} \Gamma_{0|0} \\ \Gamma_{1|0} \\ \vdots \\ \Gamma_{n-1|0} \end{bmatrix}}_N = \underbrace{\begin{bmatrix} -J_{red2,0|0}^\top & 0 & 0 & \dots & 0 \\ 0 & -J_{red2,1|0}^\top & 0 & \dots & 0 \\ 0 & 0 & -J_{red2,2|0}^\top & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & 0 \\ 0 & 0 & 0 & \vdots & -J_{red2,n-1|0}^\top \end{bmatrix}}_L \underbrace{\begin{bmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \\ \vdots \\ \mathbf{u}_{n-1} \end{bmatrix}}_U \tag{3.25}$$

$$-\bar{\tau}_{max} \leq N \leq \bar{\tau}_{max} \rightarrow -\bar{\tau}_{max} \leq LU \leq \bar{\tau}_{max} \tag{3.26}$$

Finally, we can write the constraint equation in 3.26, where $\bar{\tau}_{max} = ones(12 \times horizon, 1) \tau_{max}$.

The constraint in Equation 3.14 ensures the feasibility of the GRFs or inputs. Specifically, it requires that the forces on the z-axis must not be negative, as negative values

indicate no contact and, therefore, no GRFs. The formulation is detailed in equation 3.27.

$$\bar{\mathbf{u}}_{min} \leq \mathbf{U} \quad (3.27)$$

Where, $\mathbf{u}_{min} = \begin{bmatrix} -\infty & -\infty & 0 & -\infty & \dots & -\infty & -\infty & 0 \end{bmatrix}^\top$ and $\bar{\mathbf{u}}_{min} = \begin{bmatrix} \mathbf{u}^\top & \mathbf{u}^\top & \dots & \mathbf{u}^\top \end{bmatrix}^\top$.

Also, the found GRFs should lie in the friction cone, for this the friction cone is linearized as an octagonal pyramid and given as equation 3.28. The $\mathbf{D}$ matrix has size $(32 \times horizon) \times (12 \times horizon)$.

$$\mathbf{D}\mathbf{U} \leq \mathbf{0} \quad (3.28)$$

The MPC takes the following optimization problem form;

$$\min \mathbf{U}^\top \mathbf{M}\mathbf{U} - 2\alpha^\top \mathbf{U} + \mathbf{C} \quad (3.29)$$

subjected to

$$-\bar{\tau}_{max} \leq \mathbf{L}\mathbf{U} \leq \bar{\tau}_{max} \quad (3.30)$$

$$\mathbf{D}\mathbf{U} \leq \mathbf{0} \quad (3.31)$$

$$\bar{\mathbf{u}}_{min} \leq \mathbf{U} \quad (3.32)$$

The equation 3.29 is the cost function in equation 3.21. $\mathbf{U}$ is the input to the system. $\mathbf{M}$, α and $\mathbf{C}$ is formulated in equation 3.21. The objective function is convex because $\mathbf{M}$ is positive definite. The inequality constraints are affine. Therefore, the problem turns into a quadratic programming problem. Here the input $\mathbf{U}$ is GRFs; however, the GRFs cannot given to the robot as inputs directly. They first need to be converted to the joint torques via the equation $\mathbf{\Gamma} = -\mathbf{J}_{red2}^\top \mathbf{F}$, then they will be fed to the robot as the inputs.

In the flight phase of the robot, a PD controller is applied to the joint motors. The method for determining the reference angles is explained in the following sections.

3.4 Approximate Analytical Foot Step Controller

3.4.1 Formulating The Model

Determining where the robot should place its feet during touch-down is a complex challenge. In the literature, there is a heuristic foot placement controller known as the Raibert footstep controller [38]. Although it was stated by Raibert a long time ago, many of today's controllers still use it [73], [74], [75]. For trotting gait and similar gaits where the stance phase is symmetric, this controller gives good results; however, for pronking gait where the stance phase is asymmetric, this controller performs poorly as we see in the experimental results. Additionally, this footstep placement controller is designed for velocity tracking, making it challenging to control the apex height with this controller.

The approximate analytical footstep controller first models the robot's future behavior, and based on this model it finds the optimal foot locations in the touch-down. To determine the optimal foot placement location, an apex-to-apex analysis will be conducted similar to [41], [50]. This methodology is alternatively referred to as a Poincaré section at $\dot{z} = 0$ or apex return map, with the apex state being defined in equation 3.33. The apex-to-apex behavior is split into 3 regions as shown in figure 3.3, namely apex to touch-down, touch-down to lift-off, and lift-off to apex.

$$\delta = [z_{apex}, \dot{x}_{apex}]^T \quad (3.33)$$

In the first part, when the robot is in the flight phase, the system is modeled as a projectile. The accelerations during this phase are provided in equation 3.34.

$$\begin{bmatrix} \ddot{x} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} 0 \\ -g \end{bmatrix} \quad (3.34)$$

$$t_{td} = \left(\frac{2}{g}[z - z_f]\right)^{0.5} \quad (3.35)$$

$$\dot{x}_{td} = \dot{x}_{apex} \quad z_{td} = z_f \quad \dot{z}_{td} = -gt_{td} \quad (3.36)$$

The flight time can be calculated with equation 3.35. In the equations 3.35 and 3.36,

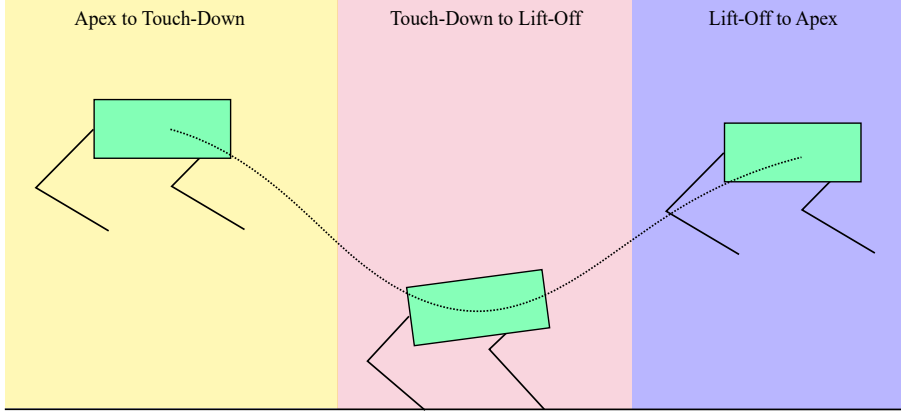


Figure 3.3: Apex-to-apex behavior of the robot. The figure illustrates one full locomotion cycle divided into three phases: Apex to Touch-Down, Touch-Down to Lift-Off, and Lift-Off to Apex, showing the robot's body motion and foot contact transitions across the cycle.

z_f is the difference between foot height and the body height, td refers to touch-down. The touch-down states are given in equation 3.36.

For the stance dynamics, equations of motions obtained from the template are analytically solved. The analytical solutions of equations 3.3, 3.5 and 3.7 are given in equations 3.37, 3.38 and 3.39. The desired values are assumed to be constant while solving these equations. Also, ϕ_{des} , x_{des} and $\dot{z}_{des}$ are assumed to be zero.

$$\phi(t) = \phi_{des} + C_{1\phi} e^{\frac{-t}{2I_\phi}(K_{d\phi} + \beta_\phi)} + C_{2\phi} e^{\frac{-t}{2I_\phi}(K_{d\phi} - \beta_\phi)} \quad (3.37)$$

$$x(t) = C_{1x} e^{\frac{-K_{dx}t}{m}} + C_{2x} + \dot{x}_{des}t \quad (3.38)$$

$$z(t) = z_{des} + C_{1z} e^{\frac{-t}{2m}(K_{dz} + \beta_z)} + C_{2z} e^{\frac{-t}{2m}(K_{dz} - \beta_z)} - \frac{mg}{K_{pz}} \quad (3.39)$$

Where,

$$\beta_z = (K_{dz}^2 - 4K_{pz}m)^{0.5} \quad \beta_\phi = (K_{d\phi}^2 - 4K_{p\phi}I_\phi)^{0.5} \quad (3.40)$$

C_{1z} , C_{2z} , C_{1x} , C_{2x} , $C_{1\phi}$ and $C_{2\phi}$ are found from initial conditions.

The lift-off to apex phase is similar to the apex to touchdown phase; however, it is observed that after lift-off, the legs exhibit dynamic behaviors to reach the desired po-

sitions, which affects the estimated time. To compensate for this unaccounted energy loss, an energy correction factor is added to the model, and this phase is solved based on the system's energy. The energy in lift-off is given in equation 3.41, the energy in apex is given in equation 3.42, and their relationship is given in equation 3.43. The equation for energy correction E_{corr} , is given in equation 3.44, and the constants a and b are found by experiments. While solving the equation 3.43, it is assumed that $\dot{x}_{lo}$ is equal to $\dot{x}_{apex}$, it is a logical assumption because the feet move less in the horizontal direction than the vertical direction; therefore, most of the energy loss is caused by the vertical movement and it affects the z_{apex} much more than $\dot{x}_{apex}$.

$$E_{lo} = 0.5m(\dot{x}_{lo}^2 + \dot{z}_{lo}^2) + mgz_{lo} \quad (3.41)$$

$$E_{apex} = 0.5m\dot{x}_{apex}^2 + mgz_{apex} \quad (3.42)$$

$$E_{lo} = E_{apex} + E_{corr}(z_{des,apex}) \quad (3.43)$$

$$E_{corr}(z_{des,apex}) = az_{des,apex} + b \quad (3.44)$$

3.4.2 Solving For The Optimal Foot Location

The z-axis determines the duration of the stance phase. When $\ddot{z}$ becomes negative in the template, the robot transitions to the flight phase. For each z_f value, we first identify t_{lo} (lift-off time) using a non-linear root solver. Subsequently, the x-displacement during the stance phase is computed using equation 3.38. To find the optimal x_{f0} , the horizontal distance from foot tip to shoulder, we minimize the moment around the ϕ axis. Calculating the moment necessitates the consideration of ground reaction forces. Assuming that the template motion precisely mirrors the robot's movement, we derive the sum of GRFs along the x and z axes from equations 3.5 and 3.7 as F_x and F_z . These are the forces that should be applied to the COM. After calculating the GRFs, a uniform distribution across the legs is assumed, as outlined in equation 3.45.

$$F_{1x} = F_{2x} = F_{3x} = F_{4x} \quad F_{1z} = F_{2z} = F_{3z} = F_{4z} \quad (3.45)$$

Because of the uniform force assumption, in Figure 3.4 the forces are calculated as $F_{bx} = F_{fx} = F_x/2$ and $F_{bz} = F_{fz} = F_z/2$. The moment is calculated in equation 3.46.

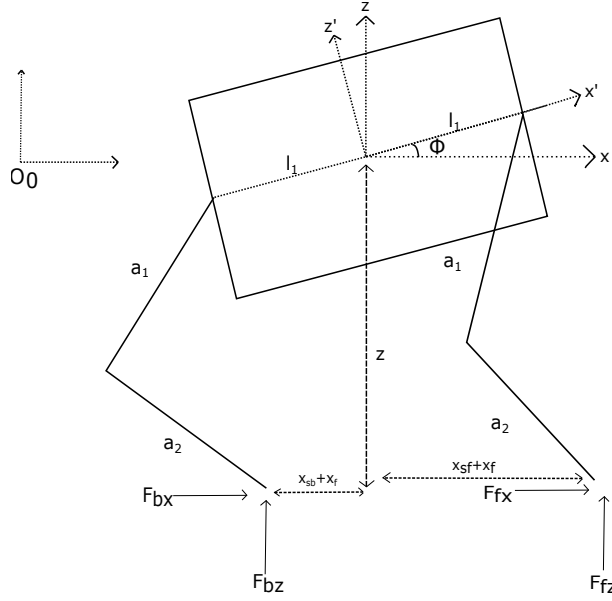


Figure 3.4: Model used for the approximate analytical footstep controller. The simplified planar model shows body orientation, leg geometry, and ground reaction forces used to compute the next foot placement.

$$\tau_\phi = (F_{bx} + F_{fx})z + F_{fz}(x_{sf} + x_f) + F_{bz}(x_{sb} + x_f) \quad (3.46)$$

In equation 3.46, x_{sb} and x_{sf} are the x axis displacement of back and front shoulders with respect to the center of mass, which can be calculated as $x_{sf} = l_1 \cos(\phi)$, $x_{sb} = -l_1 \cos(\phi)$. Equation 3.46 can be simplified as equation 3.47 since the $(F_{bx} + F_{fx})z$ part is just an offset to the cost function.

$$\tau(t) = F_x z + F_z x_f(t) \rightarrow \tilde{\tau}(t) = F_z x_f(t) \quad (3.47)$$

In equation 3.47, $x_f(t)$ can be found as $x_f(t) = x_{f0} - x(t)$, where $x(t)$ is obtained from equation 3.38. Equation 3.47 is numerically integrated between t_{td} and t_{lo} , and the optimal x_{f0} that minimizes this integration is calculated.

To find the optimal value of z_f , the cost function given in equation 3.48 is minimized. The system is modeled over the prediction horizon and the found apex states are added to the cost function. In literature, this method is called as a non-linear model predictive controller or NMPC since our state update equations are highly non-linear.

$$\min_{x_{f0}, z_{f0}} \sum_{i=0}^{k-1} |\tau_{\phi, tmp}[\boldsymbol{\eta}_k] - F_z[\boldsymbol{\eta}_k, z_{f0}]x_f[\boldsymbol{\eta}_k, x_{f0}]| \quad (3.48)$$

subject to

$$\boldsymbol{\eta}_{k+1} = g(\boldsymbol{\eta}_k) \quad (3.48a)$$

In equation 3.48a, $\boldsymbol{\eta}$ denotes the two-dimensional planar states defined in equation 3.49. These states are updated according to the analytical solutions given in 3.37, 3.38, and 3.39, together with their corresponding derivatives. Here, $\tau_{\phi, tmp}$ represents the template force in the pitch direction, as computed from equation 3.1. By solving the optimization problem formulated in 3.48, the robot is able to closely reproduce the template motion while simultaneously tracking the desired velocity and apex height.

$$\boldsymbol{\eta} = [\phi \dot{\phi} x \dot{x} z \dot{z}]^T \quad (3.49)$$

3.5 Proposed Approach

Figure 3.5 illustrates the proposed controller for the quadrupedal robot. First, the states from the robot are acquired, and then whether the robot is in the stance phase or flight phase is determined.

If the robot is in the stance phase, the template generates reference motions based on the state output by the robot. Next, the model predictive controller is employed to track these reference motions optimally. The MPC calculates the appropriate GRFs required to achieve the desired trajectory and maintain stability.

The GRFs computed by the MPC are then converted into joint torques, which are used as control inputs for the robot. This conversion is essential as the GRFs represent forces at the contact points with the ground, while joint torques are the actual commands sent to the robot's actuators. The entire system operates with the reference generation and MPC running at the same frequency, ensuring synchronized updates and consistent control performance throughout the stance phase.

If the robot is in the flight phase, first whether the robot is in apex or not is determined

by ensuring the velocity in the z-axis is close to zero within a certain tolerance. If the robot is in apex, the footstep planner solves the optimization problem in equation 3.48 for optimal foot placement. Then, found x_{f0} and z_f are converted to reference joint angles for the flight phase by solving the inverse kinematics. The robot uses a PD controller to achieve these reference joint angles. These references are only updated in apex, otherwise, they remain the same.

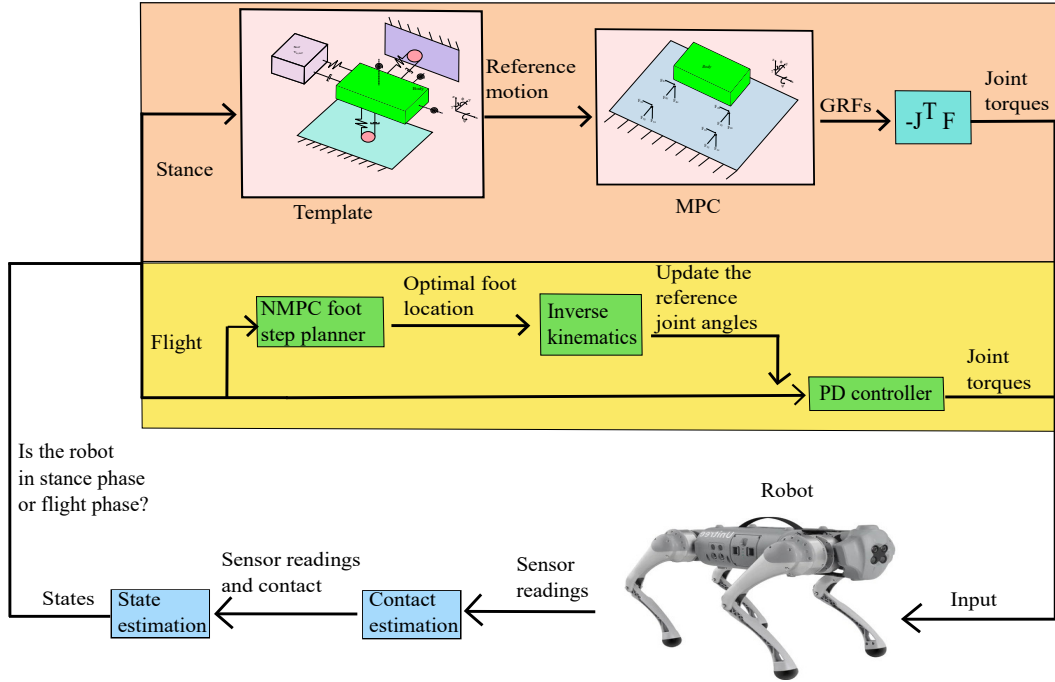


Figure 3.5: The proposed controller architecture. The diagram summarizes the stance and flight control pipelines, including the template model, MPC-based stance controller, NMPC footstep planner, inverse kinematics, and PD control, together with state and contact estimation modules.

To sum up, first, the controller detects whether the robot is in the flight phase or stance phase. If it is in flight phase it uses a PD controller to reach the desired joint angles. If it reaches an apex, it updates these desired angles. If the robot is in the stance phase, it uses the template to generate a reference motion and MPC to track this motion. This integrated approach ensures that the robot adheres to the desired pronking gait while dynamically adjusting to any deviations from the planned trajectory.

3.6 Simulation Analysis

For the simulations, the Unitree GO1 robot is utilized in MuJoCo simulator, as shown in Figure 3.6. For the pronking controller of the robot, the template parameters are selected as in Table 3.1. The gain K_{pz} is selected high, because the robot needs to pronk around a predefined height, meaning that its feet need to lose contact with the ground in order to locomote. The gain K_{px} is selected as 0 because the x direction velocity of the robot is controlled, not the position. The gains in the y direction are not as high as the others, because its effect on stability and desired state tracking is less than others.



Figure 3.6: Unitree GO1 Robot in MuJoCo Simulation Environment

For the optimal controller, the weights are selected as $\mathbf{Q} = \text{blkdiag}(2, 2, 2, 1.75, 0.75, 1)$. For the template-based controller, the horizon is selected as 1, meaning the future is not predicted, only the current time is used. For the model predictive controllers, the horizons are selected as 5, and weights are discounted in each prediction by a discount factor of 0.9. The template-based motion generation and MPC run at 500 Hz.

In Figures 3.11, 3.12, LTI-MPC (Linear Time Invariant Model Predictive Controller) means that the jacobians are not updated, once the controller starts to estimate the future, it uses the initial jacobian without any updates, for the modeling. In the LTV-MPC (Linear Time-Varying Model Predictive Controller), the jacobians are updated with Euler stepping.

During the flight phase, when the robot has no ground contact, it moves its legs forward to track velocity. This leg movement induces rotational motion in the air, caus-

Table 3.1: Parameters for the template.

Parameter	Value	Unit	Parameter	Value	Unit
$K_{p\psi}$	400	Nm	$K_{d\psi}$	200	Nms
$K_{p\phi}$	800	Nm	$K_{d\phi}$	400	Nms
$K_{p\gamma}$	300	Nm	$K_{d\gamma}$	100	Nms
K_{px}	0	N/m	K_{dx}	1000	Ns/m
K_{py}	20	N/m	K_{dy}	100	Ns/m
K_{pz}	6000	N/m	K_{dz}	50	Ns/m

ing the robot to tilt upon touchdown. To counteract this rotation, the desired pitch angle, ϕ_{des} , is dynamically adjusted using equation 3.50. The parameters in equation 3.50 were determined through experimentation at different velocities, followed by tuning the ϕ_{des} angle and fitting a polynomial to the resulting data. The optimal ϕ_{des} angles for velocities of 0.5 m/s, 1 m/s, and 1.5 m/s were found to be -0.1325, -0.225, and -0.4 radians, respectively. By adjusting the reference pitch angle, the robot lifts off with its head elevated, levels out in the air due to the rotation induced by leg masses, and lands in a horizontal orientation. This dynamic adjustment of the desired pitch angle effectively compensates for body rotation during the flight phase. Similarly, the correction term $E_{corr}(z_{des,apex})$, shown in equation 3.51, was determined using the same method.

$$\phi_{des} = -0.165x_{des}^2 + 0.0625x_{des} - 0.1225 \quad (3.50)$$

$$E_{corr}(z_{des,apex}) = 200z_{des,apex} - 36 \quad (3.51)$$

As in the previous section, quadratic programming optimization problems, inverse kinematics and linear algebra computations rely on the same set of libraries. On the other hand, for solving the NMPC problem within the footstep planner, the NLOpt library is utilized [76].

3.6.1 Results From Simulation

The first experiment evaluates the performance improvements resulting from the use of an adaptive pitch angle, as described in equation 3.50. Figure 3.7 illustrates the robot's pitch angle and horizontal velocity across various target speeds. As demonstrated, the robot maintains effective velocity tracking without the adaptive pitch angle at lower speeds, such as 0.5 m/s. However, at higher speeds, it struggles to remain stable, experiencing falls and failing to reach a stable limit cycle. This instability is primarily due to the robot's need to place its feet further forward to achieve higher speeds, which induces unwanted rotational motion during the flight phase. The adaptive reference pitch angle compensates for this rotation, enabling the robot to maintain stability at faster velocities.

The second two conducted experiments test the approximate analytical footstep planner. In order to measure its performance, the proposed footstep controller is compared to the Raibert controller. Figure 3.8 compares the two controllers under the same parameters. The desired horizontal velocity is 1.5 m/s and the apex height is 40 cm. As can be seen in Figure 3.8 the horizontal velocity $\dot{x}$ and apex heights are similar in both controllers when a stable limit cycle is achieved. The average apex horizontal velocity for the approximate analytical footstep planner is 1.21 m/s and for the Raibert's it is 1.10 m/s, the average apex height for the approximate analytical footstep planner is 38 cm, and for the Raibert's it is 36 cm. However, the oscillations around the pitch and roll axes are smaller for the approximate analytical controller. This means that the proposed approach is more stable than Raibert's footstep planner.

The third experiment compares the performance of the approximate analytical footstep planner with Raibert's footstep planner under extreme initial conditions to assess their ability to stabilize the robot. In this test, the robot is launched from a height of 0.7 meters with an initial horizontal velocity of 5 meters per second. The results, shown in Figure 3.9, depict the states for both controllers throughout the test. Raibert's footstep controller fails to achieve a stable pronking motion, resulting in the robot falling within the first cycle. In contrast, the proposed footstep planner successfully stabilizes the robot, achieving a stable pronking motion and reaching the desired states within just two cycles. This outcome highlights the robustness and ef-

fectiveness of the proposed footstep controller in handling challenging conditions and maintaining stability.

The fourth experiment is for observing the state tracking performance of the proposed controller. Figure 3.10 shows the desired states vs the robot's states. It is seen that for the lower desired velocities, the error for both the velocity and the apex height is very small, and as the robot's velocity increases the error for both velocity and apex height increases because of the torque limitations and the friction constraints. The error for the desired pitch angle has a jump at the beginning but after one cycle it settles.

The fifth experiment was designed to test the robot's limits in achieving maximum horizontal velocity while maintaining a high apex height. Figure 3.11 compares the performance of three different controllers when the desired apex height is set to 0.44 meters.

The results in Figure 3.11 show that the LTI-MPC performs the worst due to its inaccurate future state estimates. Although the TBC performs better than the LTI-MPC, it also fails after 6 seconds. In contrast, the LTV-MPC outperforms the other controllers, demonstrating stable tracking of the desired states with a steady-state error.

The observed steady-state error for the LTV-MPC can be attributed to the torque limits and friction constraints of the robot. Despite these limitations, the LTV-MPC's superior performance highlights its effectiveness in handling high-velocity and high-apex height conditions, showcasing its robustness and reliability for advanced quadrupedal locomotion.

The sixth experiment aims to assess the controller's robustness by applying a push to the robot at its apex and determining whether it could stabilize itself. For this test, the robot is allowed to reach a stable limit cycle, and then a push is applied in the y-direction (normal to the sagittal plane) by adding a constant factor to the y-velocity. The y-velocity change is set to 1.7 m/s, and the robot is given 3 seconds to achieve a stable limit cycle. At the first apex after this period, the robot receives the push in the y-direction.

The results show that the LTV-MPC, TBC, and LTI-MPC controllers can stabilize the robot up to pushes of 1.8 m/s, 1.6 m/s, and 1.4 m/s, respectively. Figure 3.12

displays the states recorded during this experiment. It is observed that the TBC and LTI-MPC controllers fail to recover from the push, whereas the LTV-MPC controller successfully stabilizes the robot. This experiment demonstrates that the proposed LTV-MPC controller offers superior robustness to disturbances compared to the other controllers.

To further assess the performance of the motion generator and the proposed controllers, we conduct an analysis following a methodology similar to [2] and [35]. The analysis focuses on stability and basins of attraction of the controller. To gauge that, two initial conditions are systematically varied and simulated, and the resulting states are recorded. The selected states for evaluation are the apex height and horizontal velocity at the apex. Throughout these simulations, the desired states remain consistent with their initial values. Each simulation spans 12 seconds, a duration validated as sufficient for reaching a stable limit cycle.

The desired states are 1.5 m/s for the apex velocity and 0.4 m for the apex height. All initial conditions either converge to the same stable limit cycle or they diverge. The convergent initial conditions lead to the state $y^* = 0.38\text{m}$ and $\dot{x}^* = 1.27\text{m/s}$. Figure 3.13 illustrates the stable initial conditions that lead to a consistent limit cycle. Dashed lines are provided as examples to highlight the convergence behavior. It is seen that the algorithm takes a maximum of two cycles to reach the desired states thanks to the footstep planner. Notably, our approach demonstrates stability across a large range of diverse initial conditions. This analysis also provides insight into the system's robustness.

The video for the simulations can be found at the following link.

<https://www.youtube.com/watch?v=ni4qoJYBPU8>

3.7 Hardware Experiments

Figure 3.14 illustrates a sequence of frames captured during a pronking motion of the Unitree GO1 robot. Pronking is characterized by a symmetric, vertical jumping behavior where all legs perform synchronized push-off and landing. The image

sequence clearly demonstrates the transition from stance to aerial phase and back, validating the robot's capability to maintain dynamic balance during vertical locomotion.

Figure 3.15 illustrates the key state variables obtained during a hardware pronking experiment conducted on the Unitree GO1 robot using the proposed controller method. The same experimental setup described in the previous section is used. The desired forward velocity is set to 0.5m/s, and the target vertical spring height is configured to 0.3m. At the beginning of the experiment, the robot is initialized in a standing posture at approximately 0.2 m height using a separate set of control gains. Once the robot reaches the desired initial conditions, the control gains are switched to the pronking-specific values, and the robot begins executing the pronking gait.

Subfigure (a) shows the horizontal position x , which increases steadily during forward motion. Subfigure (b) presents the corresponding horizontal velocity $\dot{x}$, where fluctuations are observed due to the impulsive nature of pronking. Subfigure (c) displays the vertical position z , demonstrating the periodic oscillations characteristic of the pronking gait. Similarly, Subfigure (d) shows the vertical velocity $\dot{z}$, which exhibits high-frequency changes during takeoff and landing phases. The pitch angle ϕ , shown in Subfigure (e), remains mostly bounded, while its derivative $\dot{\phi}$ in Subfigure (f) reveals the dynamic rotational behavior during aerial phases. Overall, the plots confirm that the robot successfully transitions into a pronking gait and maintains a consistent pattern across the motion cycle.

Since the pronking motion is not governed by a predefined clock or gait cycle, accurate contact detection is essential for phase transitions. To estimate ground contacts, external forces are inferred using the simplified dynamic equation shown in 3.52. This equation is derived from the system's equations of motion, where acceleration terms are neglected due to their unavailability in direct measurements and the numerical errors involved in estimating them [77].

In this equation, τ_m denotes the joint torques measured from the robot, $\mathbf{B}$ represents the Coriolis and centrifugal force vector, $\mathbf{G}$ is the gravity vector, $\mathbf{J}$ is the contact Jacobian, and $\mathbf{F}_{ext}$ is the estimated external contact force. Contact is detected by comparing $\mathbf{F}_{ext}$ against a predefined threshold.

Furthermore, to determine lift-off events, the leg's knee joint velocities are monitored. If a leg is in stance phase and its knee velocity exceeds a certain threshold, the contact status is set to zero, and the controller transitions to swing mode for that leg.

$$\mathbf{B} + \mathbf{G} \approx \boldsymbol{\tau}_m + \mathbf{J}^\top \mathbf{F}_{\text{ext}} \Rightarrow \mathbf{F}_{\text{ext}} \approx (\mathbf{J}^\top)^{-1} (\mathbf{B} + \mathbf{G} - \boldsymbol{\tau}_m) \quad (3.52)$$

The video for the hardware experiment can be found at the following link.

https://youtu.be/Ha1lgMZO_T8

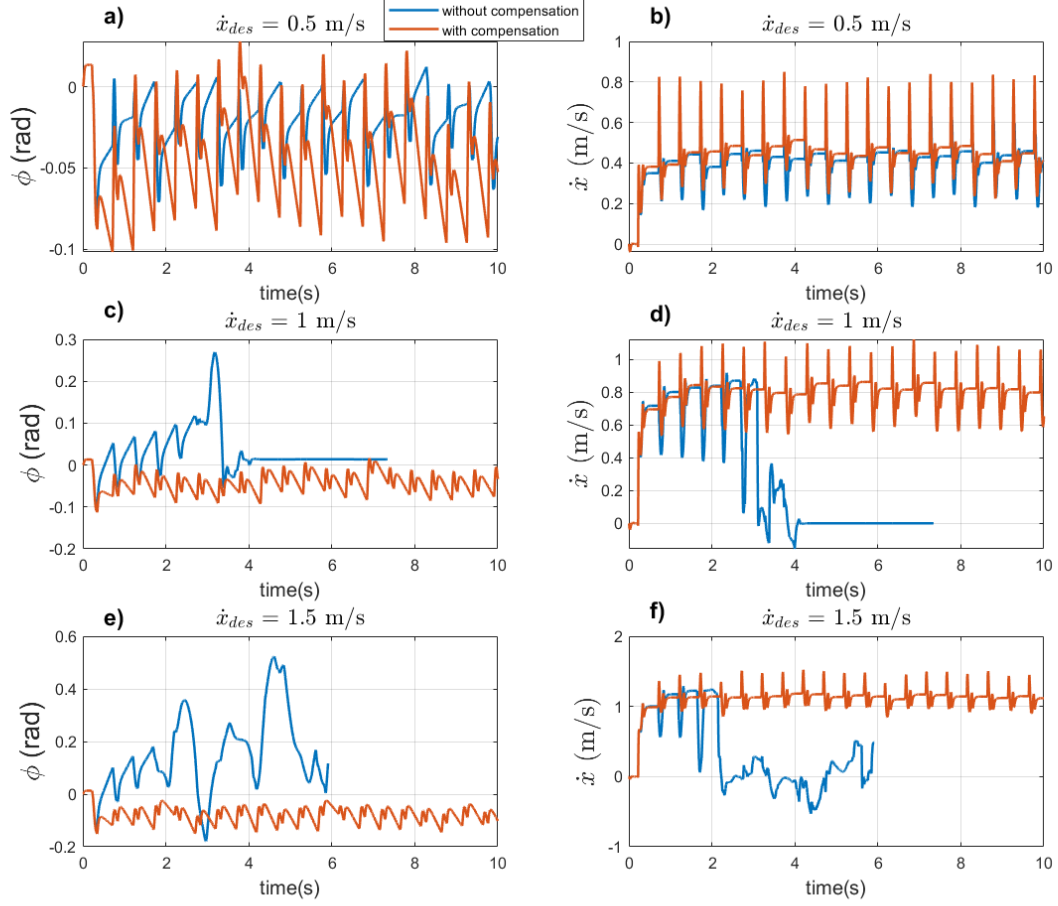


Figure 3.7: Effect of adaptive pitch reference on locomotion performance. Comparison of robot pitch angle and forward velocity with and without the adaptive pitch reference at different desired velocities: (a) pitch angle at $\dot{x}_{des} = 0.5$ m/s, (b) forward velocity at $\dot{x}_{des} = 0.5$ m/s, (c) pitch angle at $\dot{x}_{des} = 1$ m/s, (d) forward velocity at $\dot{x}_{des} = 1$ m/s, (e) pitch angle at $\dot{x}_{des} = 1.5$ m/s, (f) forward velocity at $\dot{x}_{des} = 1.5$ m/s. The adaptive pitch reference improves stability and reduces pitch oscillations across all speeds.

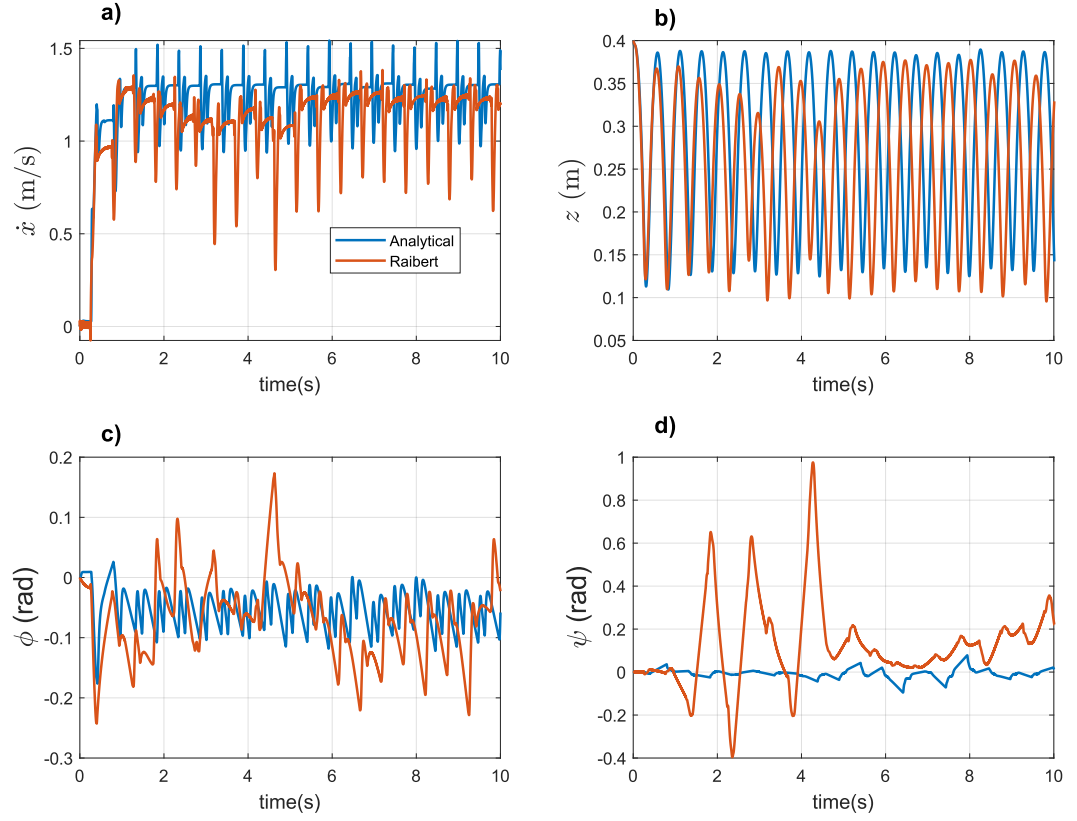


Figure 3.8: Comparison of state-tracking performance between the Raibert footstep planner and the proposed approximate analytical planner. (a) Horizontal velocity $\dot{x}$, (b) height z , (c) pitch angle ϕ , (d) roll angle ψ during a 10-second locomotion trial.

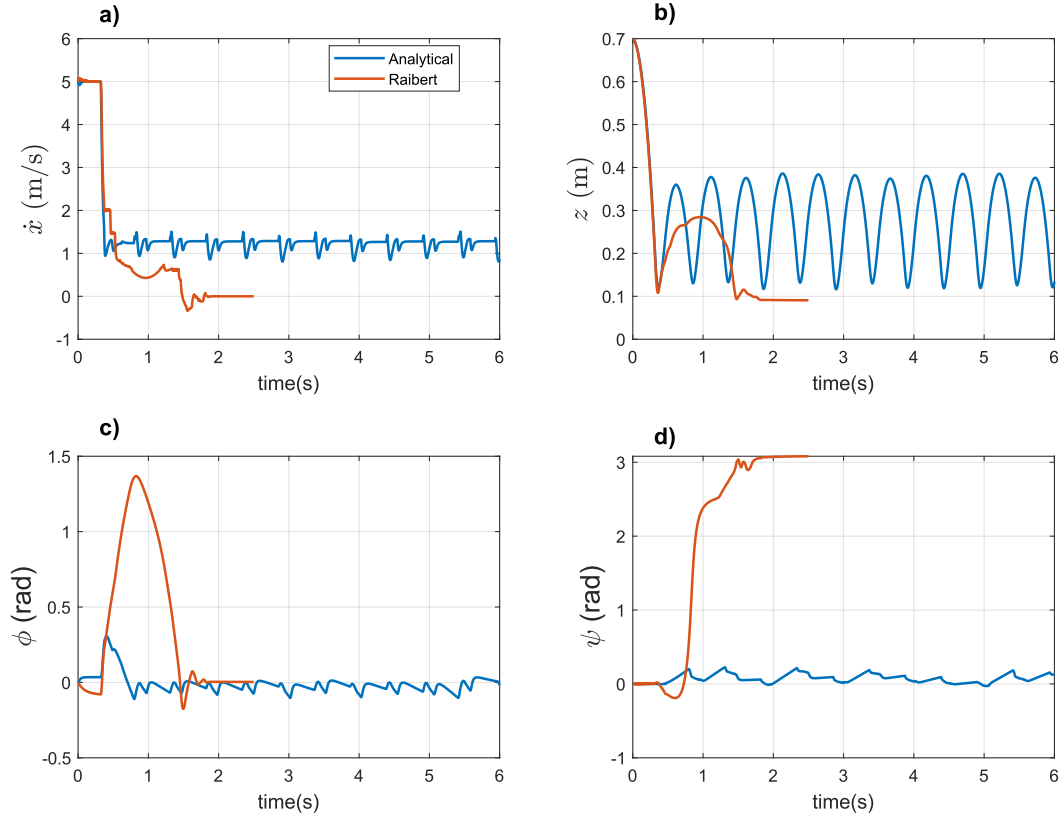


Figure 3.9: Robustness comparison between the Raibert controller and the approximate analytical foot-step planner. State responses under a disturbance: (a) horizontal velocity $\dot{x}$, (b) height z , (c) pitch angle ϕ , (d) roll angle ψ . The analytical planner maintains stability, whereas the Raibert controller diverges under the same conditions.

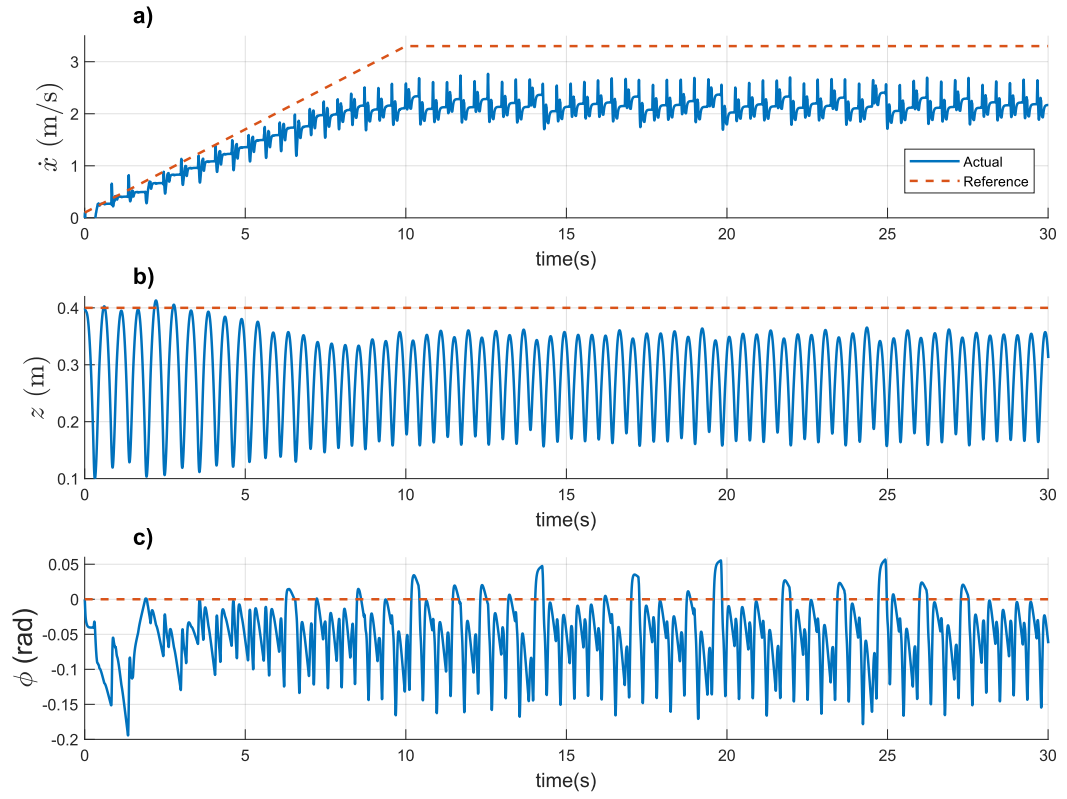


Figure 3.10: State tracking performance of the proposed controller. *a)* Horizontal Velocity $\dot{x}$ of the robot, *b)* height of the robot z (dashed line represents reference apex height), *c)* pitch angle of the robot ϕ

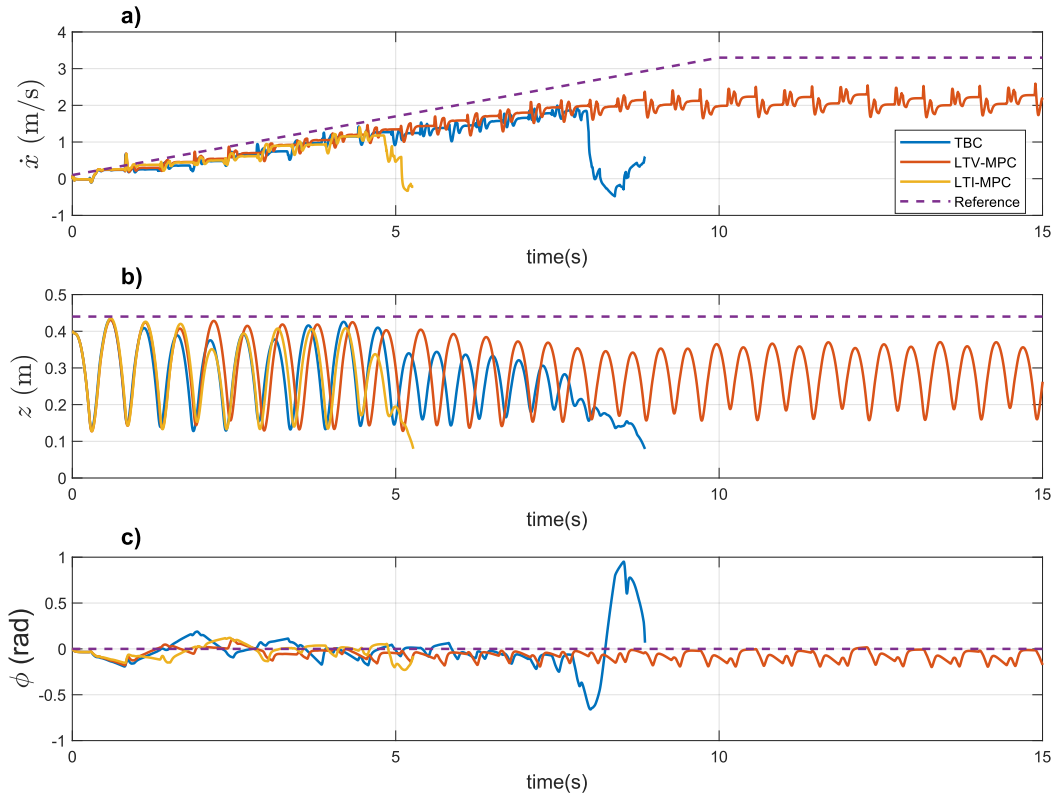


Figure 3.11: Comparison of velocity tracking performance for Template-Based Control (TBC), LTI-MPC, and LTV-MPC. (a) Horizontal velocity $\dot{x}$, (b) height z (dashed line: reference apex height), (c) pitch angle ϕ . TBC does not include future prediction, LTI-MPC incorporates future prediction with fixed Jacobians, and LTV-MPC includes both future prediction and online Jacobian updates.

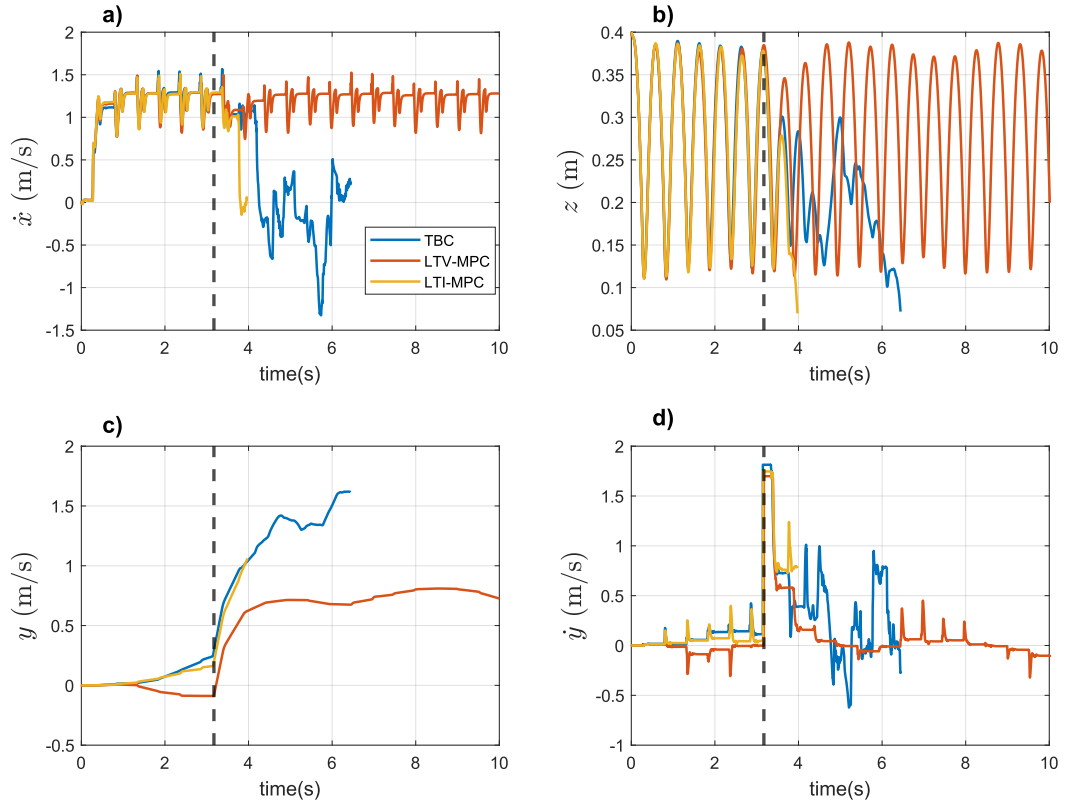


Figure 3.12: Recovery from an external push: comparison of TBC, LTV-MPC, and LTI-MPC. The dashed line indicates the push instant. Subplots show (a) horizontal velocity $\dot{x}$, (b) height z , (c) lateral position y , and (d) lateral velocity $\dot{y}$, highlighting the difference in disturbance-rejection performance among the controllers.

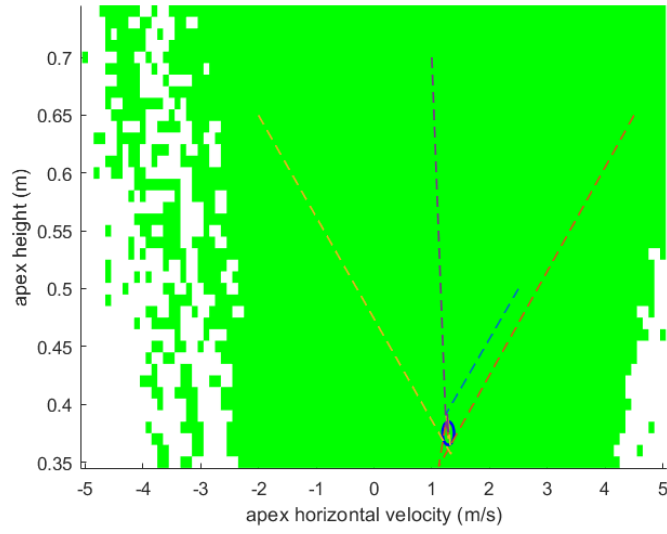


Figure 3.13: Stability region and convergence behavior. The green area represents the set of initial apex states $(\dot{x}, z)$ that converge to a stable limit cycle, while the dashed lines illustrate example convergence trajectories from different initial conditions.

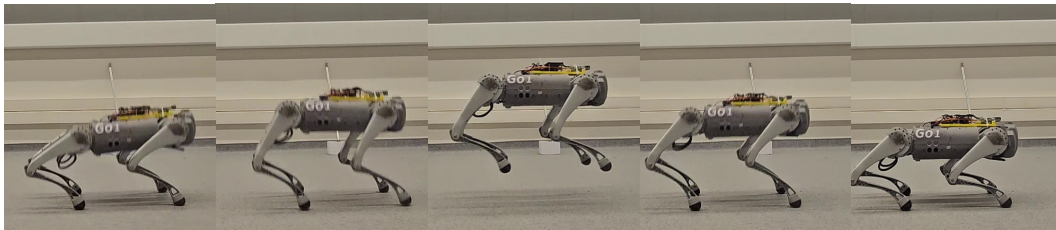


Figure 3.14: Sequential frames from a hardware experiment illustrating the pronking behavior of the Unitree GO1 quadruped robot. The motion involves a synchronized vertical push-off of all four legs, followed by an aerial phase and subsequent touch-down.

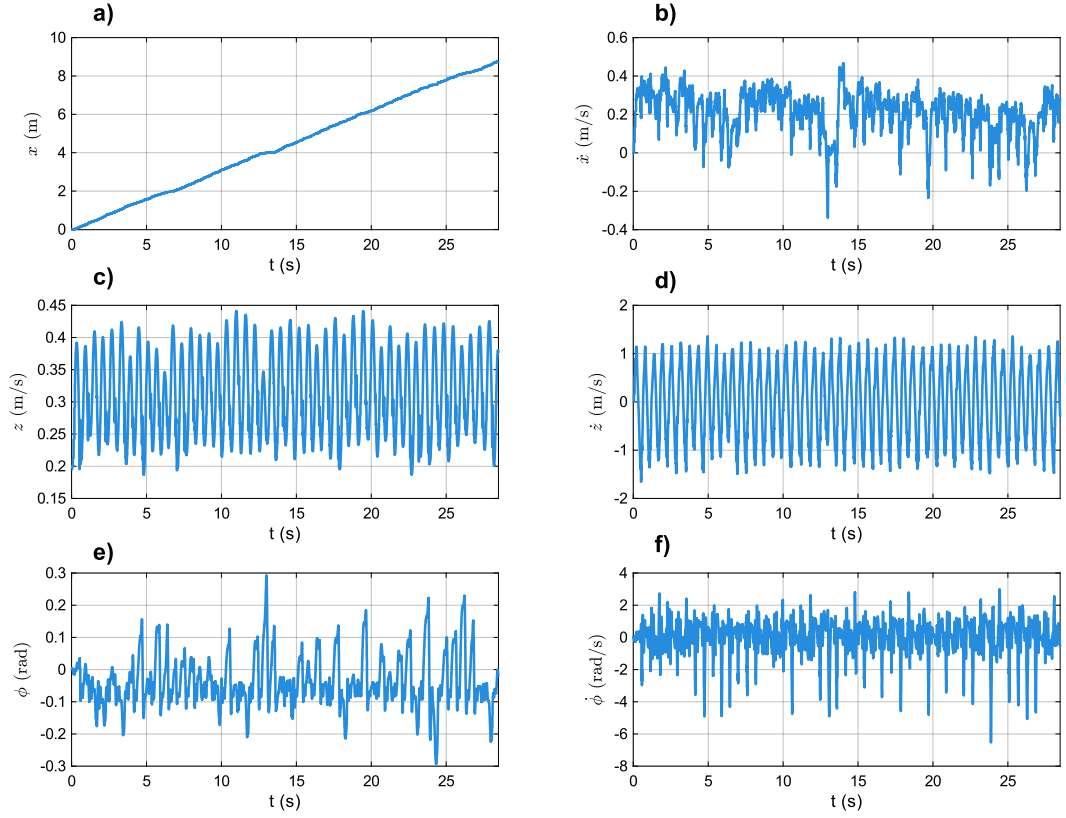


Figure 3.15: States of the hardware experiment using Unitree GO1 robot. *a)* horizontal position, *b)* horizontal velocity, *c)* vertical position, *d)* vertical velocity, *e)* pitch angle, *f)* pitch velocity

CHAPTER 4

AN OPTIMIZATION-DRIVEN FOOT PLACEMENT CONTROLLER FOR QUADRUPED LANDING

In this chapter, we introduce a control framework for achieving successful landing behaviors in quadrupedal robots. Landing refers to the phase following aerial motion in which the robot must absorb impact forces and regain stable ground contact. The chapter begins with the formulation of a template model and a stance-phase controller. Building on this foundation, a foot placement controller is proposed, formulated as a nonlinear model predictive control (NMPC) problem. The chapter concludes with simulation and hardware experiments that demonstrate the effectiveness of the proposed landing strategy.

4.1 Template Model and Stance Controller

$$\mathbf{F}_\zeta = \mathbf{K}_{p,\zeta}(\zeta_{\text{des}} - \zeta) + \mathbf{K}_{d,\zeta}(\dot{\zeta}_{\text{des}} - \dot{\zeta}) \quad (4.1)$$

The template used to model the robot's stance phase is shown in Fig. 4.1, which is the same template from the previous section. In each linear and angular axis, springs and dampers are implemented to control the forces acting on the robot. In the figure, the spirals represent the rotational springs and dampers. The forces applied to the robot's body are described in equation 4.1. In this equation, $\zeta = [\psi, \phi, \gamma, x, y, z]^\top$ represents the generalized coordinates of the body, defined as roll, pitch, yaw, and the linear axes, respectively. $\mathbf{K}_{p,\zeta}$ and $\mathbf{K}_{d,\zeta}$ are diagonal gain matrices that define the spring and damper coefficients.

The equations of motion for the quadruped robot are presented in equation 4.2. Here,

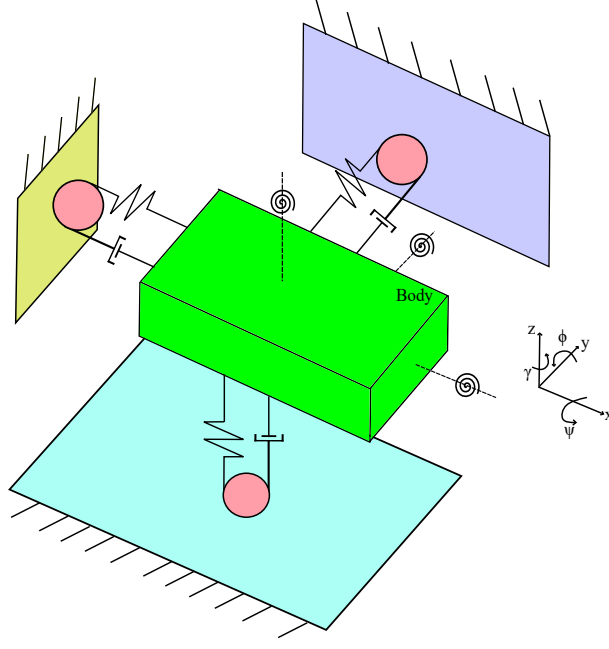


Figure 4.1: Template for stance motion. During stance, the body is regulated by a six-dimensional virtual wrench consisting of three linear and three angular virtual forces applied at the center of mass, illustrated through conceptual spring–damper elements.

M is the mass matrix, and q is the generalized coordinate vector, as defined in equation 4.3. In equation 4.3, $\theta_{12 \times 1}$ represents generalized coordinates for the leg angles. The term B represents the Coriolis forces, G denotes the gravitational forces, τ are the joint torques, J is the Jacobian matrix of the system, and F represents the GRFs. The first six elements of the q vector correspond to the coordinates of the body, which are not directly actuated. Consequently, the first six elements of the τ vector are zero.

$$M\ddot{q} + B + G = \tau + J^T F \quad (4.2)$$

$$q = [\psi \ \phi \ \gamma \ x \ y \ z \ \theta_{12 \times 1}]^T \quad (4.3)$$

A common approach to solving the control problem for quadruped robots is to ignore the leg masses and inertias. This simplification makes the Coriolis force vector zero and results in only the first 6×6 block of the mass matrix being non-zero. With this assumption, the system can be expressed as shown in equations 4.4 and 4.5. In these equations, M_{body} represents the first 6×6 block of the mass matrix, q_{body} denotes the first six elements of the generalized coordinate vector, G_{body} corresponds to the first

six elements of the gravitational force vector $\mathbf{G}$, and $\mathbf{J}_{body}^\top$ is the first six rows of the transpose of the Jacobian matrix. The term τ_{joint} represents the joint torques at the contact feet, and $\mathbf{J}_{joint}^\top$ refers to the remaining part of the transpose of the Jacobian matrix.

$$\mathbf{M}_{fb}\ddot{\mathbf{q}}_{fb} + \mathbf{G}_{fb} = \mathbf{J}_{fb}^\top \mathbf{F} \quad (4.4)$$

$$\tau_{joint} = -\mathbf{J}_{joint}^\top \mathbf{F} \quad (4.5)$$

Following these steps, the required GRFs can be determined by solving an optimization problem formulated as a quadratic program, as shown in equation 4.6. In this formulation, $\mathbf{F}_\zeta$ represents the desired COM forces calculated from the template, and $\mathbf{Q}^{tmp}$ is a diagonal weight matrix. The inequality constraints given in equations 4.6a, 4.6b and 4.6c ensure that the torque limits are not exceeded, the feet do not slip on the ground, and contact with the ground is maintained.

$$\min_U \quad \|\mathbf{J}_{fb}^\top \mathbf{U} - \mathbf{F}_\zeta\|_{\mathbf{Q}^{tmp}} \quad (4.6)$$

subjected to

$$-\tau_{\max} \leq \mathbf{L}\mathbf{U} \leq \tau_{\max} \quad (4.6a)$$

$$\mathbf{D}\mathbf{U} \leq 0 \quad (4.6b)$$

$$\bar{\mathbf{u}}_{\min} \leq \mathbf{U} \quad (4.6c)$$

In equation 4.6, $\mathbf{U}$ represents the GRFs, and its size is 12×1 when all four legs are in contact with the ground. Accordingly, the transpose of the Jacobian matrix for the floating base, $\mathbf{J}_{fb}^\top$, has a size of 6×12 . The matrix $\mathbf{L}$ is of size 12×12 , while $\mathbf{D}$ has a size of 32×12 , as the friction cone is linearized using an octagonal approximation.

Once the required GRFs are determined, they are converted to joint torques using equation 4.5.

To summarize the stance controller, first template will be used to compute desired forces that should be applied to the body. Then the quadruped robot dynamic is simplified, and we will try to find the GRFs with an optimization that will create

body forces that are closest to template forces. After that, these GRFs are converted to joint torques.

4.2 Foot Placement Controller

The objective of the foot placement controller is to enable the robot to closely follow the motion of the template. During the flight phase, the robot's stance dynamics are predicted, and optimal foot positions are determined to ensure that the resulting motion can effectively replicate the template behavior.

The flowchart of the foot placement controller is shown in Fig. 4.2. The controller runs an optimization loop in which each candidate foot location is evaluated by first estimating the corresponding touchdown state, then simulating the stance motion and computing the resulting template-based forces and moments. The cost is calculated based on the discrepancy between the template moments and those induced by the estimated GRFs. This process is repeated iteratively until convergence.

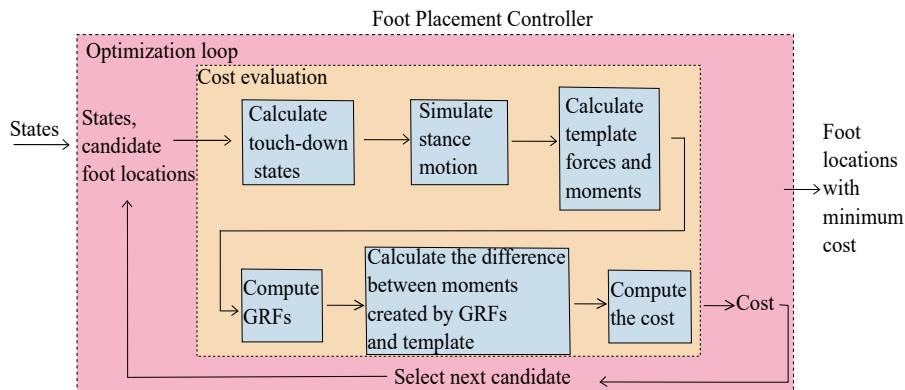


Figure 4.2: Flowchart of the foot placement controller. For each candidate foot location, the controller evaluates a cost based on the mismatch between template-generated and GRF-induced moments. The optimal location is selected through iterative optimization.

The foot placement controller is activated when all legs lose contact with the ground. It takes the current state of the robot as input and outputs the optimal foot locations for landing. The controller solves a non-linear optimization problem, which will be described in this section. This controller also neglects leg masses and assumes pro-

jectile motion for the robot's body for flight phase. However, by continuously solving the optimization problem while the robot is in air, it compensates for modeling errors.

Based on the current state, the controller first predicts the touchdown conditions by assuming ballistic (projectile) motion. The touchdown time is determined using equation 4.7, which is derived by solving a quadratic equation. Here, z_{td} represents the foot height relative to the body at touchdown. Since this height is not known in advance, it is treated as an optimization variable, allowing the solver to adjust it iteratively to find the optimal value. The terms z_i and $\dot{z}_i$ denote the initial vertical position and velocity, respectively, used by the foot placement controller. It is important to note that these initial conditions do not correspond to the initial drop state. Instead, the foot placement controller operates in a continuous and iterative manner, reinitializing the state values at the beginning of each optimization step.

$$z_{td} - z_i = \dot{z}_i t_{td} - \frac{1}{2} g t_{td}^2 \quad (4.7)$$

The linear touch down states are given in equation 4.8. While calculating the touch down states it neglects the rotational velocities in the air.

$$\begin{aligned} \dot{x}_{td} &= \dot{x}_i \\ \dot{y}_{td} &= \dot{y}_i \\ \dot{z}_{td} &= \dot{z}_i - g t_{td} \end{aligned} \quad (4.8)$$

To plan foot placement, the stance motion must be simulated prior to landing. The stance phase is modeled using the template's equations of motion. For the linear axes, the spring and damper gains are set to achieve critical damping, allowing the system to reach the desired states without oscillation. For the rotational axes, the gains are tuned to produce an overdamped response, which helps rapidly stabilize the body and prevents oscillatory behavior that could destabilize the robot during landing.

The solution for a critically damped system is given in equation 4.9, while the solution for an overdamped system is provided in equation 4.10. Here, A_1 and A_2 are coefficients determined from the initial touchdown conditions, ω_n is the natural frequency, and s_1, s_2 are the roots of the characteristic equation for the overdamped case.

$$p(t) = (A_1 + A_2 t)e^{-\omega_n t} \quad (4.9)$$

$$p(t) = A_1 e^{s_1 t} + A_2 e^{s_2 t} \quad (4.10)$$

After simulating the landing, the forces and moments generated by the template are computed over a predefined time interval t^* . For each time instant from the touch-down time t_{td} to t^* , the system states are determined using equations 4.9 and 4.10. Based on these states, the template forces are calculated using equation 4.1.

It is assumed that the linear forces generated by the template and intended to act on the body are equally distributed among the four legs. Consequently, each foot is assigned a GRF equal to one-fourth of the total force, as expressed in equation 4.11. In this equation, $\mathbf{F}_i$ denotes the GRF applied by the i -th foot. The components F_x , F_y , and F_z are obtained from the solution of equation 4.1.

$$\mathbf{F}_i = \frac{1}{4} \begin{bmatrix} F_x & F_y & F_z \end{bmatrix}^\top \quad (4.11)$$

If the computed ground reaction force exceeds the friction cone constraints, it is projected back onto the admissible friction region, as illustrated in Fig. 4.3. This is achieved by extending a vector from the origin to the original tangential force $\mathbf{F}_{leg}$, identifying its intersection with the boundary of the friction cone, and updating the force to this intersection point, denoted as $\mathbf{F}_{leg}^{updated}$. This correction preserves the vertical force component (F_z) while ensuring that the tangential components (F_x, F_y) remain within the friction limit defined by μF_z .

$$\mathbf{M}_{leg} = \sum_i (\mathbf{r}_i \times \mathbf{F}_i) \quad (4.12)$$

The moments generated by the legs are calculated using equation 4.12, where $\mathbf{r}_i$ represents the position vector from the center of mass (COM) to the i -th foot, $\mathbf{F}_i$ denotes the GRF at that foot, and $\mathbf{M}_{leg}$ is the resulting moment vector due to all GRFs.

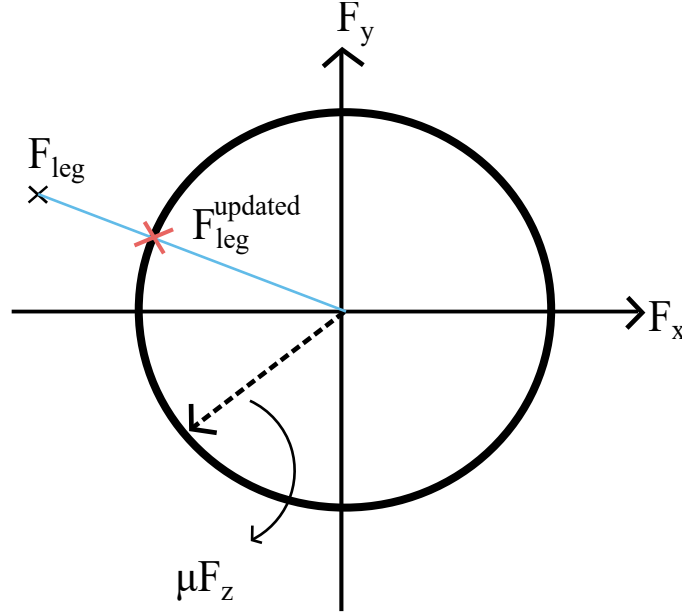


Figure 4.3: Friction cone projection. The original tangential force F_{leg} is adjusted to its projection $F_{\text{leg}}^{\text{updated}}$ on the boundary of the friction circle defined by μF_z , preserving the vertical force component.

When the robot lands, the landing yaw angle is adjusted to the desired yaw angle. Additionally, the desired x and y positions are updated to match the position of the geometric center of the legs at the instant of touch-down.

To ensure the foot position is achievable, a geometric constraint is applied, as defined in equation 4.13. In this equation, x_f and y_f represent the foot's position relative to the hip in the x and y directions, as shown in Fig. 4.4. The variable z_f denotes the foot's vertical distance from the center of mass, while l_0 represents the robot's maximum leg length.

$$x_f^2 + y_f^2 + z_f^2 \leq l_0^2 \quad (4.13)$$

It is assumed that x_f , y_f , and z_f are identical for all legs, meaning that the optimization process applies the same displacement and orientation to each foot. Furthermore, since all legs share the same z_f value, they are assumed to contact the ground simultaneously. .

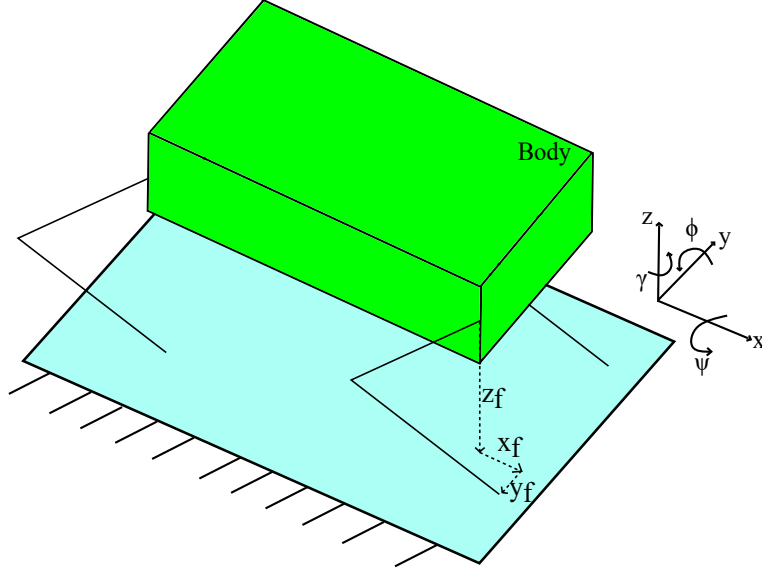


Figure 4.4: Foot locations relative to the robot body. The foot position is defined by the components x_f , y_f , and z_f expressed in the body frame, illustrating the geometric relationship between the robot's base and the foot placement.

A lower bound is also set for the foot's height relative to its body to prevent collisions with the ground during landing.

The optimization problem is defined in equation 4.14, where the foot location is represented as $\mathbf{p}_f = [x_f \ y_f \ z_f]^\top$. The cost function J evaluates the difference between the moments generated by the template and those induced by the legs about the roll, pitch, and yaw axes. It also includes a penalty on the vertical ground reaction force (GRF) in the z -direction.

In this context, F_ψ , F_ϕ , and F_γ represent the roll, pitch, and yaw moments generated by the template, respectively. The term F_z denotes the vertical force component, and Q_z is a scalar weight applied to this term. The vector $\mathbf{e}_i$ is the standard basis vector used to extract the corresponding moment component from the total moment vector generated by the legs.

This problem falls under the category of Nonlinear Model Predictive Control (NMPC), as it involves a cost function based on predicted dynamics and nonlinear state updates. The body states are updated at each time step along the prediction horizon according to the dynamics described in Equations 4.9 and 4.10, combined as Equation 4.14a.

Based on these updates, the template moments and forces are recalculated, and the estimated GRFs are updated accordingly.

$$\begin{aligned}
\min_{\mathbf{p}_f} \quad & \sum_{k=0}^N J_k(\mathbf{p}_f, \boldsymbol{\zeta}_k) = \left(F_\psi(\boldsymbol{\zeta}_k) - \mathbf{e}_1^\top \mathbf{M}_{\text{leg}}(\mathbf{p}_f, \boldsymbol{\zeta}_k) \right)^2 \\
& + \left(F_\phi(\boldsymbol{\zeta}_k) - \mathbf{e}_2^\top \mathbf{M}_{\text{leg}}(\mathbf{p}_f, \boldsymbol{\zeta}_k) \right)^2 \\
& + \left(F_\gamma(\boldsymbol{\zeta}_k) - \mathbf{e}_3^\top \mathbf{M}_{\text{leg}}(\mathbf{p}_f, \boldsymbol{\zeta}_k) \right)^2 \\
& + Q_z F_z(\boldsymbol{\zeta}_k)
\end{aligned} \tag{4.14}$$

subject to

$$\boldsymbol{\zeta}_{k+1} = g(\boldsymbol{\zeta}_k) \tag{4.14a}$$

$$\mathbf{p}_f^\top \mathbf{p}_f \leq l_0^2 \tag{4.14b}$$

$$\mathbf{e}_3^\top \mathbf{p}_f \geq z_{f,\min} \tag{4.14c}$$

At each prediction step, the relative position of each leg with respect to the center of mass, $\mathbf{r}_i$, and the corresponding template forces are updated based on the current state $\boldsymbol{\zeta}_k$. These states are obtained from the analytical equations of the template model, which also leads to the recomputation of the resulting moments as defined in equation 4.15.

$$\mathbf{M}_{\text{leg}}(\mathbf{p}_f, \boldsymbol{\zeta}_k) = \sum_i (\mathbf{r}_i(\mathbf{p}_f, \boldsymbol{\zeta}_k) \times \mathbf{F}_i(\boldsymbol{\zeta}_k)) \tag{4.15}$$

4.3 System-Level Control Architecture

This section describes the overall control architecture of the robot, which consists of a flight-phase foot placement controller (the core contribution of this work) and a stance-phase force controller integrated through a hybrid switching logic.

The proposed controller operates as shown in Fig. 4.5. It follows a hybrid control structure, switching between two modes depending on whether the robot is in the air (flight phase) or in contact with the ground (stance phase). If the robot is in the air,

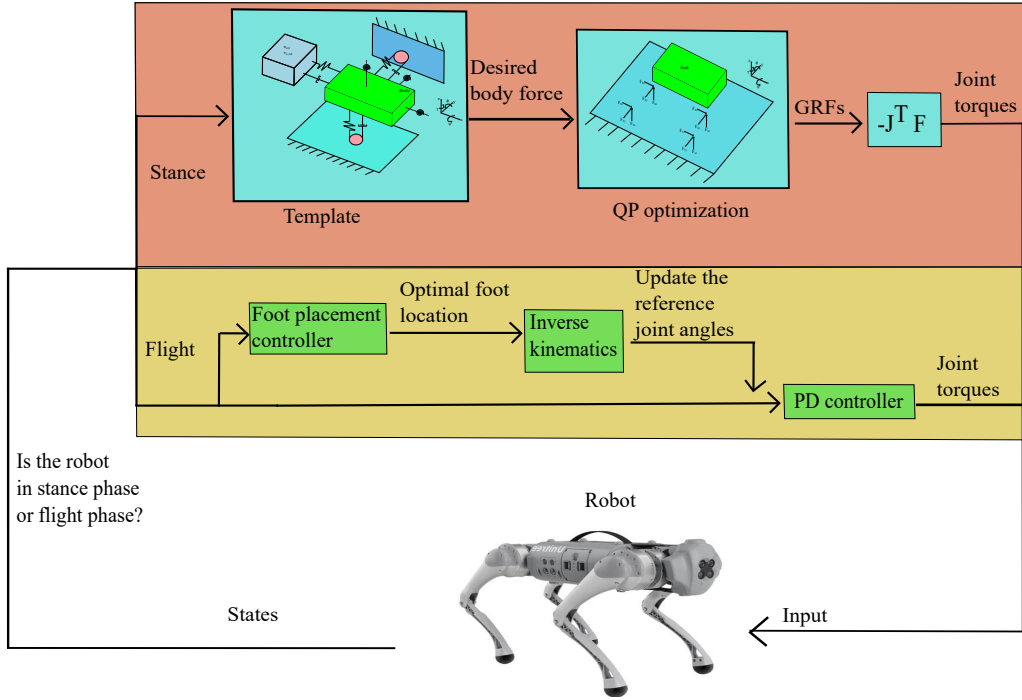


Figure 4.5: Overview of the hybrid controller. The controller switches between stance and flight modes based on the robot's state. In the stance phase, a template model and QP optimization generate desired body forces and joint torques. In the flight phase, the foot placement controller determines the optimal foot location, inverse kinematics updates the reference joint angles, and a PD controller computes the resulting joint torques.

the foot placement controller calculates the optimal foot locations based on the current body states. These optimal foot locations are then converted to reference joint angles using inverse kinematics. A PD (proportional-derivative) controller computes the joint torques based on these reference angles and sends them to the robot. During the flight phase, the foot placement controller runs in parallel with the PD controller. The PD controller operates at a higher bandwidth and tracks the last reference angles while the foot placement controller solves the optimization problem. Once the foot placement controller provides new values, the reference angles are updated accordingly. This hybrid setup allows the PD controller to handle intermediate steps while managing the computational complexity of the foot placement controller.

If the robot is on the ground, the stance controller calculates the desired body force based on the template using equation 4.1 then solves the quadratic programming prob-

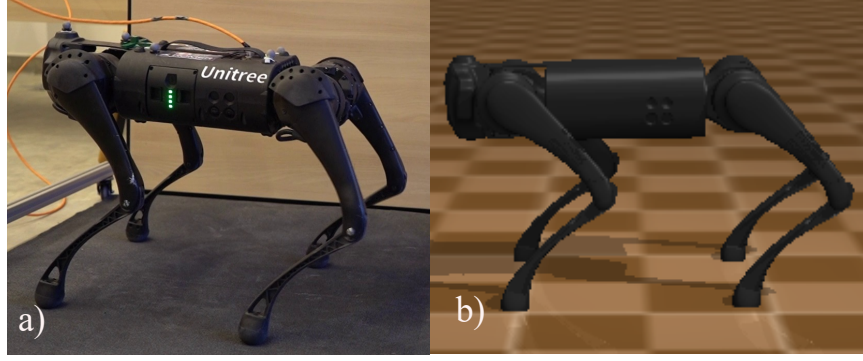


Figure 4.6: Unitree GO1 robot: (a) real robot, (b) MuJoCo simulation environment.

lem given in equation 4.6 to determine the optimal GRFs. These optimal GRFs are converted to joint torques using the equation $-J^T F$ and applied as inputs to the robot.

Notice that the robot alternates between two phases: stance and flight. During the flight phase, the controller adjusts the foot placement to ensure a stable touchdown, assuming simultaneous contact across all legs for simplicity.

4.4 Experimental Results

The experiments are done both in MuJoCo simulator and in the real robot. The Unitree GO1 robot is used for experiments given in Fig. 4.6. For solving quadratic programming problem given in equation 4.6, qpOASES library is used [63]. The qpOASES library is an open-source C++ software package for solving quadratic programming (QP) problems efficiently in real-time applications, particularly for model predictive control (MPC) and embedded optimization. For solving the non-linear optimization problem in foot placement controller, NLOPT library is utilized [78]. The NLOPT library is an open-source software package that offers a variety of algorithms for solving nonlinear optimization problems, supporting both unconstrained and constrained optimization, including nonlinear inequality and equality constraints, making it versatile for a wide range of applications. For solving the inverse kinematics root-finding equation, the GNU Scientific Library (GSL) is used [79], which is a numerical library that provides mathematical routines and algorithms for scientific computing.

In the experiments, the NMPC horizon t^* is set to 10 milliseconds. During the stance phase simulation, NMPC updates its state at 2-millisecond intervals and is continuously re-solved after each solution. The solver’s computation time is observed to be under 1 millisecond, ensuring real-time feasibility. The weight Q_z is set to 1×10^{-5} . The desired velocities, roll, and pitch are all set to zero. The desired x and y positions correspond to the geometric center of the legs at touchdown, while the desired yaw angle matches the yaw angle at touchdown. The desired z position is specified as 30 cm.

4.4.1 Simulation Experiments

In the simulation, the robot model is obtained from Google DeepMind’s website [62]. This model includes detailed information on the leg masses and inertias. The robot’s control method is switched from position control to torque control with torque limits. The control parameters for the robot are listed in Table 4.1. The linear axes are tuned to be critically damped, while the angular axes are tuned to be over-damped. The gains K_{pfa} , K_{dfa} , K_{pfh} , K_{dfh} , K_{pfk} , and K_{dfk} represent the proportional (P) and derivative (D) gains for the abduction/adduction, hip, and knee joints, respectively.

Table 4.1: Template parameters.

Parameter	Value	Unit	Parameter	Value	Unit
K_{px}	1000	N/m	K_{dx}	225.74	Ns/m
K_{py}	1000	N/m	K_{dy}	225.74	Ns/m
K_{pz}	2000	N/m	K_{dz}	319.25	Ns/m
$K_{p\psi}$	400	Nm	$K_{d\psi}$	200	Nms
$K_{p\phi}$	800	Nm	$K_{d\phi}$	400	Nms
$K_{p\gamma}$	300	Nm	$K_{d\gamma}$	100	Nms
K_{pfa}	100	Nm	K_{dfa}	1.5	Nms
K_{pfh}	150	Nm	K_{dfh}	2.25	Nms
K_{pfk}	100	Nm	K_{dfk}	0.5	Nms

To compare our results with the approach proposed in [45], we conducted equivalent

landing experiments using our NMPC-based controller. The outcomes are visualized in Fig. 4.7, which depicts the set of successful landing conditions across various drop heights.

In each experiment, the robot's body was maintained parallel to the ground, while the throwing angle was systematically varied in 30° increments. For every condition, we determined the maximum throwing velocity that still resulted in a stable landing. In the polar plot, radial distance represents velocity (m/s), and angular position denotes the throwing direction (degrees).

The results exhibit an approximate symmetry about the x-axis, which aligns with the robot's symmetric mass distribution along that axis. In contrast, the asymmetry observed along the y-axis is primarily attributed to uneven mass distribution introduced by leg configuration and joint orientation.

Compared to the VHSIP-based method in [45], our approach consistently achieves stable landings at significantly higher velocities across all tested drop heights. For instance, at a drop height of 0.5 m and 0° throwing angle, our controller achieves stable landings up to 3.6 m/s, compared to 1.75 m/s with the baseline, corresponding to a 105% improvement. At 1.0 m drop height, our method extends the maximum velocity to 5.0 m/s, whereas the baseline reaches 3.0 m/s, marking a 67% increase in stability under high-impact conditions.

These results demonstrate that our optimized foot placement strategy significantly improves landing stability, enabling the robot to handle more aggressive maneuvers across a wider range of throwing directions and drop heights. Overall, the proposed method outperforms state-of-the-art techniques in all evaluated scenarios.

To further analyze the landing performance illustrated in Fig.4.7, we selected representative trajectories corresponding to the 0° throwing direction at the maximum stable velocity for each drop height. These cases correspond to the outermost points in the polar plot. The time evolution of key state variables such as vertical position, forward velocity, lateral velocity, and vertical velocity is shown in Fig. 4.8. These plots offer insight into the landing dynamics and stability characteristics under high-velocity conditions.

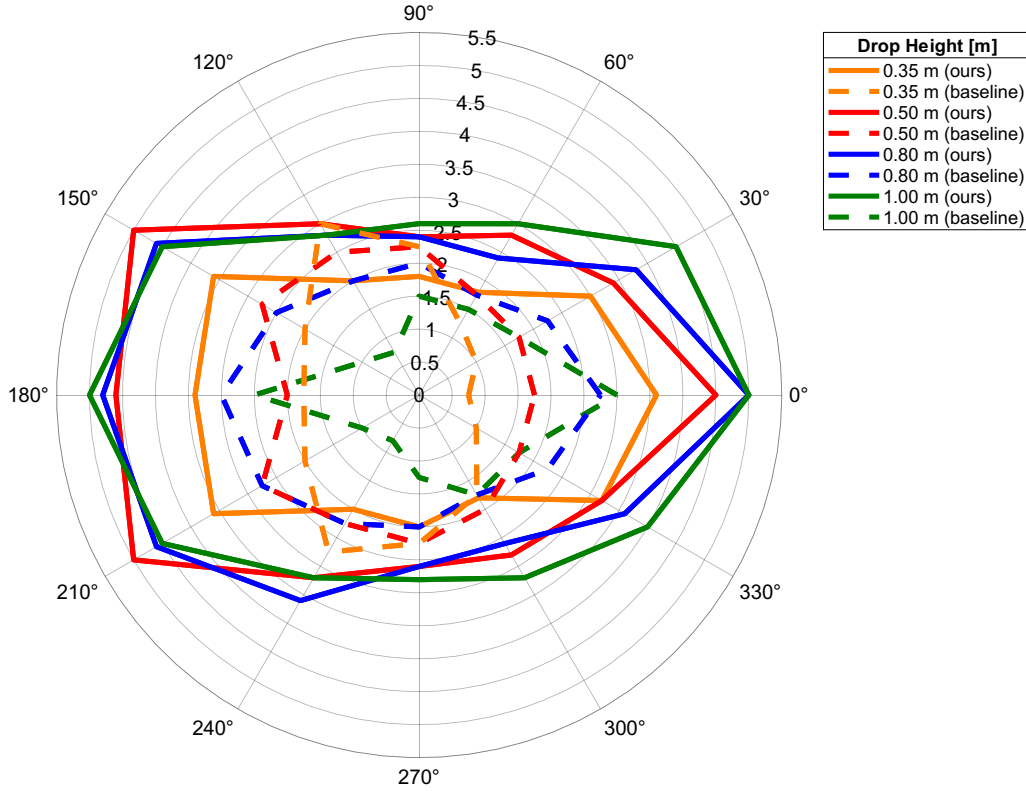


Figure 4.7: Polar plot of successful landing conditions across different drop heights. Radial distance indicates the maximum achievable throwing velocity (m/s), and angular position corresponds to the throwing direction (degrees). Solid lines represent the proposed controller, while dashed lines correspond to a baseline method [45].

In summary, the proposed controller outperforms existing methods by enabling stable landings at significantly higher linear velocities across a wide range of throwing directions and drop heights.

4.4.2 Real Robot Experiments

In real robot experiments, the GO1 robot is connected to a laptop via Ethernet. The laptop receives sensor data, computes the required joint torques, and sends them back to the robot. Since linear body states, such as height and lateral position, are not directly measured, they are estimated using kinematic data during the stance phase and an OptiTrack system during the flight phase. In contrast, angular body states are directly obtained from the gyroscope sensor. Foot force sensors detect ground contact

by comparing force readings to a predefined threshold. When the reading exceeds this threshold, the leg is considered to be in contact with the ground.

Fig. 4.9 shows the experiment conducted with the GO1 robot. The robot is thrown forward from a height of 66 cm and lands successfully. In the figure, the red dot marks the moment the robot is thrown, and the green dot marks the touchdown. The robot is in the air for approximately 20 milliseconds. The first plot shows the robot's horizontal displacement, with an average velocity of 0.66 m/s in the air and a small velocity of 0.01 m/s in the y-direction. The results show that the robot can successfully land from a height of 66 cm.

When controlling the real robot, the parameters are adjusted for more stable motion, resulting in less dynamic and agile behavior compared to the simulation. This adjustment is necessary to account for noise and uncertainties in the real system. Contact detection relies on the robot's foot force sensors, which are prone to bias and variability over time. To mitigate this, an adaptive filter is implemented that takes the first 500 readings while the robot is in flight and sets the last reading as the force threshold for contact detection. However, these sensor biases can introduce latency in detecting contact. For state estimation during the robot's flight phase, the OptiTrack system provides ground truth data but only at a frequency of 120 Hz. As it supplies only position and orientation data, velocities must be computed numerically, which adds noise. Additionally, unknown factors such as friction losses in the gearboxes and the mechanics of contact also affect performance.

Fig. 2.25 compares the simulation results with those from the real robot. In the figure, dashed lines represent simulation data, while solid lines represent real robot data. The controller parameters are the same for both cases. Due to latency in contact detection, the stance controller engages later than the actual stance of the real robot, causing the robot to approach the ground more closely. This results in the z value being lower than in the simulation. Additionally, velocities are estimated numerically, which introduces discrepancies between the real and calculated velocities. These factors account for the differences between the simulation and the real robot.

To summarize the real robot experiments the procedure is as follows. When the robot is in the air the state estimation is done by the OptiTrack system, the foot placement

controller adapts the foot positions from this data. Contact detection is done by the foot sensors which are built in the GO1 robot, and once they exceed a certain threshold, which is chosen adaptively, the state estimation switches to kinematic estimation and the controller switches to stance controller.

The results confirm that the robot can successfully land from a height of 66 cm, but differences from simulation arise due to contact detection latency, sensor noise, and mechanical uncertainties. The stance controller engages later than in simulation, causing a lower z-position at touchdown, while numerically estimated velocities introduce discrepancies.

The video for the experiment can be found at the following link.

<https://youtu.be/B5I4C7vEZuQ>

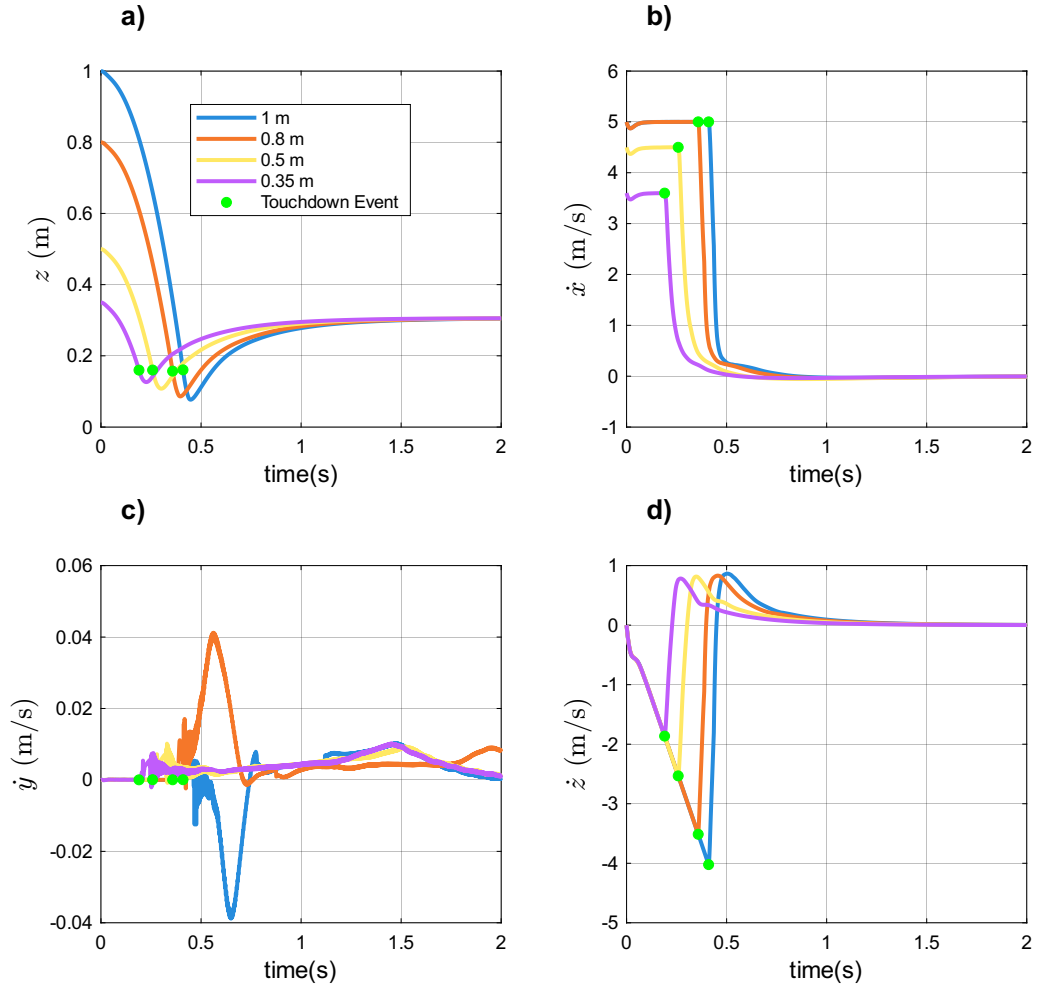


Figure 4.8: Time evolution of the robot's states during landing for different drop heights (1.00, 0.80, 0.50, and 0.35 m) at a 0° throwing direction, with each trajectory executed at the maximum velocity that results in a stable landing. Subplots show the vertical position (a), forward velocity (b), lateral velocity (c), and vertical velocity (d) of the center of mass. Green markers indicate the estimated touchdown moment for each trajectory.

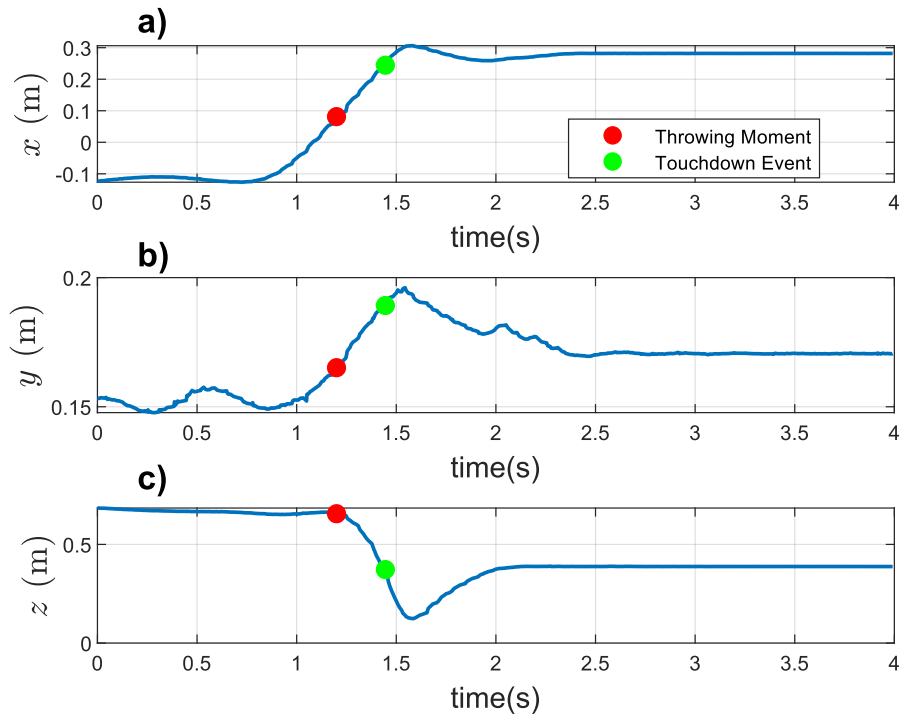


Figure 4.9: Landing states of the GO1 robot. (a) Horizontal position, (b) lateral position, (c) vertical position during the landing maneuver. The red marker denotes the throwing moment, and the green marker indicates the touchdown event.

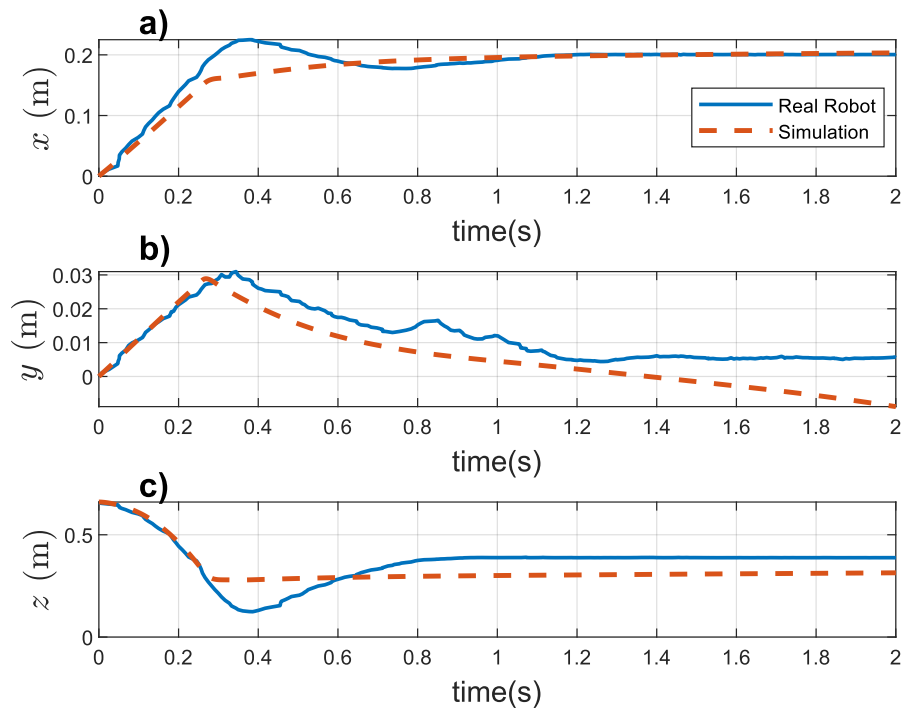


Figure 4.10: Comparison of landing behavior between the GO1 robot and simulation. (a) Horizontal position, (b) lateral position, (c) vertical position during landing, showing the agreement between real-robot measurements and the simulated model.

CHAPTER 5

CONCLUSIONS

This thesis presents a set of control strategies for achieving dynamic and versatile locomotion behaviors in quadrupedal robots by leveraging template-based models and predictive control techniques. The work focuses on three core locomotion tasks: trotting, pronking, and landing. Each control framework builds on simplified mechanical models that abstract the essential dynamics of legged motion and is validated through a combination of simulation and real-world experiments.

In the first part of the thesis, we introduced a novel extension of the clock torque-actuated spring-loaded inverted pendulum template to three-dimensional quadrupedal locomotion. By incorporating projectile motion into the model, we enabled flying trot behavior, allowing smooth transitions between walking trot, regular trot, and flying trot without requiring controller switching. The integration of a whole-body controller further enhanced the stability and performance of the overall system. The proposed framework was rigorously tested in MuJoCo simulations, demonstrating robustness in handling gait transitions and external disturbances. Hardware experiments conducted on the Unitree GO1 robot confirmed the controller’s ability to achieve stable and seamless gait transitions in real-world conditions.

The second part of the thesis focused on the generation of pronking motion through a combination of template-based control and model predictive control. A three-dimensional spring-damper template was used to produce reference trajectories, which were tracked using a time-varying MPC that captures the nonlinear dynamics of the system. Additionally, a nonlinear MPC-based footstep planner was developed based on approximate analytical solutions of the template equations. The proposed controller was first evaluated in simulation and then implemented on a physical quadruped

robot. The experimental results demonstrated stable and repeatable pronking behavior, confirming that the controller reliably guides the system to the desired gait cycle even under varying initial conditions. These findings emphasize the potential of combining TBC and MPC for advanced gait synthesis in legged robots.

In the final part, we addressed the challenge of landing control by proposing an optimization-based foot placement controller formulated as a nonlinear model predictive control problem. The stance phase was modeled using a spring-damper template, and the controller aimed to minimize the discrepancy between the moments predicted by the template and those estimated from ground reaction forces during touchdown. The method was validated in simulation and supported by preliminary real-robot experiments. While simulation results showed high performance and stability, some differences were observed in hardware due to simplifying assumptions such as equal ground reaction forces and simultaneous foot contact. To address these limitations, future work will explore more realistic modeling of GRF distributions and adaptive contact scheduling to improve performance under angular disturbances.

In summary, this thesis contributes template-based and optimization-driven control approaches for trotting, pronking, and landing in quadrupedal robots. Across all behaviors, the proposed methods were validated both in simulation and on hardware, demonstrating their feasibility and robustness.

Future directions for the landing controller include the development of adaptive contact scheduling policies that enable sequential or selective leg touchdowns based on the robot's estimated body orientation and angular velocity. Moreover, relaxing the assumption of equal ground reaction forces by incorporating dynamic feedback into the NMPC formulation could lead to more robust and responsive landings. Extending the current approach to handle vertical drops combined with lateral motion would also allow the controller to address more realistic locomotion scenarios, such as jumping between platforms or recovering from unexpected perturbations.

In addition, the proposed controllers have been validated primarily on flat terrain. By integrating a more robust contact detection algorithm, these methods could be extended to handle uneven or irregular surfaces, further enhancing the robot's adaptability in real-world environments.

REFERENCES

- [1]R. J. Full and D. E. Koditschek, “Templates and anchors: Neuromechanical hypotheses of legged locomotion on land”, *Journal of experimental biology*, vol. 202, no. 23, pp. 3325–3332, 1999.
- [2]M. M. Ankaralı and U. Saranlı, “Control of underactuated planar pronking through an embedded spring-mass hopper template”, *Autonomous Robots*, vol. 30, pp. 217–231, 2011.
- [3]Z. Jiang, M. Li, and W. Guo, “Running control of a quadruped robot in trotting gait”, in *2011 IEEE 5th International Conference on Robotics, Automation and Mechatronics (RAM)*, IEEE, 2011, pp. 172–177.
- [4]G. Zhang, Y. Li, and S. Ma, “Trotting and pacing locomotion of a position-controlled quadruped robot”, in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2021, pp. 7456–7463.
- [5]L. Zhang and Y. Tan, “Trotting motion of the quadruped model with two spinal joints and its dynamics features”, *Journal of Robotics*, vol. 2020, no. 1, p. 3 156 540, 2020. DOI: 10.1155/2020/3156540
- [6]G. D. ER and M. M. Ankaralı, “Adaptive control of underactuated planar pronking”, in *2022 9th IEEE RAS/EMBS International Conference for Biomedical Robotics and Biomechatronics (BioRob)*, IEEE, 2022, pp. 1–6.
- [7]U. Saranlı and D. E. Koditschek, “Template based control of hexapedal running”, in *2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422)*, IEEE, vol. 1, 2003, pp. 1374–1379.
- [8]B. Han, H. Yi, Z. Xu, X. Yang, and X. Luo, “3d-slip model based dynamic stability strategy for legged robots with impact disturbance rejection”, *Scientific Reports*, vol. 12, no. 1, p. 5892, 2022.
- [9]M. H. Raibert, “Running with symmetry”, *The International journal of robotics research*, vol. 5, no. 4, pp. 3–19, 1986.

- [10]J. Seipel and P. Holmes, “A simple model for clock-actuated legged locomotion”, *Regular and chaotic dynamics*, vol. 12, pp. 502–520, 2007.
- [11]S. H. Tamaddoni, F. Jafari, A. Meghdari, and S. Sohrabpour, “Biped hopping control bazsed on spring loaded inverted pendulum model”, *International Journal of Humanoid Robotics*, vol. 7, no. 02, pp. 263–280, 2010.
- [12]I. Poulakakis and J. W. Grizzle, “The spring loaded inverted pendulum as the hybrid zero dynamics of an asymmetric hopper”, *IEEE Transactions on Automatic Control*, vol. 54, no. 8, pp. 1779–1793, 2009.
- [13]M. M. Pelit, J. Chang, R. Takano, and M. Yamakita, “Bipedal walking based on improved spring loaded inverted pendulum model with swing leg (slip-sl)”, in *2020 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, IEEE, 2020, pp. 72–77.
- [14]J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, “Dynamic locomotion in the mit cheetah 3 through convex model-predictive control”, in *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, IEEE, 2018, pp. 1–9.
- [15]M. Bjelonic, R. Grandia, O. Harley, C. Galliard, S. Zimmermann, and M. Hutter, “Whole-body mpc and online gait sequence generation for wheeled-legged robots”, in *2021 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, IEEE, 2021, pp. 8388–8395.
- [16]O. Villarreal, V. Barasuol, P. M. Wensing, D. G. Caldwell, and C. Semini, “Mpc-based controller with terrain insight for dynamic legged locomotion”, in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2020, pp. 2436–2442.
- [17]D. Kim, J. Di Carlo, B. Katz, G. Bledt, and S. Kim, “Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control”, *arXiv preprint arXiv:1909.06586*, 2019.
- [18]D. Kim, S. J. Jorgensen, J. Lee, J. Ahn, J. Luo, and L. Sentis, “Dynamic locomotion for passive-ankle biped robots and humanoids using whole-body locomotion control”, *The International Journal of Robotics Research*, vol. 39, no. 8, pp. 936–956, 2020.
- [19]H. M. Herr and T. A. McMahon, “A trotting horse model”, *The International Journal of Robotics Research*, vol. 19, no. 6, pp. 566–581, 2000.

- [20]Z. Shen, Y. Ma, and T. Tsuchiya, “Feedback linearization-based tracking control of a tilt-rotor with cat-trot gait plan”, *International Journal of Advanced Robotic Systems*, vol. 19, no. 4, p. 17 298 806 221 109 360, 2022.
- [21]T. Fukui, S. Matsukawa, Y. Habu, and Y. Fukuoka, “Gait transition from pacing by a quadrupedal simulated model and robot with phase modulation by vestibular feedback”, *Robotics*, vol. 11, no. 1, p. 3, 2021.
- [22]E. W. McClain, A. S. Filsoofi, and S. Meek, “Stability and control of a pacing quadruped using stabilizing switching design”, *IEEE robotics and automation letters*, vol. 6, no. 4, pp. 8466–8473, 2021.
- [23]O. K. Adak and K. Erbatur, “Bound gait reference generation of a quadruped robot via contact force planning”, *International Journal of Mechanical Engineering and Robotics Research*, vol. 11, no. 3, pp. 129–137, 2022.
- [24]C. Lei, C. Dongliang, and D. Wei, “Effects of spinal structure on quadruped bounding gait”, *Robotica*, vol. 40, no. 11, pp. 3911–3929, 2022.
- [25]X. He et al., “Running gait and control of quadruped robot based on slip model”, *Biomimetics*, vol. 9, no. 1, 2024.
- [26]A. Fukuhara, M. Gunji, Y. Masuda, K. Tadakuma, and A. Ishiguro, “Simulation study on galloping quadruped robot with flexible shoulder hammock structure”, in *Proc. of the 9.5 th Int. Symp. on Adaptive Motion of Animals and Machines (AMAM2021)*, 2021, pp. 32–33.
- [27]Y. Yang, T. Zhang, E. Coumans, J. Tan, and B. Boots, “Fast and efficient locomotion via learned gait transitions”, in *Conference on robot learning*, PMLR, 2022, pp. 773–783.
- [28]C. Gehring, S. Coros, M. Hutter, M. Bloesch, M. A. Hoepflinger, and R. Siegwart, “Control of dynamic gaits for a quadrupedal robot”, in *2013 IEEE international conference on Robotics and automation*, IEEE, 2013, pp. 3287–3292.
- [29]D. Owaki and A. Ishiguro, “A quadruped robot exhibiting spontaneous gait transitions from walking to trotting to galloping”, *Scientific reports*, vol. 7, no. 1, p. 277, 2017.
- [30]I. M. Koo et al., “Biologically inspired gait transition control for a quadruped walking robot”, *Autonomous Robots*, vol. 39, pp. 169–182, 2015.
- [31]J. Humphreys, J. Li, Y. Wan, H. Gao, and C. Zhou, “Bio-inspired gait transitions for quadruped locomotion”, *IEEE Robotics and Automation Letters*, 2023.

- [32]P. Zeng, L. Yuan, Z. Dong, Z. Zhang, and H. An, “Overall locomotion control of a quadruped robot with running trot gait”, in *2021 40th Chinese Control Conference (CCC)*, IEEE, 2021, pp. 4032–4038.
- [33]T. M. Caro, “Ungulate antipredator behaviour: Preliminary and comparative data from african bovids”, *Behaviour*, pp. 189–228, 1994.
- [34]C. D. Fitzgibbon and J. H. Fanshawe, “Stotting in thomson’s gazelles: An honest signal of condition”, *Behavioral Ecology and Sociobiology*, vol. 23, pp. 69–74, 1988.
- [35]M. M. Ankaralı, U. Saranlı, and A. Saranlı, “Control of underactuated planar hexapedal pronking through a dynamically embedded slip monopod”, in *2010 IEEE International Conference on Robotics and Automation*, IEEE, 2010, pp. 4721–4727.
- [36]U. Saranlı, M. Buehler, and D. E. Koditschek, “Rhex: A simple and highly mobile hexapod robot”, *The International Journal of Robotics Research*, vol. 20, no. 7, pp. 616–631, 2001.
- [37]J. Pratt, C.-M. Chew, A. Torres, P. Dilworth, and G. Pratt, “Virtual model control: An intuitive approach for bipedal locomotion”, *The International Journal of Robotics Research*, vol. 20, no. 2, pp. 129–143, 2001.
- [38]M. H. Raibert, *Legged robots that balance*. MIT press, 1986.
- [39]J. Yu, D. Hong, and M. Haberland, “Energetic efficiency of a compositional controller on a monopod with an articulated leg and slip dynamics”, in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2018, pp. 2221–2228.
- [40]F. Risbourg, T. Corbères, P.-A. Léziart, T. Flayols, N. Mansard, and S. Tonneau, “Real-time footstep planning and control of the solo quadruped robot in 3d environments”, in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2022, pp. 12 950–12 956.
- [41]U. Saranlı, Ö. Arslan, M. M. Ankaralı, and Ö. Morgül, “Approximate analytic solutions to non-symmetric stance trajectories of the passive spring-loaded inverted pendulum with damping”, *Nonlinear Dynamics*, vol. 62, pp. 729–742, 2010.
- [42]J. Wang et al., “Automatic gait pattern selection for legged robots”, in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2020, pp. 3990–3997.

- [43]A. M. Johnson, T. Libby, E. Chang-Siu, M. Tomizuka, R. J. Full, and D. E. Koditschek, “Tail assisted dynamic self righting”, in *Adaptive mobile robotics*, World Scientific, 2012, pp. 611–620.
- [44]Y. Tang, J. An, X. Chu, S. Wang, C. Y. Wong, and K. S. Au, “Towards safe landing of falling quadruped robots using a 3-dof morphable inertial tail”, in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2023, pp. 1141–1147.
- [45]F. Roscia, M. Focchi, A. Del Prete, D. G. Caldwell, and C. Semini, “Reactive landing controller for quadruped robots”, *IEEE Robotics and Automation Letters*, 2023.
- [46]S. H. Jeon, S. Kim, and D. Kim, “Online optimal landing control of the mit mini cheetah”, in *2022 International Conference on Robotics and Automation (ICRA)*, IEEE, 2022, pp. 178–184.
- [47]J. Ding, V. Atanassov, E. Panichi, J. Kober, and C. Della Santina, “Robust quadrupedal jumping with impact-aware landing: Exploiting parallel elasticity”, *IEEE Transactions on Robotics*, 2024.
- [48]J. Qi, H. Gao, H. Yu, M. Huo, W. Feng, and Z. Deng, “Integrated attitude and landing control for quadruped robots in asteroid landing mission scenarios using reinforcement learning”, *Acta Astronautica*, vol. 204, pp. 599–610, 2023.
- [49]N. Rudin, H. Kolvenbach, V. Tsounis, and M. Hutter, “Cat-like jumping and landing of legged robots in low gravity using deep reinforcement learning”, *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 317–328, 2021.
- [50]M. Ankarali, O. Arslan, and U. Saranlı, “An analytical solution to the stance dynamics of passive spring-loaded inverted pendulum with damping”, in *Mobile Robotics: Solutions and Challenges*, World Scientific, 2010, pp. 693–700.
- [51]M. Sombolestan, Y. Chen, and Q. Nguyen, “Adaptive force-based control for legged robots”, in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2021, pp. 7440–7447.
- [52]Q. Nguyen, M. J. Powell, B. Katz, J. Di Carlo, and S. Kim, “Optimized jumping on the mit cheetah 3 robot”, in *2019 International Conference on Robotics and Automation (ICRA)*, IEEE, 2019, pp. 7448–7454.
- [53]M. Sombolestan and Q. Nguyen, “Adaptive force-based control of dynamic legged locomotion over uneven terrain”, *IEEE Transactions on Robotics*, 2024.

- [54]S. Xu, L. Zhu, H.-T. Zhang, and C. P. Ho, “Robust convex model predictive control for quadruped locomotion under uncertainties”, *IEEE Transactions on Robotics*, 2023.
- [55]M. Bloesch et al., “State estimation for legged robots: Consistent fusion of leg kinematics and imu”, 2013.
- [56]G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, “Mit cheetah 3: Design and control of a robust, dynamic quadruped robot”, in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2018, pp. 2245–2252.
- [57]E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control”, in *2012 IEEE/RSJ international conference on intelligent robots and systems*, IEEE, 2012, pp. 5026–5033.
- [58]Y. Tassa, T. Erez, and E. Todorov, “Synthesis and stabilization of complex behaviors through online trajectory optimization”, in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2012, pp. 4906–4913.
- [59]Z. Xie, P. Gergondet, F. Kanehiro, et al., “Learning bipedal walking for humanoids with current feedback”, *IEEE Access*, vol. 11, pp. 82 013–82 023, 2023.
- [60]A. Mastrogeorgiou, A. Papatheodorou, K. Koutsoukis, and E. Papadopoulos, “Learning energy-efficient trotting for legged robots”, in *Climbing and Walking Robots Conference*, Springer, 2022, pp. 204–215.
- [61]Z. Li et al., “Reinforcement learning for robust parameterized locomotion control of bipedal robots”, in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2021, pp. 2811–2817.
- [62]Google DeepMind, *MuJoCo Menagerie GO1 Model*, https://github.com/google-deepmind/mujoco_menagerie/blob/main/unitree_go1/go1.xml, Accessed: September 1, 2024, 2024.
- [63]H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl, “Qpoases: A parametric active-set algorithm for quadratic programming”, *Mathematical Programming Computation*, vol. 6, pp. 327–363, 2014.
- [64]F. Toso, P. G. Carlet, A. Favato, and S. Bolognani, “On-line continuous control set mpc for pmsm drives current loops at high sampling rate using qpoases”, in *2019 IEEE Energy Conversion Congress and Exposition (ECCE)*, IEEE, 2019, pp. 6615–6620.

- [65]Y. Jiang, J. Oravec, B. Houska, and M. Kvasnica, “Parallel mpc for linear systems with input constraints”, *IEEE Transactions on Automatic Control*, vol. 66, no. 7, pp. 3401–3408, 2020.
- [66]A. Zheng and S. S. Narayanan, “Optimal control for quadruped locomotion using ltv mpc”, *arXiv preprint arXiv:2212.05154*, 2022.
- [67]A. Schperberg, S. Di Cairano, and M. Menner, “Auto-tuning of controller and on-line trajectory planner for legged robots”, *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7802–7809, 2022.
- [68]M. Galassi, *GNU Scientific Library: Reference Manual* (GNU manual). Network Theory, 2009, ISBN: 9780954612078. [Online]. Available: <https://books.google.com.tr/books?id=CUuNPgAACAAJ>
- [69]S. Bochkanov, *ALGLIB*, <http://www.alglib.net>, Accessed: October 2, 2024, 2024.
- [70]G. Guennebaud, B. Jacob, et al., *Eigen v3*, <http://eigen.tuxfamily.org>, 2010.
- [71]G. Secer and U. Saranlı, “Control of planar spring–mass running through virtual tuning of radial leg damping”, *IEEE Transactions on Robotics*, vol. 34, no. 5, pp. 1370–1383, 2018.
- [72]M. Hutter, C. D. Remy, M. A. Höpfinger, and R. Siegwart, “Slip running with an articulated robotic leg”, in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2010, pp. 4934–4939.
- [73]D. Youm, H. Jung, H. Kim, J. Hwangbo, H.-W. Park, and S. Ha, “Imitating and finetuning model predictive control for robust and symmetric quadrupedal locomotion”, *IEEE Robotics and Automation Letters*, 2023.
- [74]J. Hooks et al., “Alphred: A multi-modal operations quadruped robot for package delivery applications”, *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5409–5416, 2020.
- [75]Z. Xie, X. Da, B. Babich, A. Garg, and M. v. de Panne, “Glide: Generalizable quadrupedal locomotion in diverse environments with a centroidal model”, in *International Workshop on the Algorithmic Foundations of Robotics*, Springer, 2022, pp. 523–539.
- [76]S. G. Johnson, *The NLOpt nonlinear-optimization package*, <https://github.com/stevengj/nlopt>, 2007.

- [77]M. Camurri et al., “Probabilistic contact estimation and impact detection for state estimation of quadruped robots”, *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 1023–1030, 2017.
- [78]S. G. Johnson and J. Schueller, “Nlopt: Nonlinear optimization library”, *Astrophysics Source Code Library*, ascl–2111, 2021.
- [79]M. Galassi et al., *GNU scientific library*. Network Theory Limited Godalming, 2002.

Appendix A

EQUATIONS OF MOTION OF A QUADRUPEDAL ROBOT

To derive the equations of motion for the quadruped robot, the kinematic equations must first be formulated. This entire process is carried out in MATLAB using the Symbolic Math Toolbox. The derivation begins with the kinematics of the robot's body, where the Euler 321 (ZYX) angle convention is adopted. The corresponding rotation matrices are provided in equations A.1, A.2, and A.3. The standard basis vectors used throughout the derivation are defined in equation A.4.

$$\mathbf{R}_1(\Psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\Psi) & -\sin(\Psi) \\ 0 & \sin(\Psi) & \cos(\Psi) \end{bmatrix} \quad (\text{A.1})$$

$$\mathbf{R}_2(\Psi) = \begin{bmatrix} \cos(\Psi) & 0 & \sin(\Psi) \\ 0 & 1 & 0 \\ -\sin(\Psi) & 0 & \cos(\Psi) \end{bmatrix} \quad (\text{A.2})$$

$$\mathbf{R}_3(\Psi) = \begin{bmatrix} \cos(\Psi) & -\sin(\Psi) & 0 \\ \sin(\Psi) & \cos(\Psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.3})$$

$$\begin{aligned} \mathbf{u}_1 &= \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^\top \\ \mathbf{u}_2 &= \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}^\top \\ \mathbf{u}_3 &= \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^\top \end{aligned} \quad (\text{A.4})$$

$$\mathbf{R}_{body} = \mathbf{R}_3(\gamma)\mathbf{R}_2(\phi)\mathbf{R}_1(\psi) \quad (\text{A.5})$$

The position of the robot's body is defined in equation A.6, while the linear and angular velocity expressions are presented in equation A.7.

$$\mathbf{p}_{body} = [x \quad y \quad z]^\top \quad (\text{A.6})$$

$$\begin{aligned} \mathbf{v}_{body} &= [\dot{x} \quad \dot{y} \quad \dot{z}]^\top \\ \boldsymbol{\omega}_{body} &= \dot{\gamma} \mathbf{u}_3 + \dot{\phi} \mathbf{R}_3(\gamma) \mathbf{u}_2 + \dot{\psi} \mathbf{R}_3(\gamma) \mathbf{R}_2(\phi) \mathbf{u}_1 \end{aligned} \quad (\text{A.7})$$

To derive the equations of motion, it is necessary to determine the positions and orientations of each joint, as well as the foot positions. The procedure for a single leg is outlined below. The generalized coordinates of the robot are defined in equation A.8. For the i -th leg, the joint angles are denoted by θ_{ij} , where $j = 0$ corresponds to the abduction/adduction joint, $j = 1$ to the hip joint, and $j = 2$ to the knee joint.

$$\mathbf{q} = [\psi \quad \phi \quad \gamma \quad x \quad y \quad z \quad \theta_{10} \quad \theta_{11} \quad \theta_{12} \quad \theta_{20} \quad \theta_{21} \quad \theta_{22} \quad \theta_{30} \quad \theta_{31} \quad \theta_{32} \quad \theta_{40} \quad \theta_{41} \quad \theta_{42}]^\top \quad (\text{A.8})$$

The positions of the joints and the foot for the first leg are given in equations A.9, A.10, A.11, and A.12. The physical parameters of the robot are illustrated in Figure A.1. It is assumed that the robot's center of mass coincides with its geometric center.

$$\mathbf{p}_{abad} = \mathbf{p}_{body} + l_1 \mathbf{R}_{body} \mathbf{u}_1 - l_2 \mathbf{R}_{body} \mathbf{u}_2 \quad (\text{A.9})$$

$$\mathbf{p}_{hip} = \mathbf{p}_{abad} - b_1 \mathbf{R}_{body} \mathbf{R}_1(\theta_{10}) \mathbf{u}_2 \quad (\text{A.10})$$

$$\mathbf{p}_{knee} = \mathbf{p}_{hip} + a_1 \mathbf{R}_{body} \mathbf{R}_1(\theta_{10}) \mathbf{R}_2(\theta_{11}) \mathbf{u}_1 \quad (\text{A.11})$$

$$\mathbf{p}_{foot} = \mathbf{p}_{knee} + a_2 \mathbf{R}_{body} \mathbf{R}_1(\theta_{10}) \mathbf{R}_2(\theta_{11}) \mathbf{R}_2(\theta_{12}) \mathbf{u}_1 \quad (\text{A.12})$$

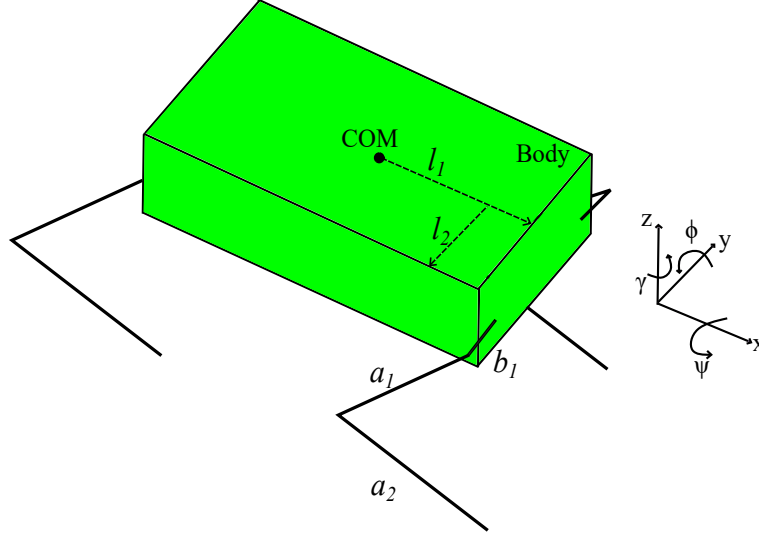


Figure A.1: Robot body parameters. The geometric properties of the robot body are shown, including the center of mass (COM), body dimensions l_1 and l_2 , and hip locations given by a_1 , a_2 , and b_1 . The body-fixed coordinate frame is also illustrated.

The linear velocities are obtained by taking the time derivative of the corresponding position vectors. The angular velocities for each joint are presented in equations A.13, A.14, and A.15.

$$\omega_{abad} = \omega_{body} + \dot{\theta}_{10} \mathbf{R}_{body} \mathbf{u}_1 \quad (\text{A.13})$$

$$\omega_{hip} = \omega_{abad} + \dot{\theta}_{11} \mathbf{R}_{body} \mathbf{R}_1(\theta_{10}) \mathbf{u}_2 \quad (\text{A.14})$$

$$\omega_{knee} = \omega_{hip} + \dot{\theta}_{12} \mathbf{R}_{body} \mathbf{R}_1(\theta_{10}) \mathbf{R}_2(\theta_{11}) \mathbf{u}_2 \quad (\text{A.15})$$

Subsequently, the contact Jacobians are computed using MATLAB's `jacobian` function. Once the Jacobians are obtained, the kinetic and potential energies of the system are calculated. To derive the equations of motion, Lagrangian mechanics is applied as shown in equation A.16, where D_k represents the dissipation term, K is the kinetic energy, U is the potential energy, F denotes the external force and moment vector (force wrench), Q_k is the actuation vector, q_k are the generalized coordinates,

and e_k refers to the standard basis vector. All expressions are computed symbolically within MATLAB.

$$\dot{P}_k - S_k + G_k = Q_k - D_k + T_k \quad (\text{A.16})$$

Where;

$$P_k = \frac{\partial K}{\partial \dot{q}_k} \quad S_k = \frac{\partial K}{\partial q_k} \quad G_k = \frac{\partial U}{\partial q_k} \quad T_k = e_k^\top J^\top F$$

After the equations of motion are obtained, they are manipulated to fit the standard form given in equation A.17. In this equation, M denotes the mass matrix, B represents the Coriolis force vector, and G corresponds to the gravitational force vector. The mass matrix M is computed by taking the Jacobian of the equations of motion with respect to $\ddot{q}$. The combined Coriolis and gravitational force vector, $B + G$, is obtained by substituting zeros on the left-hand side of the equations of motion and evaluating the remaining terms.

$$M(q)\ddot{q} + B(q, \dot{q}) + G(q) = \tau + J(q)^\top F \quad (\text{A.17})$$